PRISM

Professional Remote Infrastructure & Services Management

Easy Infrastructure and Account Management System

Arsitektur Sistem, Technology Stack, Alur Kerja, dan Spesifikasi Service

Dokumen Teknis Lengkap — Versi 4.2

Maret 2026

Klasifikasi: Internal — Dokumen Teknis

1. Gambaran Umum Arsitektur PRISM 3
1.1 Komponen Utama 3
2. Technology Stack 5
2.1 Ringkasan Stack 5
2.2 Frontend: React + Ant Design 6
Arsitektur Frontend 6
Pemetaan Komponen Ant Design ke Fitur PRISM 6
2.3 Backend (Hub): Go 7
Arsitektur Backend 7
Modul Internal Hub 8
WebSocket Message Compression 9
2.4 Agent: Go 9
Arsitektur Agent 9
Service Plugin Agent 10
2.5 Database: SQLite 11
Konfigurasi SQLite untuk Production 11
Strategi Index Telemetry 12
Skema Tabel Utama 12
Strategi Migrasi Database 13
2.6 Cache: Valkey 14
Strategi Caching 14
Pub/Sub untuk Real-time Events 14
Perencanaan Cluster Mode Valkey 15
2.7 Data Flow: Bagaimana Komponen Berinteraksi 15
Read Path (Admin membuka dashboard) 15
Write Path (Admin mengirim command ke service) 15
Telemetry Path (Agent mengirim data periodik) 16
3. Alur Kerja PRISM 17
3.1 Fase 1: Agent Melapor ke Hub (Otomatis) 17
3.2 Fase 2: Admin Melihat dan Meng-approve Agent 18
Pending Agent di Frontend 18
Proses Approval 18
3.3 Fase 3: Server Card Tampil 19
Anatomi Server Card 19
Kategori Service 19
3.4 Fase 4: Modal Konfigurasi Service 19
Struktur Umum Modal 20

4. Manajemen Akun 21
4.1 Dua Jenis Akun dalam PRISM 21
Management Credentials (Akun Manajemen) 21
Application Accounts (Akun Aplikasi) 21
4.2 Management Credentials 21
Skema Tabel management_credentials 21
Aturan Management Credentials 22
4.3 Konsep Project sebagai Resource Group 22
Contoh: Anatomi Project "E-Commerce Platform" 23
4.4 Kategorisasi Resource: Accounts dan Processes 24
Application Accounts (Akun Aplikasi) 24
Processes (Proses Custom) 24
Contoh Independent Processes 24
4.5 Menu Projects 25
Halaman Project List 25
Halaman Project Detail 26
Project Lifecycle 27
4.6 Menu Accounts 27
Halaman Account List 27
Fitur Cross-Reference 28
4.7 Interaksi Projects dan Accounts dengan Service Modal 28
5. Custom Services: Process Supervisors dan Container Runtime 30
5.1 Jenis Process Supervisor yang Didukung 30
5.2 Bagaimana Agent Mendeteksi Process Supervisors 30
5.3 Container Runtime: Docker dan Podman 31
Container sebagai Resource Project 31
5.4 Tampilan di Frontend 32
6. Arsitektur Keamanan 33
6.1 Lapisan Keamanan 33
6.2 Hierarki Credential 33
6.3 Manajemen Sertifikat 34
Certificate Renewal Automation 34
CRL atau OCSP 34
CA Key Rotation 34
Certificate Monitoring Dashboard 34
7. Model RBAC Granular dengan Resource-Level Access 35
7.1 Latar Belakang & Kebutuhan 35

7.2 Desain yang Direkomendasikan 36
7.2.1 Model Permission 36
7.2.2 Database Schema Baru 36
7.2.3 Contoh Policy 37
7.3 Langkah Implementasi 37
7.4 Prioritas & Estimasi 38
8. Enkripsi Database SQLite at Rest dengan SQLCipher 39
8.1 Latar Belakang & Kebutuhan 39
8.2 Desain yang Direkomendasikan 39
8.2.1 Konfigurasi SQLCipher 39
8.2.2 Key Management 40
8.2.3 Migration Path 40
8.3 Langkah Implementasi 40
8.4 Prioritas & Estimasi 41
9. Secrets & Credential Management Terpusat 42
9.1 Latar Belakang & Kebutuhan 42
9.2 Desain yang Direkomendasikan 43
9.2.1 Opsi Solusi 43
9.2.2 Rekomendasi Utama 43
9.2.3 Secrets Inventory 43
9.2.4 Integrasi Pattern 44
9.3 Langkah Implementasi 44
9.4 Prioritas & Estimasi 44
10. Protokol Komunikasi 45
10.1 WebSocket Message Types 45
10.2 Resilience 45
10.3 Rate Limiting dan Throttling 45
10.4 API Versioning dan Dokumentasi 46
11. Deployment Topology 47
11.1 Prinsip: Agent-Initiated Outbound Connection 47
11.2 Skenario Deployment 47
Skenario 1: Single LAN (Sederhana) 47
Skenario 2: Agent di LAN Belakang NAT 48
Skenario 3: Multi-Site / Branch Office 48
Skenario 4: Hybrid Cloud 48
12. Zero Trust Network dan Secure Tunnel 50
12.1 Prinsip Zero Trust di PRISM 50

12.2 Kapan Memerlukan Secure Tunnel? 50
12.3 Opsi 1: Cloudflare Tunnel (cloudflared) 51
Arsitektur dengan Cloudflare Tunnel 51
Keuntungan Cloudflare Tunnel 51
Konfigurasi Cloudflare Tunnel untuk PRISM 51
12.4 Opsi 2: WireGuard VPN 52
Arsitektur dengan WireGuard 52
Model Deployment WireGuard 52
Keuntungan WireGuard 52
Konfigurasi WireGuard untuk PRISM 53
12.5 Perbandingan: Cloudflare Tunnel vs WireGuard 53
12.6 Rekomendasi Pemilihan 54
13. Observability dan Monitoring Internal 55
13.1 Hub Health Metrics 55
13.2 Agent Self-Reporting 55
13.3 Structured Logging Standard 55
13.4 Integrasi OpenTelemetry 55
14. Health Check Endpoints (Liveness, Readiness, Startup) 56
14.1 Latar Belakang & Kebutuhan 56
14.2 Desain yang Direkomendasikan 56
14.2.1 Hub Endpoints 56
14.2.2 Agent Endpoints 56
14.2.3 Response Format 57
14.2.4 Concurrent Dependency Checks 57
14.3 Langkah Implementasi 57
14.4 Prioritas & Estimasi 58
15. Sistem Webhook & Notifikasi Eksternal 59
15.1 Latar Belakang & Kebutuhan 59
15.2 Desain yang Direkomendasikan 59
15.2.1 Event Types 60
15.2.2 Database Schema 60
15.2.3 Delivery Engine 61
15.2.4 Security 61
15.3 Langkah Implementasi 61
15.4 Prioritas & Estimasi 62
16. Configuration Drift Detection 63
16.1 Latar Belakang & Kebutuhan 63

16.2 Desain yang Direkomendasikan 64
Arsitektur 64
Database Schema 64
Severity Classification 65
16.3 Langkah Implementasi 65
16.4 Prioritas & Estimasi 65
17. Strategi Backup dan Disaster Recovery 66
17.1 SQLite Backup Strategy 66
17.2 Definisi RPO dan RTO 66
17.3 Hub Failover Plan 66
17.4 Valkey Persistence 66
17.5 Agent Behavior Selama Extended Outage 66
18. Data Retention & Automated Cleanup Policy 68
18.1 Latar Belakang & Kebutuhan 68
18.2 Desain yang Direkomendasikan 69
18.2.1 Retention Policies 69
18.2.2 Downsampling Strategy 69
18.2.3 Database Schema 69
18.2.4 Archive Strategy 70
18.2.5 Dashboard 70
18.3 Langkah Implementasi 70
18.4 Prioritas & Estimasi 70
19. Concurrency dan Performance Tuning 71
19.1 SQLite Write Serialization 71
19.2 Connection Pool Settings 71
19.3 WebSocket Connection Limits 71
19.4 Memory Budget 71
19.5 Agent Graceful Degradation untuk Plugin Errors 71
20. Strategi Update dan Rollback Agent 72
20.1 Push Update via Hub 72
20.2 Compatibility Matrix 72
20.3 Rollback Procedure 72
20.4 Canary Deployment 72
20.5 Self-Update vs. External Orchestration 72
21. Plugin SDK & Panduan Pengembangan Plugin 73
21.1 Latar Belakang & Kebutuhan 73
21.2 Desain yang Direkomendasikan 74

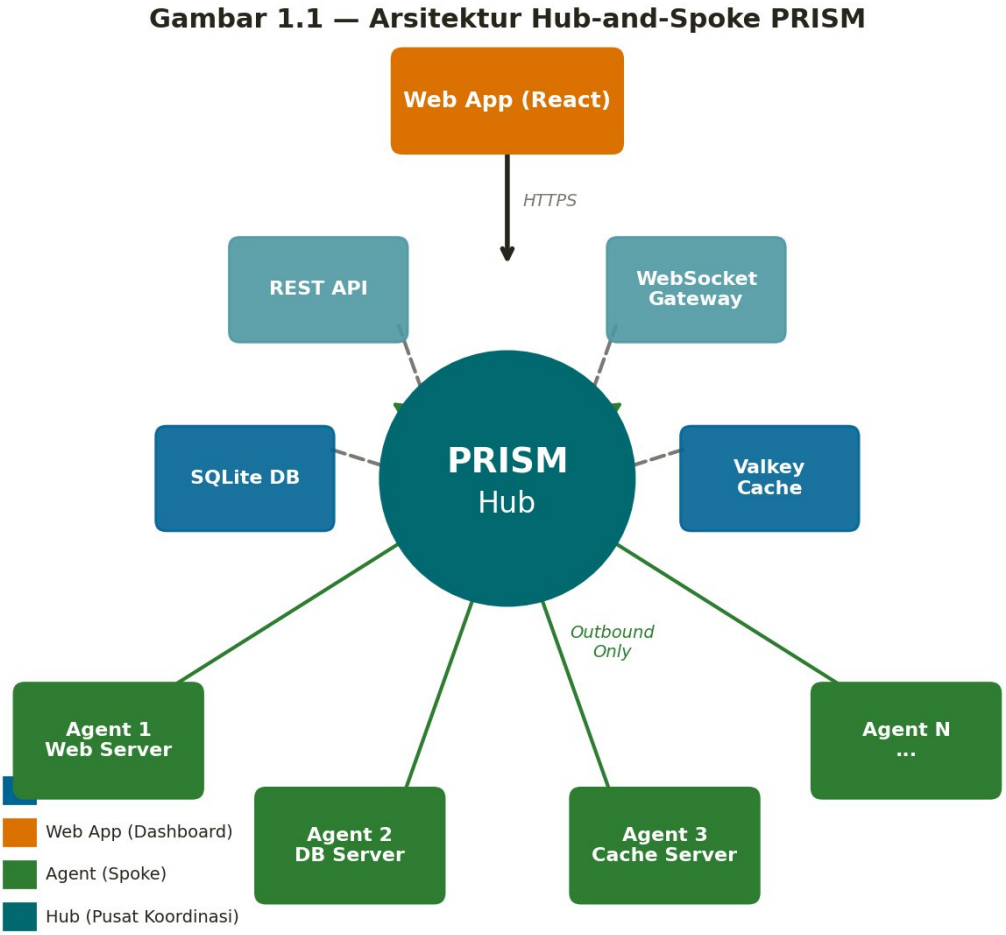
Plugin Interface 74
Plugin Metadata 74
Plugin Lifecycle 74
Plugin Registry di Hub 75
21.3 Langkah Implementasi 75
21.4 Prioritas & Estimasi 75
22. Data Export/Import & Portability 77
22.1 Latar Belakang & Kebutuhan 77
22.2 Desain yang Direkomendasikan 77
22.2.1 Export Scopes & Formats 77
22.2.2 API Endpoints 78
22.2.3 Import Validation & Conflict Resolution 78
22.2.4 Scheduled Exports 78
22.3 Langkah Implementasi 78
22.4 Prioritas & Estimasi 79
23. Perencanaan Pembangunan 80
23.1 Roadmap Pengembangan Bertahap 80
Fase 1 — Foundation (Bulan 1–3) 80
Fase 2 — Service Management (Bulan 4–6) 80
Fase 3 — Advanced Features (Bulan 7–9) 80
Fase 4 — Hardening & Scale (Bulan 10–12) 81
Ringkasan Roadmap 81
23.2 Strategi Testing 81
Unit Tests 81
Integration Tests 81
End-to-End Tests 81
Performance Tests 82
Security Tests 82
23.3 CI/CD Pipeline 82
23.4 Dokumentasi Teknis Tambahan 82
23.5 Estimasi Effort dan Tim 82
23.6 Prioritas Implementasi Rekomendasi 83
Rekomendasi 10: Optimalisasi Skalabilitas Telemetri dengan Time-Series Partitioning 85
10.1 Latar Belakang 85
10.2 Arsitektur Time-Series Partitioning 85
10.3 Skema Database 85

10.4 Alur Kerja 86
10.5 Konfigurasi 86
10.6 Estimasi Performa 86
10.7 Langkah Implementasi 87
10.8 Prioritas & Estimasi 87
Rekomendasi 11: Penguatan Keamanan Update Agent dengan Code Signing 88
11.1 Latar Belakang 88
11.2 Arsitektur Code Signing 88
11.3 Alur Update yang Aman 88
11.4 Struktur Manifest 89
11.5 Key Management 89
11.6 Proteksi Tambahan 89
11.7 Langkah Implementasi 90
11.8 Prioritas & Estimasi 90
Rekomendasi 12: Integrasi Identity Provider (IdP) Eksternal dengan Group-to-Role Mapping 91
12.1 Latar Belakang 91
12.2 Arsitektur Integrasi 91
12.3 Group-to-Role Mapping 91
12.4 Konfigurasi 92
12.5 Fitur Utama 92
12.6 Langkah Implementasi 93
12.7 Prioritas & Estimasi 93
Rekomendasi 13: Circuit Breaker pada Plugin Agent 94
13.1 Latar Belakang 94
13.2 Arsitektur Circuit Breaker 94
13.3 Integrasi dengan Plugin Manager 94
13.4 Konfigurasi per Plugin 94
13.5 Monitoring & Reporting 95
13.6 Interaksi dengan Service Discovery 95
13.7 Langkah Implementasi 95
13.8 Prioritas & Estimasi 96
Lampiran A: Detail Modal Konfigurasi per Service 97
A.1 MySQL 97
A.2 PostgreSQL 97
A.3 MongoDB 98
A.4 Valkey 98

A.5 RabbitMQ 99
A.6 MQTT (Mosquitto) 99
A.7 MinIO 100
A.8 Garage 100
A.9 SeaweedFS 101
A.10 vsftpd 101
A.11 SFTPGo 102
A.12 Nginx 102
A.13 Caddy 103
A.14 UFW 103
A.15 firewalld 103
A.16 iptables 104
A.17 nftables 104
A.18 CrowdSec 105
A.19 PM2 105
A.20 Supervisor (supervisord) 106
A.21 systemd User Services 106
A.22 Docker 107
A.23 Podman 108
Lampiran B: Skema Database Lengkap 110
B.1 Tabel servers 110
B.2 Tabel services 110
B.3 Tabel management_credentials 111
B.4 Tabel accounts (Diperbarui) 111
B.5 Tabel agent_certificates 112
B.6 Tabel enrollment_keys 112
B.7 Tabel commands 113
B.8 Tabel telemetry 113
B.9 Tabel audit_log 114
B.10 Tabel users 114
B.11 Tabel permissions (BARU — Rekomendasi #1: RBAC Granular) 114
B.12 Tabel role_permissions (BARU — Rekomendasi #1: RBAC Granular) 115
B.13 Tabel user_server_access (BARU — Rekomendasi #1: RBAC Granular) 115
B.14 Tabel server_groups (BARU — Rekomendasi #1: RBAC Granular) 115
B.15 Tabel webhook_subscriptions (BARU — Rekomendasi #4: Webhook) 116
B.16 Tabel webhook_deliveries (BARU — Rekomendasi #4: Webhook) 116
B.17 Tabel configuration_snapshots (BARU — Rekomendasi #5: Drift Detection)

.....117
B.18 Tabel drift_events (BARU — Rekomendasi #5: Drift Detection).....117
B.19 Tabel retention_policies (BARU — Rekomendasi #6: Data Retention).....118

1. Gambaran Umum Arsitektur PRISM



Gambar 1.1 — Arsitektur Hub-and-Spoke PRISM

PRISM mengadopsi arsitektur hub-and-spoke di mana Prism Hub bertindak sebagai pusat koordinasi sentral dan setiap Prism Agent di server target bertindak sebagai spoke. Seluruh komunikasi terpusat melalui Hub untuk memudahkan manajemen, monitoring, dan keamanan.

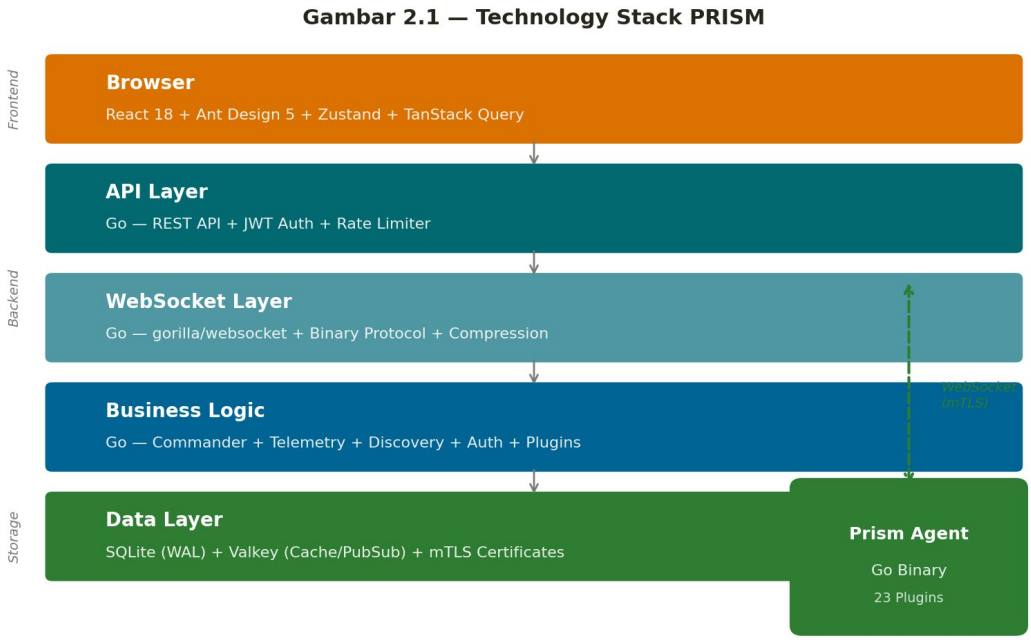
1.1 Komponen Utama

Komponen	Peran	Lokasi
Prism Agent	Lightweight daemon di setiap server target. Service discovery, monitoring, eksekusi perintah.	Setiap server yang dikelola
Prism Hub/Backend	Pusat koordinasi: mengelola koneksi agent, mengagregasi data, REST API, WebSocket gateway.	Server pusat (dedicated)
Prism Web App	Dashboard web: monitoring, approval agent, manajemen service dan akun.	Browser (prism.smartsystem.id)

SQLite Database	Penyimpanan utama: data server, service, akun, telemetry, audit trail.	Co-located dengan Hub (embedded)
Valkey Cache	In-memory cache: session, real-time metrics, agent state, frequently accessed data.	Co-located atau dedicated instance

Agent selalu menginisiasi koneksi ke Hub (outbound only). Hub tidak pernah menginisiasi koneksi ke agent, meminimalkan attack surface karena server target tidak perlu membuka port inbound tambahan.

2. Technology Stack



Gambar 2.1 — Technology Stack PRISM

PRISM dibangun dengan technology stack yang dipilih berdasarkan pertimbangan performa, efisiensi resource, kemudahan deployment, dan keandalan.

2.1 Ringkasan Stack

Layer	Teknologi	Justifikasi
Frontend (Web App)	React + Ant Design (AntD)	React memberikan component-based architecture yang scalable. Ant Design menyediakan enterprise-grade UI component library dengan design system yang konsisten, kaya fitur, dan mendukung theming.
Backend (Hub)	Go (Golang)	Performa tinggi dengan built-in concurrency (goroutines) untuk menangani ribuan WebSocket connections secara efisien. Compiled binary tanpa runtime dependency, memudahkan deployment. Memory footprint rendah.
Agent	Go (Golang)	Binary tunggal yang ringan dan cross-platform. Tidak memerlukan runtime di server target. Startup cepat, memory usage minimal, native support untuk system calls.
Database	SQLite	Embedded database tanpa server process terpisah, zero-configuration.

		Ideal untuk single-node deployment. File-based sehingga mudah di-backup. Mendukung WAL mode untuk concurrent read.
Cache	Valkey (Redis-compatible)	In-memory data store untuk caching layer. Mengurangi beban query ke SQLite. Menyimpan real-time state. Mendukung pub/sub untuk event distribution.

2.2 Frontend: React + Ant Design

Prism Web App dibangun menggunakan React sebagai UI framework dengan Ant Design (AntD) sebagai component library utama. Kombinasi ini memberikan developer experience yang produktif dan user experience yang konsisten.

Arsitektur Frontend

Aspek	Detail
Framework	React 18+ dengan functional components dan hooks
UI Library	Ant Design 5.x — enterprise-grade component library
State Management	Zustand sebagai pilihan utama untuk global state management (client state). TanStack Query untuk server state management — menangani caching, revalidation, dan synchronization data dari API. Kombinasi Zustand (client state) + TanStack Query (server state) memberikan separation of concerns yang jelas dan menghindari state duplication.
Routing	React Router v6 dengan lazy loading per route untuk optimasi bundle size
API Communication	Axios atau fetch API untuk REST calls ke Hub. Native WebSocket API untuk real-time data (telemetry, logs, notifications).
Build Tool	Vite — fast HMR untuk development, optimized build untuk production
Styling	Ant Design token-based theming + CSS Modules untuk custom styling
Authentication	JWT token disimpan di httpOnly cookie. Refresh token rotation otomatis.

Pemetaan Komponen Ant Design ke Fitur PRISM

Fitur PRISM	Komponen AntD Utama	Keterangan
Halaman Servers (list)	Table, Tag, Badge, Button, Input.Search	Tabel server dengan status badge, filter, search, dan tombol aksi
Pending Agent Panel	Alert, Descriptions, Space, Button, Popconfirm	Panel amber/warning untuk agent pending dengan tombol Approve/Reject

Form Akuisisi Server	Modal, Form, Input, Input.TextArea, Button	Modal dialog dengan form nama dan deskripsi server
Server Card	Card, List, Tag, Progress, Statistic, Divider	Card dengan header info, list service, dan quick stats
Service List dalam Card	List.Item, Avatar (icon), Tag (status), Typography.Text	Setiap service ditampilkan sebagai list item dengan icon kategori dan status tag
Modal Konfigurasi Service	Modal (fullscreen/large), Tabs, Descriptions, Table, Form, Switch, Select	Modal besar dengan tab-based layout untuk Overview/Accounts/Configuration/Logs
Tab Overview	Descriptions, Statistic, Row/Col, Progress	Layout grid menampilkan metrik dan informasi service
Tab Accounts	Table (editable), Button, Drawer, Form, Tag, Select	Tabel user/akun dengan inline edit, drawer untuk create/edit, tag untuk kategorisasi
Tab Configuration	Table, Collapse, Form, Switch, InputNumber, Select, Popconfirm	Tabel konfigurasi dengan collapsible sections dan form edit
Tab Logs	Typography.Text (monospace), Input.Search, Select (log level filter)	Live log viewer dengan auto-scroll, search, dan filter level
Aksi Service (Start/Stop/Restart)	Button, Popconfirm, notification, message	Tombol aksi dengan konfirmasi dan feedback notification
Dashboard Overview	Row/Col, Card, Statistic, Chart (via Ant Design Charts)	Dashboard utama dengan KPI cards dan visualisasi
User Management / RBAC	Table, Form, Transfer, Tree	Manajemen user internal PRISM dengan role assignment
Audit Log	Table (dengan pagination dan filter), DatePicker.RangePicker, Tag	Log audit dengan filter tanggal, user, dan aksi
Notification / Alert	notification, message, Badge (di header), Drawer (notification center)	Sistem notifikasi real-time untuk alert dan event penting

2.3 Backend (Hub): Go

Prism Hub dibangun dengan Go (Golang) yang memberikan performa tinggi dan native concurrency untuk menangani ribuan koneksi WebSocket secara simultan.

Arsitektur Backend

Aspek	Detail

Bahasa	Go 1.22+
HTTP Framework	Standard library net/http + gorilla/mux atau chi router. Middleware chain untuk auth, logging, CORS, rate limiting.
WebSocket	gorilla/websocket atau nhooyr.io/websocket untuk koneksi persisten agent. Connection pool dengan health check.
Database Access	mattn/go-sqlite3 (CGO) atau modernc.org/sqlite (pure Go) dengan database/sql interface. Query builder opsional (sqlc atau squirrel).
Cache Client	go-redis/redis (v9) sebagai client library Valkey. Connection pooling built-in.
TLS/mTLS	crypto/tls standard library untuk mTLS handshake. crypto/x509 untuk certificate management (internal CA).
JWT	golang-jwt/jwt (v5) untuk signing dan validasi JWT token (RS256).
Concurrency Model	Goroutine per agent connection. Channel-based communication antar komponen. sync.Pool untuk object reuse.
Logging	Structured logging dengan zerolog atau slog (Go 1.21+ built-in).
Configuration	Environment variables + config file (YAML/TOML). Viper library untuk config management.
Binary Output	Single compiled binary. Tidak memerlukan runtime terpisah. Didistribusikan via container image atau direct binary.

Modul Internal Hub

Modul	Tanggung Jawab	Interaksi Utama
api	REST API endpoints untuk Web App. Routing, request validation, response formatting.	Menerima request dari frontend, query ke db/cache, kirim command ke agent via ws
ws (WebSocket Gateway)	Mengelola koneksi WebSocket dari agent. Connection lifecycle, message routing, heartbeat monitoring.	Menerima data dari agent, forward command dari api module
auth	Authentication dan authorization. JWT issuance/validation, mTLS certificate management, RBAC enforcement.	Digunakan oleh api dan ws untuk validasi setiap request
enrollment	Proses registrasi agent baru. PSK validation, CSR signing, certificate issuance.	Dipanggil oleh ws saat menerima agent.hello message
discovery	Memproses hasil service discovery dari agent. Service identification, metadata extraction.	Menerima data dari ws, simpan ke db
commander	Membuat dan mendistribusikan command ke agent. Command queue, delivery confirmation, retry logic.	Dipanggil oleh api, dikirim via ws
telemetry	Memproses data telemetry dari agent. Aggregasi, threshold checking, alert	Menerima dari ws, simpan ke db/cache, trigger

	triggering.	notification
cache	Abstraksi layer untuk Valkey. Get/set/invalidate pattern, pub/sub subscription.	Digunakan oleh semua modul untuk caching
db	Database access layer. Migration, query functions, transaction management.	Digunakan oleh semua modul untuk persistence
sync	Sinkronisasi data ke PATRA. Data transformation, API client, retry mechanism.	Membaca dari db, kirim ke Patra Hub API
notifier	Sistem notifikasi. Alert routing, WebSocket push ke frontend, webhook dispatch.	Dipanggil oleh telemetry dan commander saat event penting

WebSocket Message Compression

PRISM menggunakan per-message deflate compression (RFC 7692) untuk WebSocket. Kompresi ini mengurangi penggunaan bandwidth secara signifikan, terutama penting untuk deployment multi-site dengan bandwidth terbatas.

Library gorilla/websocket yang dipilih sudah mendukung fitur ini melalui konfigurasi EnableCompression. Overhead CPU untuk kompresi umumnya minimal dibandingkan penghematan bandwidth yang didapat.

2.4 Agent: Go

Prism Agent juga dibangun dengan Go, menghasilkan single binary yang ringan dan dapat di-deploy di berbagai platform tanpa dependency tambahan.

Arsitektur Agent

Aspek	Detail
Binary Size	Estimasi 10–20 MB (single static binary, compressed). Tidak memerlukan runtime.
Target Platform	Linux (amd64, arm64). Opsional: FreeBSD, macOS untuk development.
Privilege	Berjalan sebagai systemd service dengan user dedicated (prism-agent). Capability-based permission untuk akses service.
Resource Usage	Target: < 50 MB RAM, < 1% CPU idle. Spike saat discovery atau command execution.
Configuration	File konfigurasi YAML: hub_url, enrollment_key, agent_id (auto-generated), tls cert paths, scan interval.
Plugin Architecture	Setiap jenis service dikelola oleh plugin/module terpisah (mysql_plugin, nginx_plugin, dll.). Plugin register otomatis saat startup berdasarkan service yang terdeteksi.
Offline Mode	Jika koneksi ke Hub terputus, agent tetap mengumpulkan data ke local buffer (SQLite lokal atau in-memory). Flush ke Hub saat koneksi pulih.

Service Plugin Agent

Agent menggunakan arsitektur plugin di mana setiap jenis service yang didukung memiliki plugin tersendiri. Plugin bertanggung jawab atas: detection, collection, account management, dan command execution.

Plugin	Detection Method	Data Collection	Account Operations
mysql	Cek process mysqld, port 3306, socket file	SHOW STATUS, SHOW VARIABLES, SHOW DATABASES, SHOW PROCESSLIST	CREATE/ALTER/DROP USER, GRANT/REVOKE
postgresql	Cek process postgres, port 5432, pg_isready	pg_stat_activity, pg_stat_database, pg_roles	CREATE/ALTER/DROP ROLE, GRANT/REVOKE
mongodb	Cek process mongod, port 27017	db.serverStatus(), db.stats(), rs.status()	db.createUser(), db.updateUser(), db.dropUser()
valkey	Cek process valkey-server, port 6379	INFO command (semua section)	ACL SETUSER, ACL DELUSER, ACL LIST
rabbitmq	Cek process beam.smp, management API port 15672	RabbitMQ Management HTTP API	PUT/DELETE /api/users, /api/permissions
mqtt	Cek process mosquitto, port 1883	SYS topics, dynamic security plugin API	Dynamic security plugin: createClient/deleteClient
minio	Cek process minio, S3 port 9000	MinIO Admin API: info, storage info	Admin API: add-user, set-policy, add-service-account
garage	Cek process garage, API port 3900	Admin API: /status, /health	Admin API: key create/delete, bucket allow
seaweedfs	Cek process weed master/filer, port 9333/8888	Master /cluster/status, filer /	S3 IAM via filer config
vsftpd	Cek process vsftpd, port 21, config /etc/vsftpd.conf	Parse config, check PAM/virtual users	Edit user DB (db_load), PAM config
sftpgo	Cek process sftpgo, API port 8080	SFTPGo REST API: /api/v2/status, /api/v2/connections	REST API: /api/v2/users (CRUD)
nginx	Cek process nginx, port 80/443	stub_status module, parse sites-enabled	N/A (config-based)
caddy	Cek process caddy, admin API port 2019	Admin API: /config/, /reverse_proxy/upstreams	N/A (config-based)

ufw	Cek ufw command available, /etc/ufw/	ufw status verbose, ufw app list	N/A (rule-based)
firewalld	Cek process firewalld, firewall-cmd	firewall-cmd --list-all-zones	N/A (zone/rule-based)
iptables	Cek iptables command, kernel module	iptables -L -n -v (per table)	N/A (rule-based)
nftables	Cek nft command, nftables service	nft list ruleset	N/A (rule-based)
crowdsec	Cek process crowdsec, cscli available	cscli metrics, cscli alerts list, cscli decisions list	N/A (decision/scenario-based)
pm2	Cek process PM2 (pm2 ping), pm2 jlist	pm2 jlist (JSON process list)	N/A (process-based)
supervisor	Cek process supervisord, supervisorctl available	supervisorctl status, supervisorctl avail	N/A (program-based)
systemd_user	Scan systemd user units via systemctl --user	systemctl --user list-units, status per unit	N/A (unit-based, user scope only)
docker	Cek docker daemon (docker info), socket /var/run/docker.sock	docker ps, docker stats, docker inspect, docker images	N/A (container-based). Image pull/remove, network/volume management.
podman	Cek podman binary, podman info (rootless atau rootful)	podman ps, podman stats, podman inspect, podman images	N/A (container-based). Rootless by default. Compatible API dengan Docker.

2.5 Database: SQLite

PRISM menggunakan SQLite sebagai database utama. SQLite dipilih karena sifatnya yang embedded (tidak memerlukan server process terpisah), zero-configuration, dan menghasilkan single file yang mudah di-backup dan di-migrate.

Konfigurasi SQLite untuk Production

Parameter	Nilai	Alasan
Journal Mode	WAL (Write-Ahead Logging)	Memungkinkan concurrent read saat write berlangsung. Menghilangkan lock contention antara API read dan telemetry write.
Busy Timeout	5000 ms	Menunggu hingga 5 detik jika database locked, mencegah SQLITE_BUSY error pada concurrent access.

Synchronous	NORMAL	Keseimbangan antara durability dan performa. FULL terlalu lambat untuk telemetry, OFF terlalu berisiko.
Cache Size	-64000 (64 MB)	Memperbesar page cache SQLite untuk mengurangi disk I/O pada query yang sering diakses.
Foreign Keys	ON	Menegakkan referential integrity antar tabel.
Auto Vacuum	INCREMENTAL	Mengoptimalkan ukuran file database secara bertahap tanpa blocking.
mmap_size	268435456 (256 MB)	Memory-mapped I/O memungkinkan SQLite mengakses file database secara langsung melalui virtual memory, mengurangi overhead system call untuk operasi read. Pada workload yang didominasi read seperti telemetri dashboard, peningkatan performa bisa mencapai 20–30%.

Strategi Index Telemetry

Tabel telemetri akan berkembang sangat cepat. Diperlukan strategi indexing yang tepat:

- Composite index pada (server_id, collected_at) untuk query berdasarkan rentang waktu per server.
- Partial index pada service_id untuk query spesifik per layanan.
- Pertimbangkan WITHOUT ROWID untuk tabel yang sering di-query guna mengurangi overhead storage.

Contoh implementasi:

CREATE INDEX idx_telemetry_server_time ON telemetry(server_id, collected_at);

CREATE INDEX idx_telemetry_service ON telemetry(service_id) WHERE service_id IS NOT NULL;

Skema Tabel Utama

Berikut adalah tabel-tabel utama dalam database PRISM (lihat Lampiran B untuk skema lengkap termasuk tabel management_credentials):

Tabel	Primary Key	Kolom Utama	Relasi
servers	id (TEXT, UUID)	name, description, hostname, ip_address, os, status (pending/active/unreachable/removed), agent_version, created_at, updated_at	Has many: services, accounts
services	id (TEXT, UUID)	server_id (FK), name, type, version, port, status (running/stopped/error), config_path, pid, discovered_at, updated_at	Belongs to: servers. Has many: accounts

management_credentials	id (TEXT, UUID)	service_id (FK), credential_type (root/admin/default), username_encrypted, password_encrypted, connection_params_encrypted, last_verified_at, status, created_at, updated_at	Belongs to: services
accounts	id (TEXT, UUID)	service_id (FK), server_id (FK), username, account_type (user/service), category (project/independent), project_name, permissions (JSON), status (active/disabled), created_at	Belongs to: services, servers
agent_certificates	id (TEXT, UUID)	server_id (FK), fingerprint, issued_at, expires_at, revoked_at, revocation_reason	Belongs to: servers
enrollment_keys	id (TEXT, UUID)	key_hash, created_by, expires_at, used_at, used_by_server_id	Used by: servers
commands	id (TEXT, UUID)	server_id (FK), service_id (FK), type, payload (JSON), status (pending/sent/executed/failed), issued_by, issued_at, executed_at, result (JSON)	Belongs to: servers, services
telemetry	id (INTEGER, auto)	server_id (FK), service_id (FK), metrics (JSON), collected_at	Belongs to: servers, services. Partitioned by time.
audit_log	id (INTEGER, auto)	user_id, action, resource_type, resource_id, details (JSON), ip_address, timestamp	Standalone audit trail
users	id (TEXT, UUID)	username, email, password_hash, role (admin/operator/viewer), mfa_enabled, last_login, status	PRISM internal users

Catatan tentang Skalabilitas SQLite: SQLite dengan WAL mode mampu menangani ratusan server dan ribuan service dengan baik. Untuk deployment yang lebih besar (1000+ server), tabel telemetry dapat dirotasi menggunakan time-based partitioning (tabel per hari/minggu) atau di-migrate ke time-series database terpisah. Data yang sering diakses di-cache di Valkey untuk mengurangi beban query.

Strategi Migrasi Database

Dokumen ini mendokumentasikan jalur migrasi untuk skenario scaling di masa depan:

- Jalur migrasi yang jelas: Definisi path migrasi dari SQLite ke PostgreSQL/CockroachDB jika scaling melampaui single-node.
- Strategi zero-downtime migration: Pendekatan migrasi data tanpa downtime menggunakan dual-write atau shadow database.

- Database versioning: Gunakan schema migration tooling seperti golang-migrate atau goose untuk pengelolaan versi schema.
- Kebijakan retensi data telemetri: Tentukan berapa lama data disimpan, kapan di-archive, dan kapan di-purge.

2.6 Cache: Valkey

Valkey (fork Redis yang fully open-source) digunakan sebagai caching layer di PRISM. Valkey menyimpan data yang sering diakses dan data real-time yang berubah cepat, mengurangi beban query ke SQLite dan meningkatkan responsiveness Web App.

Strategi Caching

Data	Key Pattern	TTL	Strategi
Agent online status	agent:{server_id}:status	60 detik (diperbarui tiap heartbeat)	Write-through: setiap heartbeat update Valkey. Jika key expired = agent unreachable.
Service status per server	server:{server_id}:services	30 detik	Write-through: diperbarui saat telemetry masuk. Frontend membaca dari cache.
Real-time metrics	metrics:{server_id}:{service_id}	60 detik	Write-through: data CPU, memory, connections. Dioverwrite setiap telemetry push.
Server list (active)	servers:active:list	5 menit	Cache-aside: populate saat pertama kali diakses. Invalidate saat ada perubahan server.
JWT refresh token mapping	auth:refresh:{token_hash}	24 jam	Write-through: disimpan saat token issued. Dihapus saat revoke atau rotation.
User session	session:{user_id}	Sesuai session config	Write-through: menyimpan data session user Web App.
Command status	cmd:{command_id}:status	1 jam	Write-through: update real-time saat command dispatched/executed/failed.
Rate limiting counters	ratelimit:{ip}:{endpoint}	1 menit (sliding window)	Increment on request: menghitung request count per IP per endpoint.
Dashboard aggregate stats	dashboard:stats	30 detik	Cache-aside: aggregate query di-cache. Invalidate periodik.

Pub/Sub untuk Real-time Events

Channel	Publisher	Subscriber	Event
events:server:{id}	ws module (saat menerima data agent)	API module (forward ke frontend WebSocket)	Agent connect/disconnect, status change, new telemetry
events:service:{id}	ws module, commander module	API module	Service start/stop/error, config change, account change
events:alerts	telemetry module	API module, notifier module	Threshold breach, agent unreachable, certificate expiring
events:commands:{id}	commander module	API module	Command dispatched, executed, failed (untuk progress tracking di frontend)

Perencanaan Cluster Mode Valkey

Desain saat ini mengasumsikan single Valkey instance. Berikut jalur scaling yang didokumentasikan untuk masa depan:

- Valkey Sentinel — Minimal 3 node untuk automatic failover, cocok untuk skala menengah.
- Valkey Cluster — Untuk kebutuhan sharding data ketika single instance tidak cukup.

Meskipun mungkin tidak dibutuhkan di awal, mendokumentasikan jalur evolusi ini membantu perencanaan infrastruktur jangka panjang.

2.7 Data Flow: Bagaimana Komponen Berinteraksi

Berikut adalah alur data untuk operasi-operasi utama di PRISM, menunjukkan bagaimana setiap komponen teknologi berinteraksi:

Read Path (Admin membuka dashboard)

1. Browser (React) mengirim GET request ke Hub REST API.
2. Hub (Go) memeriksa JWT token di request header untuk autentikasi.
3. Hub mengecek Valkey cache terlebih dahulu untuk data yang diminta.
4. Cache hit: data langsung dikembalikan ke frontend. Cache miss: Hub query ke SQLite, hasilnya disimpan ke Valkey, lalu dikembalikan.
5. Frontend (React) merender data menggunakan komponen Ant Design (Table, Card, Statistic).

Write Path (Admin mengirim command ke service)

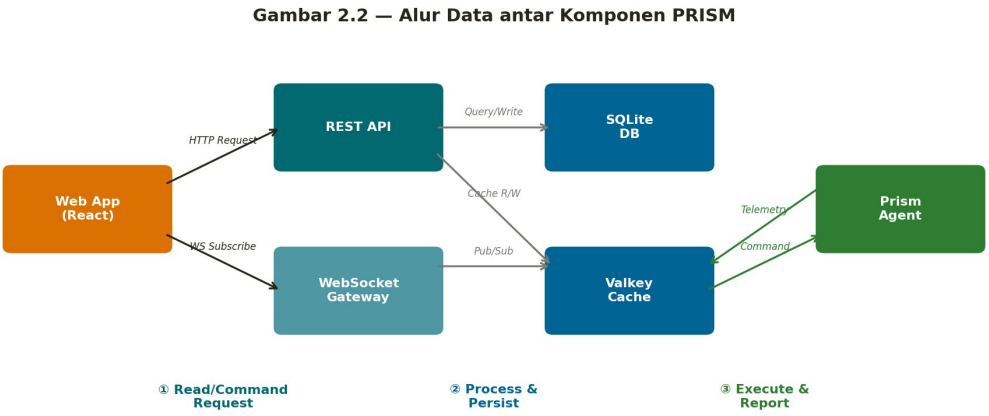
6. Frontend mengirim POST request ke Hub REST API dengan command payload.
7. Hub (api module) memvalidasi request, menyimpan command ke SQLite dengan status "pending".

8. Hub (commander module) mengirim command ke agent via WebSocket, update status ke "sent" di Valkey.
9. Hub publish event ke Valkey Pub/Sub channel events:commands:{id}.
10. Agent (Go) menerima command, mengeksekusi, mengirim result kembali via WebSocket.
11. Hub (ws module) menerima result, update status command di SQLite dan Valkey, publish event.
12. Frontend menerima event via WebSocket, menampilkan notification.

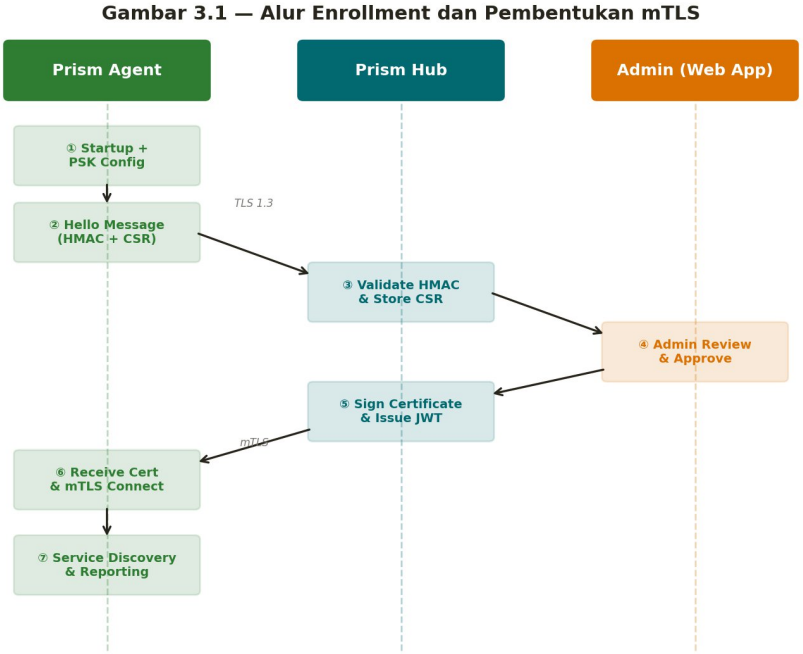
Telemetry Path (Agent mengirim data periodik)

13. Agent (Go) mengumpulkan metrik dari service plugins setiap 60 detik.
14. Agent mengirim data via WebSocket ke Hub.
15. Hub (ws module) menerima data, forward ke telemetry module.
16. Hub (telemetry module) memproses data: update Valkey cache (real-time metrics), insert ke SQLite (historical), cek threshold untuk alerting.
17. Hub jika ada threshold breach, publish alert ke Valkey Pub/Sub, trigger notifier module.
18. Frontend menerima update via WebSocket subscription, re-render komponen Statistic/Progress secara real-time.

3. Alur Kerja PRISM



Gambar 2.2 — Alur Data antar Komponen PRISM



Gambar 3.1 — Alur Enrollment dan Pembentukan mTLS

Berikut adalah urutan proses kerja lengkap dari saat agent pertama kali melapor hingga administrator dapat mengelola service secara penuh.

3.1 Fase 1: Agent Melapor ke Hub (Otomatis)

Ketika Prism Agent pertama kali dijalankan, agent secara otomatis menginisiasi koneksi ke Hub untuk mendaftarkan diri. Proses ini berjalan tanpa intervensi administrator.

19. Agent Startup: Agent binary dijalankan di server target. Konfigurasi awal berisi Hub URL dan Pre-Shared Enrollment Key (PSK) yang bersifat one-time-use.
20. Hello Message: Agent mengirim pesan pertama ke Hub melalui TLS 1.3 berisi: HMAC-SHA256 dari PSK + timestamp, Certificate Signing Request (CSR) berisi public key agent, dan metadata server (hostname, OS, IP, agent version).
21. Hub Validates: Hub memverifikasi HMAC terhadap PSK yang tersimpan. Jika valid dan PSK belum expired/terpakai, Hub menandai agent sebagai "pending approval".
22. Certificate Issuance: Hub (sebagai internal CA) menandatangani CSR dan mengirim kembali: signed client certificate (X.509), CA certificate, dan initial JWT access + refresh token.
23. mTLS Established: Agent membentuk koneksi baru menggunakan mutual TLS. Kedua pihak saling memverifikasi certificate. Enrollment key diinvalidasi.
24. Service Discovery: Agent melakukan scanning otomatis (systemd units, docker containers, process list) dan melaporkan hasilnya ke Hub.
25. Status: Pending Approval. Agent terhubung aman dengan status "pending approval", menunggu administrator.

3.2 Fase 2: Admin Melihat dan Meng-approve Agent

Administrator membuka Prism Web App dan navigasi ke tab Servers.

Pending Agent di Frontend

Agent berstatus "pending approval" ditampilkan dalam panel terpisah (AntD Alert component dengan type "warning") dengan informasi:

Informasi	Komponen AntD	Contoh
Hostname	Descriptions.Item	srv-prod-db-01
IP Address	Descriptions.Item	10.0.1.45
OS	Descriptions.Item + Tag	Ubuntu 24.04 LTS
Agent Version	Tag	v1.2.0
Detected Services	Badge (count)	5 services
Connected Since	Descriptions.Item	2026-03-19 14:32 WIB
Status	Tag (color: gold)	Pending Approval
Aksi	Button (primary) + Popconfirm (danger)	[Approve] [Reject]

Proses Approval

26. Admin menekan tombol "Approve". Muncul AntD Modal dengan Form di dalamnya.
27. Admin mengisi form: Server Name (Input, required) dan Description (Input.TextArea, optional).

28. Admin menekan "Save" / "Acquire". Frontend mengirim POST ke Hub API. Hub mengubah status agent dari "pending" ke "active", menyimpan nama dan deskripsi, menginstruksikan agent memulai full monitoring.
29. Feedback: AntD notification (type: success) muncul. Agent berpindah dari panel pending ke daftar server aktif.

3.3 Fase 3: Server Card Tampil

Setelah approval, server muncul sebagai Card (AntD Card component) di halaman Servers. Card ini me-list semua service yang terdeteksi oleh agent.

Anatomi Server Card

Bagian Card	Komponen AntD	Isi
Card Header	Card title + extra	Server Name, Status Badge (running=green, warning=amber, error=red), IP Address
Metadata Row	Space + Tag	OS tag, uptime, agent version, last heartbeat (Tooltip: exact timestamp)
Quick Stats	Row + Col + Statistic	Total: 5 | Running: 4 | Stopped: 0 | Error: 1
Service List	List + List.Item	Setiap service: icon kategori (Avatar), nama + versi (Text), status (Tag)
Footer	Card actions	Edit Server, Restart Agent, Remove (Popconfirm)

Kategori Service

Kategori	Service	Tag Color (AntD)
Database	MySQL, PostgreSQL, MongoDB, Valkey	green
Message Broker	RabbitMQ, MQTT (Mosquitto)	purple
Object Storage	MinIO, Garage, SeaweedFS	blue
File Transfer	vsftpd, SFTPGo	cyan
Web Server / Reverse Proxy	Nginx, Caddy	orange
Firewall & Security	UFW, firewalld, iptables, nftables, CrowdSec	red
Process Supervisor	PM2, Supervisor, systemd user services	geekblue
Container Runtime	Docker, Podman	magenta

3.4 Fase 4: Modal Konfigurasi Service

Ketika admin mengklik service dari Server Card, muncul Modal besar (AntD Modal width: 900px+) dengan Tabs layout berisi 4 tab: Overview, Accounts, Configuration, dan Logs.

Struktur Umum Modal

Tab	Komponen AntD Utama	Isi
Header (di atas tabs)	Space, Tag, Typography.Title, Statistic	Nama service, versi, status badge, uptime, PID
Overview	Descriptions, Row/Col, Statistic, Progress	Port, config path, memory, CPU, connections, service-specific metrics
Accounts	Table (sortable, filterable), Button, Drawer, Form, Tag	Daftar Application Accounts + CRUD. Kategorisasi project-based / independent. (Management Credentials ditampilkan terpisah di tab Connection.)
Configuration	Descriptions, Table, Collapse, Form, Switch, Select	Field konfigurasi spesifik per service type
Logs	div (monospace, auto-scroll), Input.Search, Select	Live tail log via WebSocket stream
Footer	Space, Button, Popconfirm	Start / Stop / Restart / Apply Config / Refresh

Detail field konfigurasi per service type tersedia di Lampiran A. Setiap service memiliki field Overview, Accounts, dan Configuration yang berbeda sesuai kemampuan service tersebut.

4. Manajemen Akun

PRISM mengelola akun dalam dua jenis utama yang berbeda secara fundamental: Management Credentials dan Application Accounts. Bagian ini menjelaskan pemisahan tersebut serta konsep project sebagai resource group.

4.1 Dua Jenis Akun dalam PRISM

Pemisahan jenis akun merupakan perubahan fundamental dalam arsitektur PRISM v4.0. Berikut definisi kedua jenis akun:

Management Credentials (Akun Manajemen)

Management Credentials adalah akun root/admin/default yang digunakan oleh Agent untuk terhubung dan mengelola service. Akun ini bersifat service-level dan bukan dibuat oleh pengguna. Contoh:

- MySQL root account
- PostgreSQL postgres superuser
- MongoDB admin user
- MinIO admin access key dan secret key
- RabbitMQ default admin
- Valkey default password

Management Credentials digunakan Agent untuk melakukan operasi CRUD pada Application Accounts, mengumpulkan telemetri, dan mengeksekusi perintah manajemen pada service.

Application Accounts (Akun Aplikasi)

Application Accounts adalah akun yang dibuat untuk project atau aplikasi tertentu. Akun ini mengikuti kategorisasi project-based atau independent. Contoh:

- MySQL user untuk aplikasi e-commerce
- MinIO access key untuk storage project tertentu
- RabbitMQ user untuk microservices project
- PostgreSQL role untuk data analytics pipeline

4.2 Management Credentials

Management Credentials disimpan dalam tabel terpisah (management_credentials) dengan enkripsi dan kontrol akses yang ketat.

Skema Tabel management_credentials

Kolom	Tipe	Keterangan
id	TEXT (UUID)	Primary key
service_id	TEXT (FK)	Referensi ke tabel services
credential_type	TEXT	root / admin / default
username_encrypted	BLOB	Username terenkripsi (AES-256-GCM)

password_encrypted	BLOB	Password terenkripsi (AES-256-GCM)
connection_params_encrypted	BLOB	Parameter koneksi terenkripsi (host, port, dsb.)
last_verified_at	DATETIME	Timestamp terakhir credential diverifikasi berhasil
status	TEXT	active / inactive / error
created_at	DATETIME	Timestamp pembuatan
updated_at	DATETIME	Timestamp pembaruan terakhir

Aturan Management Credentials

- Ditampilkan di Service Modal di bawah tab "Connection" atau section "Management Credential" — BUKAN di tab Accounts.
- Harus dienkripsi at rest menggunakan AES-256-GCM.
- Di-mask di UI (hanya menampilkan 4 karakter terakhir).
- Wajib audit logging untuk setiap akses dan perubahan.
- Digunakan oleh Agent untuk operasi service: CRUD application accounts, telemetry collection, command execution.

4.3 Konsep Project sebagai Resource Group

Dalam PRISM, sebuah project bukan sekadar label atau tag. Project adalah resource group — kumpulan seluruh resource infrastruktur yang dibutuhkan sebuah aplikasi atau sistem untuk berjalan. Sebuah project biasanya terdiri dari kombinasi beberapa komponen berikut:

Komponen	Fungsi dalam Project	Service di PRISM	Contoh Resource
Database	Menyimpan data aplikasi	MySQL, PostgreSQL, MongoDB, Valkey	Database + user + permission khusus untuk project
Message Broker	Komunikasi antar microservice, event-driven architecture	RabbitMQ, MQTT	Virtual host, exchange, queue, user + permission
Object Storage	Menyimpan file upload, media, backup, static assets	MinIO, Garage, SeaweedFS	Bucket + access key + policy khusus project
Worker / Helper	Background job processing, scheduled tasks	PM2, Supervisor, systemd user	Process: email-worker, pdf-generator, cron-scheduler
AI Worker	Machine learning inference, NLP processing	PM2, Supervisor, systemd user	Process: ai-inference-api, embedding-worker, llm-gateway
Data Pipeline Worker	ETL, data sync, log aggregation	PM2, Supervisor, systemd user	Process: etl-pipeline, kafka-consumer, log-processor

Reverse Proxy / Web Server	Routing traffic ke aplikasi, SSL termination	Nginx, Caddy	Site config / server block untuk domain project
File Transfer	Upload/download file untuk integrasi pihak ketiga	vsftpd, SFTPGo	User FTP/SFTP + folder + quota khusus project
Firewall Rules	Membuka port atau allow IP yang dibutuhkan	UFW, firewalld, iptables, nftables	Rules yang spesifik untuk kebutuhan network project
Security	Proteksi terhadap brute force dan abuse	CrowdSec	Scenario dan bouncer config yang relevan
Container	Menjalankan service atau aplikasi dalam container	Docker, Podman	Container instances, images, volumes, networks khusus project

Contoh: Anatomi Project "E-Commerce Platform"

Resource	Server	Service	Detail
PostgreSQL DB user	srv-db-01	PostgreSQL	User: ecommerce_app, DB: ecommerce_prod, Permission: ALL on ecommerce_prod
Valkey namespace	srv-cache-01	Valkey	ACL user: ecommerce, key pattern: ecom:*, permission: +@all ~ecom:*
RabbitMQ vhost	srv-mq-01	RabbitMQ	VHost: /ecommerce, User: ecom_service, Queues: order.created, payment.processed, shipment.updated
MinIO bucket	srv-storage-01	MinIO	Bucket: ecommerce-assets, Access key: ecom-s3-key, Policy: readwrite on ecommerce-assets/*
API server	srv-app-01	PM2	Process: ecom-api (cluster mode, 4 instances)
Order worker	srv-app-01	PM2	Process: ecom-order-worker
AI recommendation	srv-ai-01	systemd user	Unit: ecom-recommender.service
ETL pipeline	srv-data-01	Supervisor	Program: ecom-etl-sync
Web server	srv-web-01	Nginx	Site: ecommerce.example.com → upstream srv-app-01:3000
Firewall	srv-web-01	UFW	Allow 443/tcp from any, Allow 3000/tcp from srv-web-01 only
Redis container	srv-app-01	Docker	Container: ecom-session-redis (redis:7-alpine), port 6380

Semua resource di atas terkait satu sama lain dan memiliki lifecycle yang sama. Ketika project e-commerce selesai atau di-decommission, seluruh resource ini perlu di-review dan di-cleanup secara bersamaan. Inilah value utama dari model project di PRISM.

4.4 Kategorisasi Resource: Accounts dan Processes

Kategorisasi project-based vs independent berlaku untuk dua jenis resource yang dikelola PRISM: Application Accounts (akun aplikasi) dan processes (proses custom di PM2/Supervisor/systemd user). Kedua jenis resource ini mengikuti pola kategorisasi yang sama.

Penting: Dalam konteks kategorisasi ini, "Accounts" merujuk pada Application Accounts saja — bukan Management Credentials. Management Credentials dikelola secara terpisah di level service.

Application Accounts (Akun Aplikasi)

Kategori	Deskripsi	Contoh
Project-based	Akun yang terikat langsung pada suatu project. Lifecycle mengikuti lifecycle project.	MySQL user untuk app e-commerce, MinIO access key untuk storage project, RabbitMQ vhost untuk microservices project X
Independent	Akun yang tidak terikat pada project. Bersifat long-lived untuk operasional, monitoring, atau shared access.	Akun monitoring Grafana, akun backup scheduler, shared S3 key untuk cross-project assets

Processes (Proses Custom)

Kategori	Deskripsi	Contoh
Project-based	Proses yang merupakan bagian dari stack sebuah project. Lifecycle mengikuti lifecycle project.	API server project X (PM2), order-worker project X (Supervisor), AI inference service project Y (systemd user)
Independent	Proses yang berjalan untuk keperluan infrastruktur umum, maintenance, atau lintas project.	Storage Cleanup, Certificate Sync, Log Rotator, Database Backup Runner, Health Check Monitor, dll.

Contoh Independent Processes

Proses	Supervisor	Fungsi	Contoh Detail
Storage Cleanup	PM2 / systemd user	Menghapus file lama, temporary file	Cron-style process yang scan /data/uploads/ dan hapus file > 30 hari
Certificate Sync	PM2 / systemd	Upload/download	Watch certificate

	user	SSL/TLS certificate antar server	directory, sync ke server Nginx/Caddy lain via SCP/rsync
Log Rotator / Archiver	Supervisor / systemd user	Kompresi, pindahkan, atau hapus log lama	Compress log > 7 hari ke .gz, pindahkan ke cold storage (MinIO)
Database Backup Runner	PM2 / Supervisor	Backup database terjadwal ke object storage	Dump PostgreSQL setiap jam, encrypt, upload ke MinIO bucket
Health Check Monitor	PM2 / systemd user	Memonitor endpoint kesehatan service	Cek HTTP 200 pada /health setiap 30 detik, kirim webhook ke Slack jika gagal
Metrics Exporter	systemd user	Custom exporter untuk metrik non-standar	Baca status dari hardware sensor, expose sebagai /metrics endpoint
Data Sync / Replicator	Supervisor / PM2	Sinkronisasi data antar sistem	Sync tabel tertentu dari production DB ke analytics DB setiap 15 menit
Queue Consumer (shared)	PM2 / Supervisor	Consumer pesan dari broker untuk lintas project	Consume dari exchange "audit.events", tulis ke audit log database
DNS/Domain Updater	systemd user	Memperbarui DNS record secara periodik	Update Cloudflare DNS record dengan IP terbaru server
Security Scanner	Supervisor / systemd user	Vulnerability scan, file integrity check	Scan /var/www dan /app setiap malam, lapor ke admin jika ditemukan file mencurigakan

Independent Process ≠ Tidak Terpantau: Meskipun tidak terikat project, independent processes tetap sepenuhnya terpantau oleh PRISM: status (running/stopped/error), CPU/memory usage, restart count, dan log. Perbedaannya hanya pada kategorisasi dan lifecycle — independent processes tidak terpengaruh ketika suatu project di-decommission.

4.5 Menu Projects

Menu Projects menampilkan seluruh project beserta resource infrastructure yang terasosiasi. Setiap project adalah pandangan lengkap terhadap seluruh akun, proses, dan konfigurasi yang dibutuhkan aplikasi/sistem tersebut.

Halaman Project List

Elemen	Komponen AntD	Keterangan
Project Table / Card Grid	Table atau Card + List	Daftar project dengan mode tampilan table (dense) atau card grid (visual). Toggle via

		segmented control.
Kolom: Project Name	Typography.Link	Nama project, clickable untuk masuk ke detail project
Kolom: Description	Typography.Text	Deskripsi singkat project
Kolom: Owner / Team	Tag	Nama team atau person yang memiliki project
Kolom: Resources	Badge (multi)	Breakdown resource: 3 accounts, 2 processes, 1 site config — ditampilkan sebagai badges
Kolom: Servers	Badge	Jumlah server yang memiliki resource project ini
Kolom: Status	Tag (active/completed/archived)	Status lifecycle project
Kolom: Created	Typography.Text	Tanggal project dibuat
Aksi	Button (View, Edit, Archive)	Aksi per project
Tombol: + New Project	Button (primary)	Membuat project baru
Filter & Search	Input.Search, Select (status, owner, server)	Pencarian dan filtering

Halaman Project Detail

Halaman detail project menampilkan resource map lengkap — semua akun, proses, dan konfigurasi yang terkait project, dikelompokkan agar mudah dipahami:

Section	Komponen AntD	Isi
Project Info	Descriptions	Nama, deskripsi, owner/team, status, created/updated date
Resource Summary	Row + Col + Statistic	Ringkasan: 4 database accounts, 2 storage keys, 1 broker vhost, 3 worker processes, 1 web config. Total across N servers.
Tab: Accounts	Table (groupable by server/service)	Semua Application Accounts project: username, service, server, status, permissions. CRUD dari sini.
Tab: Processes	Table (groupable by server/supervisor)	Semua custom process project
Tab: Containers	Table (groupable by server/runtime)	Semua container project: Docker/Podman containers, images, status, port mapping, volume mounts. Start/stop/restart dari sini.
Tab: Web / Proxy Config	Table	Site configs di Nginx/Caddy yang terkait project ini

Tab: Firewall Rules	Table	Firewall rules yang di-tag untuk project ini (jika applicable)
Tab: Resource Map	Collapse / Tree atau Visualisasi graf	Visualisasi hubungan antar resource: server → service → akun/proses
Quick Actions	Button, Popconfirm	Disable All Accounts, Stop All Processes, Export Resource List, Decommission Project
Audit Trail	Table (timeline)	History perubahan: resource ditambahkan/dihapus, permission diubah, process started/stopped

Project Lifecycle

Status	Arti	Dampak pada Resource
Active	Project sedang berjalan. Resource aktif digunakan.	Semua resource berstatus normal. Monitoring dan alerting aktif.
Completed	Project selesai. Resource tidak lagi dibutuhkan.	PRISM menampilkan summary resource dan menawarkan opsi: disable all accounts, stop all processes, atau keep with warning. Resource yang belum di-cleanup mendapat label "orphan".
Archived	Project di-archive untuk referensi historis.	Seluruh resource seharusnya sudah di-disable/delete. Jika masih ada yang aktif, PRISM menampilkan alert.

4.6 Menu Accounts

Menu Accounts menampilkan unified view seluruh Application Accounts dari semua service di semua server yang dikelola PRISM. Menu ini fokus pada akun aplikasi (database users, access keys, broker users, FTP users, dll.) — bukan Management Credentials.

Halaman Account List

Elemen	Komponen AntD	Keterangan
Account Table	Table (with pagination, sorting, multi-column filter)	Tabel utama yang menampilkan seluruh Application Accounts dari seluruh server
Kolom: Username	Typography.Text	Username atau key ID akun
Kolom: Service	Tag (color per kategori)	Nama service dan tipe (MySQL, MinIO, RabbitMQ, dll.)
Kolom: Server	Typography.Link	Nama server di mana akun

		berada, clickable ke server detail
Kolom: Category	Tag (blue=project, gray=independent)	Kategorisasi: project-based atau independent
Kolom: Project	Typography.Link atau "---"	Nama project (jika project-based) atau dash (jika independent)
Kolom: Type	Tag	Tipe akun: user, service account
Kolom: Status	Badge (green=active, gray=disabled)	Status akun
Kolom: Permissions	Tooltip + Tag	Ringkasan permission (hover untuk detail)
Kolom: Last Activity	Typography.Text	Last login atau last used timestamp
Filter Panel	Form (inline), Select, DatePicker	Filter by: server, service type, category, project, status, date range
Bulk Actions	Button + Checkbox selection	Disable selected, change category, assign to project, export
Tombol: + New Account	Button (primary)	Membuat akun baru (pilih server → service → isi detail)

Fitur Cross-Reference

Fitur	Deskripsi
Search Global	Mencari akun berdasarkan username di seluruh server dan service. Berguna untuk audit: "di mana saja user X memiliki akun?"
Filter by Project	Menampilkan hanya akun yang terkait project tertentu, lintas server dan service.
Orphan Detection	Mendeteksi akun yang masih project-based tetapi project-nya sudah berstatus completed/archived. Alert untuk cleanup.
Duplicate Detection	Mendeteksi username yang sama di beberapa service/server. Berguna untuk standarisasi naming convention.
Compliance View	Menampilkan akun yang belum di-review dalam X hari, akun tanpa MFA (jika applicable), akun dengan excessive permissions.
Export	Export daftar akun ke CSV/Excel untuk keperluan audit external.

4.7 Interaksi Projects dan Accounts dengan Service Modal

Menu Projects dan Accounts terintegrasi erat dengan modal konfigurasi service di Server Card. Tab Accounts pada service modal menampilkan Application Accounts, sedangkan Management Credentials ditampilkan di tab Connection yang terpisah.

Aksi	Alur

Buat akun dari Service Modal	Admin klik service di Server Card → Tab Accounts → + New Account → Isi username, permissions → Pilih kategori (project/independent) → Jika project: pilih project dari dropdown → Save. Akun otomatis muncul di menu Accounts dan di halaman Project detail.
Buat akun dari menu Accounts	Admin klik + New Account di halaman Accounts → Pilih server → Pilih service → Isi detail + kategori → Save. Akun otomatis muncul di Tab Accounts pada service modal yang relevan.
Assign akun ke project	Admin bisa mengubah kategori akun dari independent ke project-based (atau sebaliknya) baik dari Service Modal maupun dari menu Accounts. Perubahan tersinkronisasi secara real-time.
Project selesai	Admin mengubah status project ke "completed" di menu Projects → PRISM menampilkan daftar semua akun terkait → Admin dapat memilih: disable all, delete all, atau keep (dengan warning).

5. Custom Services: Process Supervisors dan Container Runtime

Selain service-service yang sudah dikenal (database, message broker, storage, dll.), PRISM juga mendukung pengelolaan custom service dan process melalui tiga jenis process supervisor (PM2, Supervisor, systemd user services) dan dua container runtime (Docker, Podman). Ini memungkinkan administrator mengelola aplikasi custom, microservices, background workers, containerized services, dan proses lainnya.

5.1 Jenis Process Supervisor yang Didukung

Supervisor	Bahasa/Runtime Target	Kelebihan	Scope
PM2	Node.js (utama), Python, Go, binary apapun	Cluster mode, auto-restart, log management, startup script, ecosystem file. Sangat populer untuk aplikasi Node.js.	User-level process
Supervisor (supervisord)	Bahasa/runtime apapun	Mature, stabil, konfigurasi INI-style sederhana. Mendukung process groups, event listeners, XML-RPC API.	System-level atau user-level
systemd user services	Bahasa/runtime apapun	Terintegrasi dengan OS. Tidak memerlukan software tambahan. Mendukung dependencies, resource limits, sandboxing.	User-level (systemctl --user). Bukan system-wide.

Penting: systemd User Services, Bukan System-Wide. PRISM hanya mengelola systemd services yang berjalan di scope user (systemctl --user), bukan system-wide services. Ini karena: (1) system-wide services dikelola oleh plugin khusus PRISM (mysql, nginx, dll.), (2) user services memberikan isolasi yang lebih baik, (3) tidak memerlukan root access untuk start/stop/restart, (4) mendukung lingering (loginctl enable-linger) agar service tetap berjalan.

5.2 Bagaimana Agent Mendeteksi Process Supervisors

Prism Agent memiliki plugin khusus untuk setiap process supervisor. Saat service discovery, agent memeriksa keberadaan supervisor dan mendaftarkan proses-proses yang dikelolanya:

Supervisor	Detection	Process Enumeration
PM2	Cek apakah PM2 daemon berjalan (pm2 ping). Cek PM2 home directory (~/.pm2).	pm2 jlist menghasilkan JSON array seluruh process: id, name, status, mode (fork/cluster), instances, CPU, memory, restarts, uptime, script path, log paths.
Supervisor	Cek process supervisord running.	supervisorctl status menghasilkan

	Cek config /etc/supervisor/ atau custom path.	list program: name, state (RUNNING/STOPPED/FATAL), PID, uptime. XML-RPC API untuk data lebih detail.
systemd user	Cek systemd user instance aktif untuk setiap user yang di-configure. Gunakan systemctl --user --machine=<user>@ untuk enumerasi.	systemctl --user list-units --type=service menghasilkan list unit: name, load state, active state, sub state, description. Hanya unit yang di-enable atau running yang dilaporkan.

5.3 Container Runtime: Docker dan Podman

PRISM juga mengelola container yang berjalan melalui Docker atau Podman. Container mengikuti pola kategorisasi yang sama: setiap container (atau Compose project / Podman pod) dapat di-tag sebagai project-based atau independent.

Runtime	Detection	Keunggulan	Catatan
Docker	Cek Docker daemon via socket /var/run/docker.sock atau docker info	Ekosistem terbesar, Docker Compose untuk multi-container app, Docker Hub registry.	Memerlukan daemon (dockerd) berjalan sebagai root (atau rootless mode).
Podman	Cek podman binary, podman info	Rootless by default (lebih aman), daemonless, Pod sebagai unit grouping (mirip Kubernetes pod), OCI-compliant.	Kompatibel dengan Docker CLI dan Dockerfile. Podman Compose atau Quadlet untuk orchestration.

Container sebagai Resource Project

Dalam konteks project, container sering digunakan untuk menjalankan service pendukung yang tidak tersedia sebagai native service di server. Contoh umum:

Use Case	Contoh Container	Keterangan
Service pendukung project	redis:7-alpine, memcached:latest, elasticsearch:8	Service yang dibutuhkan project tapi tidak di-manage secara native oleh PRISM
Aplikasi project (containerized)	myapp/api:v2.1, myapp/worker:v2.1	Aplikasi project yang di-deploy sebagai container, biasanya via Compose
Development/staging tools	adminer, pgadmin4, mailhog, minio/console	Tools tambahan untuk development atau staging
AI/ML inference	ollama, vllm, tritonserver	Model serving yang lebih mudah di-deploy sebagai container
Data pipeline	apache/airflow,	Orchestrator pipeline yang

	prefecthq/prefect	berjalan sebagai container

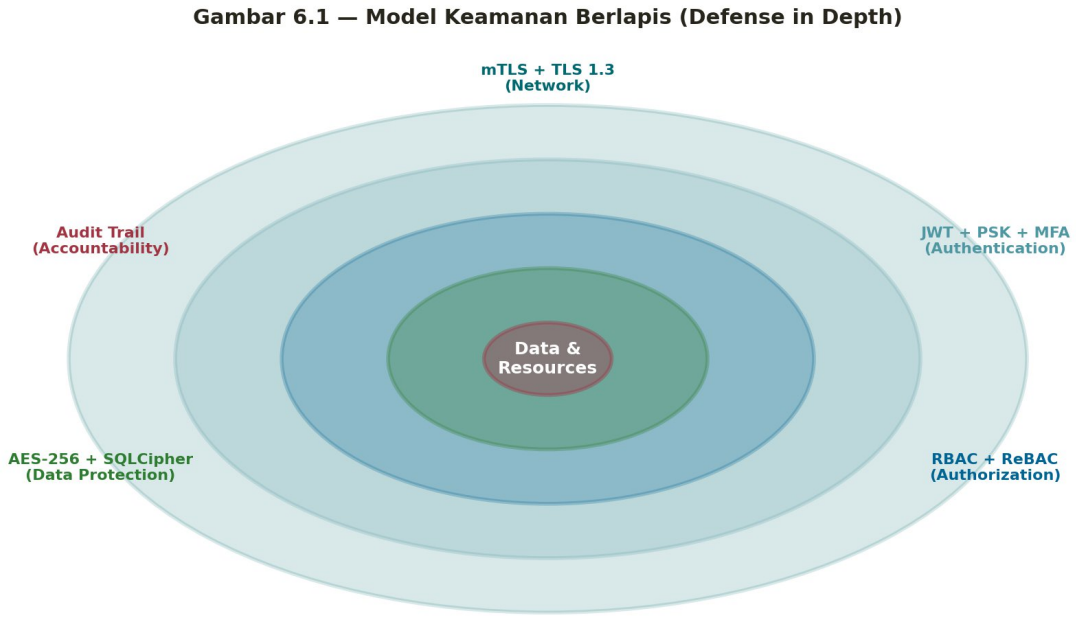
5.4 Tampilan di Frontend

Process supervisors dan container runtimes ditampilkan di Server Card masing-masing dalam kategorinya. Setiap entry menunjukkan jumlah proses/container yang dikelola.

Entry di Service List	Kategori	Status	Badge
PM2 (v5.3.0)	Process Supervisor (geekblue)	running	8 processes
Supervisor (v4.2.5)	Process Supervisor (geekblue)	running	3 programs
systemd user: deploy	Process Supervisor (geekblue)	active	5 units
Docker (v27.1.0)	Container Runtime (magenta)	running	12 containers (10 running, 2 stopped)
Podman (v5.0.2)	Container Runtime (magenta)	active	6 containers in 2 pods

Ketika entry diklik, muncul modal konfigurasi yang menampilkan daftar proses/container/unit beserta aksi yang dapat dilakukan. Detail field per modal tersedia di Lampiran A.

6. Arsitektur Keamanan



Gambar 6.1 — Model Keamanan Berlapis (Defense in Depth)

6.1 Lapisan Keamanan

Lapisan	Mekanisme	Fungsi
Transport	TLS 1.3 + Certificate Pinning	Enkripsi seluruh komunikasi dan pencegahan MITM
Identity	Mutual TLS (mTLS) + X.509 Certificate	Verifikasi identitas dua arah Hub ↔ Agent
Session	JWT (RS256) + Token Rotation	Otorisasi per request, session management
Enrollment	Pre-Shared Key + HMAC-SHA256	One-time agent onboarding
Cache Security	Valkey AUTH + TLS (opsional) + bind localhost	Proteksi akses cache layer

6.2 Hierarki Credential

Credential	Masa Berlaku	Penyimpanan
Enrollment Key (PSK)	1 jam (one-time)	Hub: SQLite (hashed). Agent: config file (sementara, dihapus setelah enrollment).
X.509 Client Certificate	1 tahun	Agent: filesystem (0600 permission). Hub: CA store.
JWT Access Token	15 menit	Agent: memory only. Frontend: httpOnly cookie.

JWT Refresh Token	24 jam (rotated)	Agent: encrypted file. Hub: Valkey + SQLite (hashed).
Hub CA Certificate	5 tahun	Hub: secure storage (HSM opsional).
Valkey AUTH password	Persistent (rotate manual)	Hub: environment variable atau secret manager.

6.3 Manajemen Sertifikat

Berikut aspek-aspek penting manajemen sertifikat yang perlu dicakup dalam operasional PRISM:

Certificate Renewal Automation

Auto-renewal sebelum expiry (misalnya 30 hari sebelumnya) tanpa intervensi manual. Agent secara otomatis mengirim CSR baru ke Hub saat sertifikat mendekati masa kedaluwarsa.

CRL atau OCSP

Mekanisme Certificate Revocation List (CRL) atau Online Certificate Status Protocol (OCSP) untuk validasi sertifikat secara real-time. Hub memelihara CRL yang dapat diakses oleh semua agent.

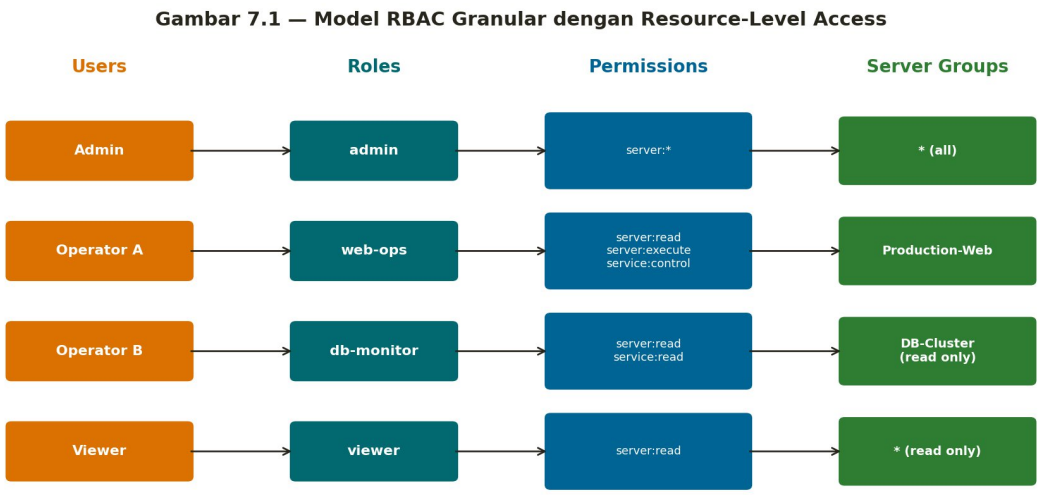
CA Key Rotation

Rencana ketika sertifikat CA Hub (masa berlaku 5 tahun) mendekati expiry — strategi migrasi CA meliputi: pembuatan CA baru, penerbitan ulang sertifikat agent secara bertahap, dan periode transisi di mana kedua CA diterima.

Certificate Monitoring Dashboard

Tampilkan status expiry sertifikat untuk semua agent di Web App, dengan notifikasi otomatis saat mendekati batas. Dashboard mencakup: daftar sertifikat mendekati expiry, sertifikat yang sudah di-revoke, dan health status keseluruhan PKI.

7. Model RBAC Granular dengan Resource-Level Access



Gambar 7.1 — Model RBAC Granular dengan Resource-Level Access

7.1 Latar Belakang & Kebutuhan

Sistem PRISM saat ini mengimplementasikan model otorisasi berbasis tiga peran statis: admin, operator, dan viewer. Model ini memadai untuk fase awal pengembangan, namun seiring pertumbuhan jumlah server yang dikelola dan kompleksitas tim operasional, model peran statis ini menunjukkan berbagai keterbatasan. Seorang operator yang seharusnya hanya bertanggung jawab atas server web produksi, misalnya, saat ini memiliki akses yang sama terhadap seluruh server termasuk server database dan server staging.

Dalam praktik keamanan modern, prinsip least privilege mengharuskan setiap pengguna hanya memiliki akses terhadap sumber daya yang benar-benar diperlukan untuk menjalankan tugasnya. Tanpa mekanisme pembatasan akses berbasis resource, risiko kesalahan operasional meningkat — seorang operator dapat secara tidak sengaja menjalankan perintah restart pada server yang bukan menjadi tanggung jawabnya. Lebih lanjut, audit dan compliance menjadi sulit ketika tidak ada jejak yang jelas mengenai siapa yang memiliki akses terhadap resource apa.

Evolusi menuju RBAC (Role-Based Access Control) yang dikombinasikan dengan ReBAC (Resource-Based Access Control) merupakan langkah yang diperlukan untuk menjawab kebutuhan ini. Model RBAC+ReBAC memungkinkan pendefinisian permission yang melekat pada resource tertentu, sehingga kontrol akses dapat diatur pada tingkat server individual, grup server, atau bahkan service tertentu dalam sebuah server. Pendekatan ini sejalan dengan pola otorisasi resource-based yang banyak diadopsi oleh platform infrastruktur modern.

Dari perspektif multi-tenancy dan skalabilitas tim, RBAC granular juga membuka kemungkinan untuk mendelegasikan manajemen server kepada tim-tim yang

berbeda tanpa memberikan akses berlebih. Tim jaringan dapat fokus pada server load balancer, tim database pada server PostgreSQL, dan tim aplikasi pada server web — masing-masing dengan set permission yang terdefinisi secara eksplisit.

Implementasi ini juga memberikan fondasi penting untuk integrasi masa depan dengan sistem SSO (Single Sign-On) dan identity provider eksternal seperti LDAP atau OIDC, di mana mapping antara group/role dari identity provider dan permission PRISM perlu dilakukan secara granular.

7.2 Desain yang Direkomendasikan

7.2.1 Model Permission

Setiap permission mengikuti format resource:action, memungkinkan pengelompokan yang intuitif dan mudah diaudit. Berikut adalah daftar permission dasar yang direkomendasikan:

Permission	Deskripsi	Kategori
server:read	Melihat informasi dan status server	Server
server:execute	Menjalankan perintah pada server	Server
server:configure	Mengubah konfigurasi server	Server
server:delete	Menghapus server dari sistem	Server
service:read	Melihat status dan log service	Service
service:configure	Mengubah konfigurasi service	Service
service:control	Start/stop/restart service	Service
user:read	Melihat daftar dan profil pengguna	User
user:manage	Membuat, mengubah, menghapus pengguna	User
role:manage	Mengelola role dan permission	Role
audit:read	Mengakses audit trail	Audit
system:configure	Mengubah pengaturan global sistem	System
backup:manage	Mengelola backup dan restore	Backup
webhook:manage	Mengelola konfigurasi webhook	Webhook

7.2.2 Database Schema Baru

Diperlukan empat tabel baru untuk mendukung model RBAC granular:

Tabel	Kolom	Tipe	Keterangan
permissions	id	INTEGER PK	Auto-increment
permissions	name	TEXT UNIQUE	Format resource:action
permissions	description	TEXT	Deskripsi permission
permissions	category	TEXT	Kategori grouping

permissions	created_at	DATETIME	Timestamp pembuatan
role_permissions	id	INTEGER PK	Auto-increment
role_permissions	role_id	INTEGER FK	Referensi ke roles.id
role_permissions	permission_id	INTEGER FK	Referensi ke permissions.id
user_server_access	id	INTEGER PK	Auto-increment
user_server_access	user_id	INTEGER FK	Referensi ke users.id
user_server_access	server_group_id	INTEGER FK	Referensi ke server_groups.id
user_server_access	permission_id	INTEGER FK	Permission spesifik untuk resource
user_server_access	granted_by	INTEGER FK	User yang memberikan akses
user_server_access	granted_at	DATETIME	Timestamp pemberian akses
server_groups	id	INTEGER PK	Auto-increment
server_groups	name	TEXT UNIQUE	Nama grup (misal: Production-Web)
server_groups	description	TEXT	Deskripsi grup server
server_groups	tags	TEXT (JSON)	Tags untuk flexible grouping

7.2.3 Contoh Policy

Berikut contoh skenario penggunaan policy berbasis resource:

- Operator A diberikan akses server:read, server:execute, dan service:control pada server group "Production-Web" (berisi 5 server web Nginx).
- Operator B diberikan akses server:read dan service:read pada semua server group — perannya hanya monitoring.
- Admin C memiliki akses penuh (wildcard) pada semua resource, termasuk user:manage dan role:manage.
- DBA D memiliki akses server:read, server:configure, dan service:configure hanya pada server group "Database-Cluster".

Setiap pemeriksaan akses dicatat dalam audit trail, termasuk informasi user, resource yang diakses, permission yang dicek, dan hasilnya (granted/denied).

7.3 Langkah Implementasi

30. Rancang dan jalankan migrasi database untuk membuat tabel permissions, role_permissions, user_server_access, dan server_groups.

31. Buat seed data permission berdasarkan daftar permission di atas dan mapping default untuk tiga role existing (admin mendapat semua permission, operator mendapat subset operasional, viewer mendapat read-only).
32. Implementasikan middleware authorization di Go backend yang melakukan pemeriksaan permission pada setiap API endpoint. Middleware harus mengecek baik role-level permission maupun resource-level access.
33. Bangun API endpoints CRUD untuk manajemen permission, role-permission mapping, server groups, dan user-server-access assignment.
34. Tambahkan halaman "Access Control" di dashboard React dengan komponen: daftar role dan permission-nya, server group management, dan user access assignment matrix.
35. Integrasikan permission check dengan audit trail — setiap akses dicatat termasuk denied attempts.
36. Tulis unit test dan integration test untuk semua skenario otorisasi, termasuk edge case seperti user tanpa server group assignment.
37. Lakukan migrasi bertahap: aktifkan fitur dengan feature flag, jalankan dalam mode audit-only (log tanpa enforce) selama 1 sprint, lalu switch ke enforcement.

7.4 Prioritas & Estimasi

Prioritas: High

Estimasi Effort: 3–4 sprint (backend middleware + database migration + frontend UI + testing)

Justifikasi: Keamanan akses merupakan fondasi yang dibutuhkan sebelum sistem digunakan oleh tim yang lebih besar. Tanpa granular access control, risiko akses tidak sah terhadap resource kritis meningkat secara signifikan.

8. Enkripsi Database SQLite at Rest dengan SQLCipher

8.1 Latar Belakang & Kebutuhan

PRISM menyimpan berbagai data sensitif dalam database SQLite termasuk credential terenkripsi (AES-256-GCM), konfigurasi server, audit trail, dan metadata operasional. Meskipun credential individual telah dienkripsi pada level aplikasi, file database itu sendiri tersimpan dalam format plaintext di filesystem. Hal ini berarti siapa pun yang memiliki akses ke file .sqlite — baik melalui akses fisik ke server, backup yang tidak terproteksi, atau kompromi sistem operasi — dapat membaca seluruh konten database menggunakan tool standar seperti sqlite3 CLI.

Enkripsi at-rest menjadi persyaratan keamanan yang semakin penting, khususnya dalam konteks compliance framework seperti ISO 27001 dan standar keamanan internal perusahaan. Meskipun PRISM belum secara eksplisit ditargetkan untuk sertifikasi tertentu, penerapan enkripsi at-rest sejak awal akan mempermudah proses compliance di masa depan dan memperkuat postur keamanan secara keseluruhan.

SQLCipher merupakan ekstensi open-source untuk SQLite yang menyediakan transparent 256-bit AES encryption pada level halaman database. Keunggulan utama SQLCipher adalah transparansi — aplikasi tetap menggunakan API SQLite standar tanpa perubahan query, sementara enkripsi dan dekripsi terjadi secara otomatis di level I/O. Binding Go tersedia melalui library go-sqlcipher yang kompatibel dengan interface database/sql standar.

Penting untuk dicatat bahwa enkripsi at-rest melindungi data ketika database tidak sedang digunakan (data on disk). Ketika database dibuka dengan key yang benar, data di memory tetap dalam format plaintext. Oleh karena itu, enkripsi at-rest harus dikombinasikan dengan kontrol akses yang ketat terhadap proses PRISM Hub dan manajemen key yang aman — yang dibahas pada Rekomendasi 3 (Secrets Management).

Dari sisi performa, SQLCipher menambahkan overhead enkripsi/dekripsi pada setiap operasi I/O. Pengujian benchmark menunjukkan overhead sekitar 5–15% tergantung pada workload dan konfigurasi cipher_page_size. Untuk workload PRISM yang didominasi oleh read operations (telemetry query, status check), overhead ini dapat diterima.

8.2 Desain yang Direkomendasikan

8.2.1 Konfigurasi SQLCipher

Berikut parameter konfigurasi yang direkomendasikan untuk integrasi SQLCipher pada PRISM:

Parameter	Nilai	Keterangan
Algoritma enkripsi	AES-256-CBC	Default SQLCipher, standar industri
cipher_page_size	4096	Match SQLite default page size untuk performa optimal
kdf_iter	256000	PBKDF2 iterations — keamanan vs

		performa
Key source	Environment variable	PRISM_DB_ENCRYPTION_KEY
Key format	Raw 256-bit hex	64 karakter hexadecimal
WAL mode	Enabled	Kompatibel penuh dengan SQLCipher
HMAC	Enabled (default)	Integrity verification per-page
Plaintext header	Disabled	Seluruh file terenkripsi termasuk header

8.2.2 Key Management

Kunci enkripsi database dikelola dengan pendekatan berikut:

- Kunci disimpan sebagai environment variable PRISM_DB_ENCRYPTION_KEY dan diset saat startup oleh secrets manager (lihat Rekomendasi 3).
- Kunci tidak pernah ditulis ke disk dalam format plaintext — hanya ada di memory proses PRISM Hub.
- Rotasi kunci dilakukan menggunakan PRAGMA rekey yang memungkinkan re-enkripsi database dengan kunci baru tanpa downtime signifikan.
- Backup database tetap dalam format terenkripsi — file backup adalah salinan byte-per-byte dari file database yang sudah terenkripsi.

8.2.3 Migration Path

Migrasi dari SQLite plain ke SQLCipher dilakukan dengan pendekatan ATTACH DATABASE:

- Buka database plain existing sebagai source.
- Buat database SQLCipher baru sebagai target dengan kunci enkripsi yang dikonfigurasi.
- Gunakan perintah ATTACH DATABASE untuk meng-copy seluruh tabel dan data dari source ke target.
- Verifikasi integritas data target dengan PRAGMA integrity_check.
- Rename file: backup database lama, ganti target menjadi database utama.
- Update konfigurasi PRISM Hub untuk menggunakan driver SQLCipher.

8.3 Langkah Implementasi

38. Tambahkan dependency go-sqlcipher (github.com/mutecomm/go-sqlcipher) ke Go module PRISM Hub sebagai pengganti driver SQLite standar.
39. Buat script migrasi yang melakukan konversi database plain ke SQLCipher secara otomatis (one-time migration) dengan validasi integritas di akhir proses.
40. Modifikasi database connection layer di Hub untuk membaca kunci dari environment variable dan menjalankan PRAGMA statements saat koneksi dibuka.

41. Implementasikan key rotation command (CLI dan API) yang menjalankan PRAGMA rekey dengan kunci baru dan memperbarui kunci di secrets manager.
42. Update prosedur backup agar memverifikasi bahwa file backup tetap terenkripsi (cek header file bukan format SQLite plain).
43. Lakukan performance benchmarking: bandingkan latency dan throughput sebelum dan sesudah enkripsi pada workload tipikal PRISM.
44. Dokumentasikan prosedur disaster recovery termasuk skenario kehilangan encryption key.

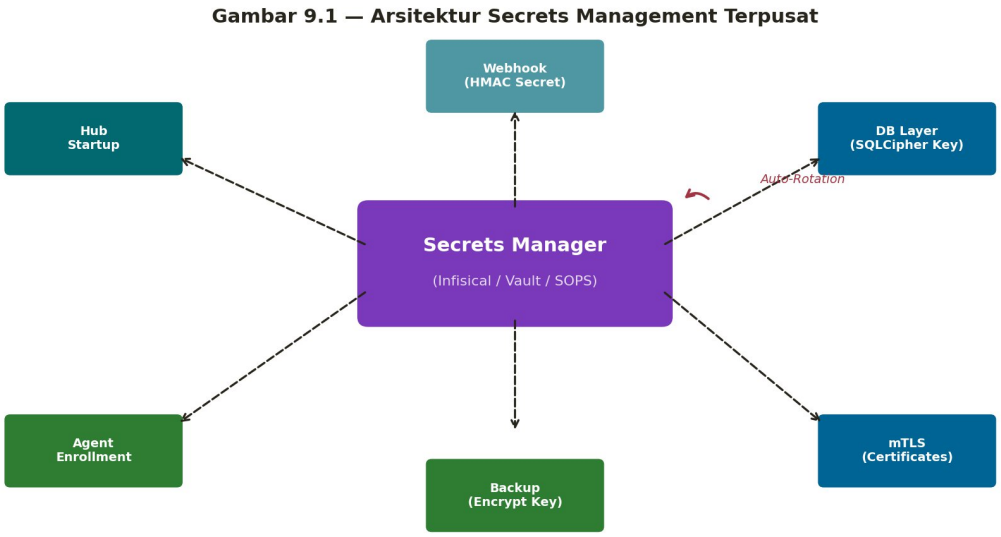
8.4 Prioritas & Estimasi

Prioritas: High

Estimasi Effort: 1–2 sprint (driver replacement + migration script + performance testing)

Justifikasi: Enkripsi at-rest merupakan baseline keamanan yang melindungi data sensitif dari akses fisik tidak sah. Implementasi relatif straightforward dengan dampak minimal terhadap kode aplikasi existing.

9. Secrets & Credential Management Terpusat



Gambar 9.1 — Arsitektur Secrets Management Terpusat

9.1 Latar Belakang & Kebutuhan

Dalam arsitektur PRISM saat ini, berbagai secrets — termasuk database encryption key, mTLS private keys, API tokens, dan credential untuk service plugins — tersebar di environment variables dan configuration files di server Hub dan Agent. Pendekatan ini menimbulkan beberapa risiko: secrets dapat terekspos melalui process listing (env vars terlihat di /proc), configuration files mungkin ter-commit ke version control secara tidak sengaja, dan tidak ada mekanisme terpusat untuk melakukan rotasi secrets secara koordinasi.

Manajemen secrets yang terfragmentasi juga menyulitkan audit — sulit untuk mengetahui secrets apa saja yang aktif, kapan terakhir dirotasi, dan siapa yang memiliki akses. Dalam skenario insiden keamanan, ketidakmampuan untuk merotasi semua secrets secara cepat dan terpusat dapat memperpanjang window of exposure.

Solusi secrets management terpusat menyediakan single source of truth untuk semua secrets, dengan fitur-fitur seperti access control, audit logging, automatic rotation, dan encryption at-rest untuk secrets itu sendiri. Beberapa opsi yang tersedia mulai dari solusi enterprise-grade seperti HashiCorp Vault hingga alternatif yang lebih ringan seperti Infisical dan Mozilla SOPS.

Untuk konteks PRISM sebagai sistem manajemen infrastruktur, secrets management yang robust sangat kritis karena PRISM menyimpan dan mengelola credential untuk mengakses server-server yang dikelolanya. Kompromi terhadap secrets PRISM berpotensi memberikan akses tidak sah ke seluruh infrastruktur yang dikelola.

Pemilihan solusi harus mempertimbangkan faktor-faktor seperti kemudahan deployment (PRISM menggunakan model self-hosted), overhead operasional, kemampuan integrasi dengan Go backend, dan kemampuan untuk mendukung baik Hub maupun Agent yang terdistribusi.

9.2 Desain yang Direkomendasikan

9.2.1 Opsi Solusi

Opsi	Solusi	Kelebihan	Kekurangan	Cocok Untuk
1	HashiCorp Vault (self-hosted)	Fitur lengkap, dynamic secrets, auto-rotation, HA mode	Kompleks, perlu dedicated server, overhead operasional tinggi	Deployment skala besar, kebutuhan compliance ketat
2	Infisical (self-hosted, MIT)	UI intuitif, mudah deploy, REST API, versioning, approval workflows	Fitur lebih terbatas dari Vault, komunitas lebih kecil	Deployment medium, preferensi simplisitas
3	Mozilla SOPS (encrypted files)	Sangat ringan, no server, Git-friendly, mendukung AGE/PGP/KMS	Tidak ada UI, rotasi manual, kurang fitur audit	Deployment kecil, tim yang familiar CLI

9.2.2 Rekomendasi Utama

Untuk deployment PRISM tipikal, direkomendasikan Infisical sebagai pilihan utama dengan SOPS sebagai fallback untuk deployment minimalis. Infisical menawarkan keseimbangan terbaik antara fitur, kemudahan operasional, dan integrasi:

- Self-hosted deployment menggunakan Docker Compose (minimal resource).
- REST API yang mudah diintegrasikan dengan Go backend PRISM.
- UI dashboard untuk manajemen secrets tanpa perlu CLI.
- Built-in versioning dan audit trail.
- Environment-based secrets organization (development, staging, production).

9.2.3 Secrets Inventory

Berikut daftar secrets yang harus dikelola secara terpusat:

Secret	Digunakan Oleh	Rotasi	Sensitivitas
Database encryption key	Hub	6 bulan	Critical — akses ke seluruh database
mTLS CA private key	Hub	1 tahun	Critical — root of trust
mTLS server certificate	Hub	90 hari (auto-renew)	High — identitas server
Agent enrollment key	Agent (initial)	Per-enrollment	High — otorisasi agent baru
Agent mTLS certificate	Agent	90 hari (auto-renew)	High — identitas agent
Webhook HMAC secret	Hub	6 bulan	Medium — integritas webhook
SMTP credentials	Hub	Password policy	Medium — email notification

Plugin API tokens	Agent	Per-plugin config	Varies — tergantung plugin
Backup encryption key	Hub	1 tahun	Critical — akses ke backup

9.2.4 Integrasi Pattern

Hub dan Agent mengambil secrets pada saat startup melalui REST API call ke Infisical (atau decrypt SOPS file). Secrets di-cache di in-memory store dan di-refresh secara periodik (configurable, default 5 menit). Pada saat secrets berubah (misalnya rotasi certificate), Hub mengirim notifikasi ke Agent via WebSocket untuk trigger re-fetch.

9.3 Langkah Implementasi

45. Deploy Infisical instance (Docker Compose) pada server Hub atau dedicated server. Konfigurasi initial admin account dan organization.
46. Buat project dan environment (production, staging) di Infisical. Migrate semua existing secrets dari env vars dan config files ke Infisical.
47. Implementasikan secrets client library di Go yang melakukan: initial fetch saat startup, in-memory caching, periodic refresh, dan graceful fallback jika Infisical tidak tersedia.
48. Integrasikan secrets client dengan database layer (encryption key), mTLS layer (certificate/key), dan webhook system (HMAC secret).
49. Implementasikan rotation workflow: API endpoint untuk trigger rotasi, update secret di Infisical, notifikasi ke komponen yang menggunakan secret tersebut.
50. Konfigurasi Agent untuk mengambil enrollment key dan mTLS certificate dari Hub melalui secure enrollment channel (existing) yang sudah terintegrasi dengan secrets manager.
51. Buat monitoring dan alerting: alert jika secrets mendekati jadwal rotasi, alert jika fetch gagal, dashboard status secrets.

9.4 Prioritas & Estimasi

Prioritas: High

Estimasi Effort: 2–3 sprint (Infisical deployment + Go client library + integration + migration)

Justifikasi: Secrets management terpusat adalah prerequisite untuk operasi keamanan yang baik. Tanpa centralized secrets, rotasi dan audit menjadi manual dan rawan kesalahan.

10. Protokol Komunikasi

10.1 WebSocket Message Types

Type	Pengirim	Deskripsi
agent.hello	Agent → Hub	Registrasi awal: PSK + CSR + metadata
agent.heartbeat	Agent → Hub	Sinyal periodik (30 detik). Payload: uptime, load average.
agent.telemetry	Agent → Hub	Data service status, metrik, resource usage
agent.discovery	Agent → Hub	Hasil service discovery (initial dan periodic rescan)
agent.command_result	Agent → Hub	Konfirmasi eksekusi command + output/error
hub.command	Hub → Agent	Perintah operasional (start/stop/restart, config update)
hub.config_update	Hub → Agent	Pembaruan konfigurasi agent (scan interval, plugin enable/disable)
hub.token_refresh	Hub → Agent	JWT token baru dari refresh request

10.2 Resilience

Mekanisme	Detail Implementasi (Go)
Auto-Reconnect	Exponential backoff: 1s, 2s, 4s, ... max 5 menit. Implementasi dengan time.Sleep dan jitter random untuk mencegah thundering herd.
Offline Buffer	Agent menyimpan telemetry ke SQLite lokal (embedded) saat koneksi terputus. Background goroutine melakukan flush saat koneksi pulih.
Idempotent Commands	Setiap command memiliki UUID. Agent menyimpan set of executed command IDs (in-memory map dengan TTL). Duplikat di-skip.
Health Check	Hub mengharapkan heartbeat tiap 30 detik. Setelah 90 detik tanpa heartbeat, agent ditandai "unreachable" di SQLite + Valkey, alert dipublish.
Graceful Shutdown	Agent menangkap SIGTERM/SIGINT, mengirim agent.goodbye ke Hub, flush buffer, close WebSocket cleanly.

10.3 Rate Limiting dan Throttling

Berikut kebijakan rate limiting yang diterapkan untuk melindungi Hub dari overload:

- Rate limit policies per endpoint: Kebijakan berbeda untuk operasi read (lebih longgar) dan write (lebih ketat).

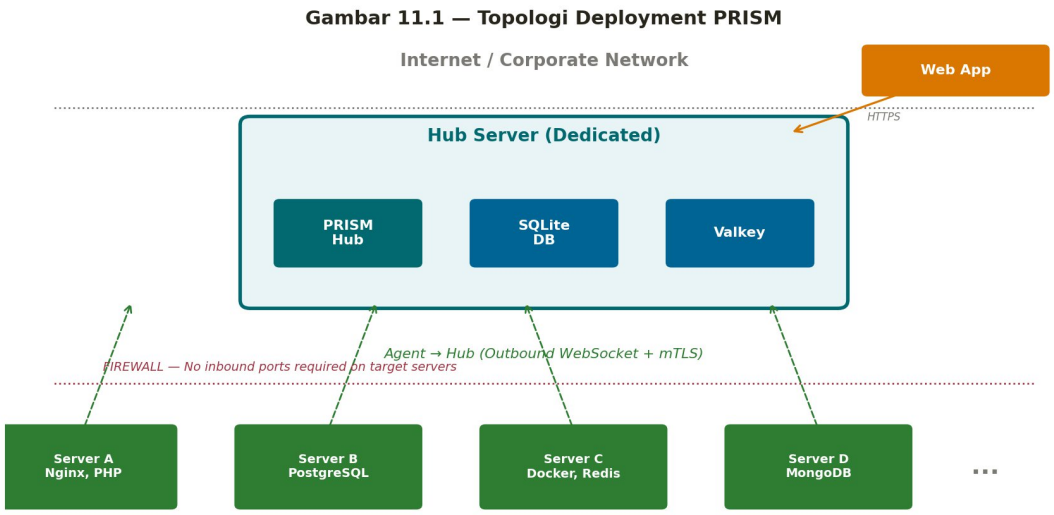
- Agent telemetry throttling: Jika agent mengirim data terlalu sering, Hub akan menginstruksikan pengurangan frekuensi melalui hub.config_update.
- Command queue depth limits: Batasi jumlah command yang dapat di-queue per agent untuk mencegah resource exhaustion.
- WebSocket message size limits: Batas ukuran maksimum per pesan WebSocket untuk mencegah memory exhaustion.
- DDoS protection di level Hub: Connection rate limiting per IP, di luar proteksi Cloudflare.

10.4 API Versioning dan Dokumentasi

Untuk memastikan kompatibilitas jangka panjang dan kemudahan integrasi:

- Strategi versioning REST API: /api/v1/ sebagai prefix untuk semua endpoint.
- Generasi spesifikasi OpenAPI/Swagger dari kode Go secara otomatis.
- Versioning skema pesan WebSocket untuk backward compatibility antara versi Hub dan Agent yang berbeda.
- Mekanisme negosiasi versi protokol Agent-Hub saat proses enrollment.

11. Deployment Topology



Gambar 11.1 — Topologi Deployment PRISM

PRISM dirancang agar fleksibel dalam berbagai topologi jaringan. Bagian ini menjelaskan skenario deployment yang didukung, termasuk jaringan LAN di belakang NAT, multi-site, dan hybrid cloud.

11.1 Prinsip: Agent-Initiated Outbound Connection

Seluruh arsitektur konektivitas PRISM bertumpu pada satu prinsip kunci: agent selalu yang menginisiasi koneksi keluar (outbound) ke Hub. Hub tidak pernah menginisiasi koneksi masuk ke agent. Prinsip ini membuat PRISM secara inheren kompatibel dengan NAT, firewall, dan jaringan yang restriktif.

Begitu koneksi WebSocket terbentuk, channel tersebut bersifat persistent dan bidirectional — Hub dapat mengirim command ke agent melalui koneksi yang sama tanpa perlu membuka port inbound di sisi agent. Heartbeat setiap 30 detik berfungsi ganda: sebagai health check sekaligus menjaga agar NAT mapping tidak expired.

11.2 Skenario Deployment

Skenario 1: Single LAN (Sederhana)

Hub dan seluruh agent berada dalam satu jaringan LAN yang sama. Ini adalah skenario paling sederhana, cocok untuk small office atau lab environment.

Aspek	Detail
Topologi	Hub di LAN (misalnya 192.168.1.10). Agent di server lain dalam LAN yang sama.
Hub URL di Agent	wss://192.168.1.10:443 atau wss://prism.local:443
NAT	Tidak ada NAT antara agent dan Hub.

Firewall	Hanya perlu allow outbound port 443 dari agent ke Hub (biasanya sudah terbuka).
Persyaratan	Hub harus memiliki IP statis atau hostname yang resolvable di LAN.

Skenario 2: Agent di LAN Belakang NAT

Skenario umum di mana agent berada di jaringan LAN kantor/site yang menggunakan NAT untuk akses internet, sementara Hub berada di cloud atau datacenter dengan public IP.

Aspek	Detail
Topologi	Hub di cloud/datacenter (public IP atau domain). Agent di LAN belakang NAT router.
Hub URL di Agent	wss://prism.smartsystem.id:443 (domain publik)
NAT Traversal	Tidak diperlukan konfigurasi khusus. Agent membuka koneksi outbound yang melewati NAT secara natural.
Firewall	Allow outbound HTTPS (port 443) dari LAN ke internet. Biasanya sudah default-allow.
Port Forwarding	TIDAK diperlukan. Agent tidak membutuhkan port inbound.
Multiple Agents per LAN	Didukung penuh. Setiap agent membuka koneksi outbound sendiri.
Aggressive NAT	Jika NAT timeout < 30 detik (misalnya carrier-grade NAT), interval heartbeat perlu diperkecil ke 15 atau 20 detik.

Skenario 3: Multi-Site / Branch Office

Aspek	Detail
Topologi	Hub terpusat di cloud/datacenter. Agent tersebar di berbagai site, masing-masing di belakang NAT/firewall site tersebut.
Konektivitas	Setiap site hanya memerlukan koneksi internet outbound ke Hub. Tidak perlu VPN site-to-site khusus untuk PRISM.
Identifikasi Site	Agent metadata menyertakan site/location identifier. Hub dapat mengelompokkan server berdasarkan site di dashboard.
Bandwidth	Minimal. Heartbeat 30 detik + telemetry 60 detik = estimasi < 5 KB/menit per agent. Ratusan agent di satu site < 500 KB/menit.
Offline Resilience	Jika link internet site terputus, agent menyimpan data ke offline buffer. Otomatis flush saat koneksi pulih.

Skenario 4: Hybrid Cloud

Aspek	Detail
Topologi	Hub di salah satu cloud atau dedicated server. Agent di on-premise (belakang NAT), di VM cloud (VPC), dan di edge device.

Cloud VM Agent	Agent di cloud VM terhubung ke Hub melalui internet atau private network (VPC peering/private link jika Hub di cloud yang sama).
On-premise Agent	Terhubung via internet outbound, sama seperti skenario NAT.
Edge Agent	Terhubung via internet (seluler/satellite/fixed). Heartbeat interval mungkin perlu disesuaikan untuk koneksi yang tidak stabil.
Security	Seluruh koneksi menggunakan mTLS + JWT terlepas dari jenis jaringan. Zero trust: tidak ada perbedaan trust level antara cloud dan on-premise agent.

12. Zero Trust Network dan Secure Tunnel

Meskipun arsitektur outbound-only PRISM sudah cukup untuk sebagian besar skenario, organisasi dengan kebijakan keamanan ketat mungkin memerlukan lapisan tambahan. Bagian ini menjelaskan integrasi PRISM dengan pendekatan Zero Trust Network Access (ZTNA) dan secure tunnel.

12.1 Prinsip Zero Trust di PRISM

Prinsip Zero Trust	Implementasi di PRISM
Never trust, always verify	Setiap request (agent maupun admin) harus membawa bukti identitas: mTLS certificate + JWT token. Tidak ada implicit trust berdasarkan posisi jaringan.
Least privilege access	JWT token berisi scope spesifik per agent. Agent hanya bisa mengakses resource yang relevan dengan server-nya. RBAC untuk admin di Web App.
Assume breach	Certificate dapat di-revoke secara instan. JWT berumur pendek (15 menit). Compromised agent tidak mempengaruhi agent lain. Audit log immutable.
Verify explicitly	Setiap command dari Hub ke agent divalidasi ulang: signature check, agent ID match, command authorization. Agent memverifikasi bahwa command berasal dari Hub yang sah via mTLS.
Micro-segmentation	Setiap agent adalah unit terisolasi. Agent A tidak bisa berkomunikasi dengan Agent B. Semua komunikasi harus melalui Hub yang menegakkan policy.

12.2 Kapan Memerlukan Secure Tunnel?

Skenario	Masalah	Solusi Tunnel
Firewall sangat restriktif	Outbound WebSocket (bahkan port 443) diblokir oleh corporate firewall atau DPI yang memblokir WebSocket upgrade.	Cloudflare Tunnel (cloudflared) atau WireGuard melewati restriksi.
Compliance requirement	Regulasi mengharuskan seluruh traffic melewati VPN atau encrypted tunnel tertentu.	WireGuard sebagai transport layer tambahan.
Hub tidak memiliki public IP	Hub berada di jaringan private/on-premise tanpa public IP, tetapi agent tersebar di luar jaringan.	Cloudflare Tunnel meng-expose Hub tanpa public IP.
Defense in depth	Organisasi menginginkan enkripsi berlapis.	WireGuard + PRISM mTLS = double encryption.
Network segmentation	Agent di segmen jaringan yang berbeda, dipisahkan oleh firewall internal yang ketat.	WireGuard mesh atau Cloudflare WARP menghubungkan segmen.

12.3 Opsi 1: Cloudflare Tunnel (cloudflared)

Cloudflare Tunnel memungkinkan Hub di-expose ke internet tanpa membuka port inbound dan tanpa public IP. Tunnel daemon (cloudflared) berjalan di sisi Hub dan membuat koneksi outbound ke edge network Cloudflare.

Arsitektur dengan Cloudflare Tunnel

Komponen	Lokasi	Fungsi
cloudflared daemon	Co-located dengan Hub	Membuat persistent tunnel dari Hub ke Cloudflare edge. Hub API dan WebSocket di-route melalui tunnel.
Cloudflare Edge	Cloudflare network (global)	Menerima koneksi dari agent, melakukan TLS termination (opsional), forward ke Hub via tunnel.
Agent	Di mana saja (LAN, cloud, edge)	Terhubung ke prism.smartsystem.id yang di-resolve ke Cloudflare edge.

Keuntungan Cloudflare Tunnel

Aspek	Detail
Tanpa Public IP	Hub tidak memerlukan public IP. cloudflared membuat koneksi outbound ke Cloudflare.
Zero-config DNS	Cloudflare otomatis mengatur DNS record untuk domain tunnel.
DDoS Protection	Traffic melewati Cloudflare edge yang menyediakan DDoS mitigation, WAF, dan rate limiting.
Cloudflare Access (ZTNA)	Dapat diintegrasikan dengan Cloudflare Access untuk identity-aware proxy.
WebSocket Support	Cloudflare Tunnel mendukung WebSocket passthrough secara penuh.
Audit & Visibility	Cloudflare dashboard menyediakan log koneksi, analytics traffic, dan security event.

Konfigurasi Cloudflare Tunnel untuk PRISM

Step	Aksi	Detail
1	Install cloudflared di server Hub	Download binary atau install via package manager. cloudflared login untuk autentikasi ke akun Cloudflare.
2	Buat tunnel	cloudflared tunnel create prism-hub. Menghasilkan tunnel ID dan credentials file.
3	Konfigurasi routing	config.yml: route traffic dari prism.smartsystem.id ke localhost:443. Pastikan opsi noTLSVerify: false agar mTLS tetap end-to-end.
4	Jalankan sebagai service	cloudflared service install. Tunnel berjalan sebagai systemd service, auto-start saat boot.
5	Konfigurasi agent	Agent cukup menggunakan hub_url:

		wss://prism.smartsystem.id:443. Tidak perlu konfigurasi tunnel di sisi agent.
6	Opsional: Cloudflare Access	Tambahkan Access Application untuk Web App. Admin harus autentikasi via SSO (Google, Okta, Azure AD).

mTLS End-to-End melalui Cloudflare Tunnel: Cloudflare Tunnel mendukung passthrough mode di mana TLS tidak di-terminate di edge. Ini memastikan mTLS antara agent dan Hub tetap end-to-end. Cloudflare edge hanya meneruskan encrypted traffic tanpa melihat payload.

12.4 Opsi 2: WireGuard VPN

WireGuard adalah modern VPN protocol yang ringan, cepat, dan mudah dikonfigurasi. WireGuard membentuk encrypted tunnel di layer 3 yang menghubungkan agent dan Hub dalam satu virtual private network.

Arsitektur dengan WireGuard

Komponen	Lokasi	Fungsi
WireGuard Server	Co-located dengan Hub (atau dedicated gateway)	Menerima koneksi WireGuard dari agent. Assign IP address dalam subnet virtual (10.100.0.0/16).
WireGuard Client	Di setiap server yang menjalankan agent	Membuat tunnel ke WireGuard server. Agent berkomunikasi dengan Hub melalui IP virtual.
Hub	Mendengarkan di IP virtual WireGuard (10.100.0.1)	Menerima koneksi WebSocket dari agent melalui tunnel.

Model Deployment WireGuard

Model	Topologi	Use Case
Star (Hub-and-Spoke)	WireGuard server di Hub. Semua agent terhubung ke satu server.	Paling sederhana. Cocok untuk < 100 agent.
Mesh (via WireGuard tools)	Setiap node saling terhubung. Menggunakan tools seperti wg-mesh, netbird, atau tailscale.	Untuk skenario peer-to-peer.
Gateway per Site	Satu WireGuard gateway per site/branch. Agent di site terhubung ke gateway lokal.	Untuk multi-site dengan banyak agent per site.

Keuntungan WireGuard

Aspek	Detail
Performa	Kernel-space implementation, overhead minimal (< 5% throughput loss). Latency sangat rendah.
Kesederhanaan	Konfigurasi minimalis: sepasang public/private key, endpoint, allowed IPs.

Roaming	Agent yang berpindah IP tetap terhubung. WireGuard otomatis re-establish tunnel.
Stealth	WireGuard tidak merespons paket dari IP yang tidak dikenal. Port terlihat "closed".
Cross-platform	Tersedia untuk Linux (kernel module), Windows, macOS, Android, iOS, FreeBSD.
Double Encryption	Traffic PRISM (mTLS + TLS 1.3) berjalan di dalam WireGuard tunnel.

Konfigurasi WireGuard untuk PRISM

Step	Server (Hub side)	Client (Agent side)
1. Generate keys	wg genkey | tee server_private.key | wg pubkey > server_public.key	wg genkey | tee agent_private.key | wg pubkey > agent_public.key
2. Interface config	[Interface] Address = 10.100.0.1/16, ListenPort = 51820, PrivateKey = <server_private>	[Interface] Address = 10.100.0.x/32, PrivateKey = <agent_private>
3. Peer config	[Peer] PublicKey = <agent_public>, AllowedIPs = 10.100.0.x/32	[Peer] PublicKey = <server_public>, Endpoint = hub-public-ip:51820, AllowedIPs = 10.100.0.1/32, PersistentKeepalive = 25
4. Start	wg-quick up wg0 (atau systemctl enable wg-quick@wg0)	wg-quick up wg0
5. Agent config	---	hub_url: wss://10.100.0.1:443 (IP virtual Hub di dalam tunnel)

PersistentKeepalive untuk NAT Traversal: Parameter PersistentKeepalive = 25 pada client config mengirim keepalive packet setiap 25 detik, menjaga NAT mapping tetap aktif.

12.5 Perbandingan: Cloudflare Tunnel vs WireGuard

Aspek	Cloudflare Tunnel	WireGuard
Setup complexity	Rendah. Install cloudflared, buat tunnel, selesai.	Sedang. Perlu generate key pair per agent, distribusi public key.
Konfigurasi agent	Tidak ada perubahan. Agent tetap konek ke domain publik.	Agent harus install WireGuard client dan dikonfigurasi tunnel.
Public IP requirement	Hub tidak perlu public IP.	WireGuard server perlu public IP atau accessible endpoint.
Overhead / Latency	Sedikit lebih tinggi (traffic via Cloudflare edge).	Minimal (kernel-space, direct tunnel).
Biaya	Gratis untuk Cloudflare Tunnel. Access gratis hingga 50 user.	Gratis dan open-source.

Ekstra fitur	DDoS protection, WAF, Cloudflare Access, analytics.	Pure encrypted tunnel.
Offline / air-gapped	Tidak bisa. Memerlukan koneksi ke Cloudflare.	Bisa. WireGuard berjalan peer-to-peer.
Scalability	Cloudflare menangani routing dan load balancing.	Perlu manajemen peer config per agent.
Vendor dependency	Bergantung pada Cloudflare (SaaS).	Tidak ada vendor dependency. Fully self-hosted.

12.6 Rekomendasi Pemilihan

Skenario	Rekomendasi	Alasan
Hub di cloud, agent tersebar, mau cepat setup	Cloudflare Tunnel	Zero config di sisi agent. Bonus DDoS protection dan Cloudflare Access.
Hub tanpa public IP, perlu ZTNA	Cloudflare Tunnel + Access	Tunnel menghilangkan kebutuhan public IP. Access menambahkan SSO layer.
Environment air-gapped atau regulated	WireGuard	Tidak ada dependency ke layanan pihak ketiga.
Perlu performa tunnel maksimal	WireGuard	Kernel-space, minimal overhead, latency rendah.
Multi-site dengan banyak agent per site	WireGuard (gateway per site)	Satu gateway per site, mengurangi jumlah tunnel ke Hub.
Tidak ada kebutuhan khusus	Tanpa tunnel (default PRISM)	Arsitektur outbound-only PRISM sudah NAT-friendly dan aman (mTLS + JWT).

Catatan Penting: Tunnel (Cloudflare atau WireGuard) adalah lapisan transport tambahan. Keamanan inti PRISM (mTLS + JWT + enrollment) tetap berjalan di atasnya. Ini berarti jika tunnel ter-compromise, komunikasi PRISM tetap terenkripsi dan terotentikasi. Defense in depth: dua lapisan enkripsi independen.

13. Observability dan Monitoring Internal

Selain monitoring terhadap target services, PRISM memerlukan self-monitoring untuk komponen internalnya sendiri. Bagian ini mendefinisikan metrik, logging, dan integrasi observability.

13.1 Hub Health Metrics

Hub perlu mengexpose metrik internal meliputi:

- CPU dan memory usage Hub process
- Goroutine count dan leak detection
- Jumlah koneksi WebSocket aktif dan message throughput
- SQLite query latency (p50, p95, p99)
- Valkey hit/miss ratio dan connection pool utilization

13.2 Agent Self-Reporting

Agent perlu melaporkan health-nya sendiri, terpisah dari telemetri service yang dimonitor:

- Memory usage dan CPU utilization agent process
- Buffer utilization (persentase penggunaan offline buffer)
- Plugin errors — jumlah dan jenis error per plugin
- Uptime dan restart count

13.3 Structured Logging Standard

Penerapan structured logging dengan correlation ID yang dapat menelusuri sebuah request dari frontend → Hub API → WebSocket → Agent → result:

- Gunakan slog (Go 1.21+) dengan definisi log level dan format secara eksplisit.
- Standar field wajib: timestamp, level, correlation_id, component, message.
- Log level guidelines: DEBUG (development), INFO (normal operation), WARN (recoverable issues), ERROR (action needed).

13.4 Integrasi OpenTelemetry

PRISM mendukung OpenTelemetry secara opsional untuk export traces dan metrics ke sistem eksternal seperti Grafana atau Prometheus. Ini memungkinkan integrasi dengan ekosistem observability yang sudah ada di organisasi pengguna.

14. Health Check Endpoints (Liveness, Readiness, Startup)

14.1 Latar Belakang & Kebutuhan

PRISM Hub dan Agent saat ini tidak memiliki endpoint health check yang terstruktur. Untuk mengetahui apakah Hub berjalan normal, administrator harus mengecek apakah proses berjalan (process check) atau mencoba mengakses API endpoint. Pendekatan ini tidak membedakan antara berbagai kondisi: proses bisa berjalan namun database tidak accessible, atau WebSocket listener tidak aktif.

Health check endpoints yang terstandarisasi memisahkan tiga aspek kesehatan yang berbeda. Pertama, liveness menunjukkan apakah proses masih berjalan dan tidak mengalami deadlock — jika liveness gagal, proses perlu di-restart. Kedua, readiness menunjukkan apakah aplikasi siap menerima traffic — jika readiness gagal, load balancer harus menghentikan pengiriman request. Ketiga, startup menunjukkan apakah initialization telah selesai — penting untuk aplikasi yang membutuhkan waktu startup lama.

Meskipun PRISM saat ini tidak berjalan di Kubernetes, implementasi health check endpoints mengikuti pola yang sudah menjadi standar industri. Pattern ini tidak hanya berguna untuk container orchestration tetapi juga untuk monitoring tools (Prometheus, Uptime Kuma), load balancer (Nginx, HAProxy), dan process manager (systemd). Dengan endpoint health check yang terstandarisasi, integrasi dengan tool-tool tersebut menjadi straightforward.

Dari perspektif operational visibility, health check endpoints memberikan informasi diagnostik yang berharga. Response body menyertakan status setiap dependency (database, cache, WebSocket) beserta latency-nya, memungkinkan deteksi dini masalah sebelum berdampak pada end-user.

Untuk Agent, health check readiness juga penting karena menunjukkan apakah agent berhasil terhubung ke Hub dan mampu men-discover service di server. Jika readiness agent gagal, Hub dapat menandai server sebagai unavailable di dashboard tanpa harus menunggu timeout yang lebih lama.

14.2 Desain yang Direkomendasikan

14.2.1 Hub Endpoints

Komponen	Endpoint	Dependency Check	Response Time Target
Hub	GET /health/live	Tidak ada (proses saja)	< 10ms
Hub	GET /health/ready	SQLite, Valkey, WebSocket listener	< 5s (timeout)
Hub	GET /health/startup	Semua initialization complete	< 10ms (cached)

14.2.2 Agent Endpoints

Komponen	Endpoint	Dependency Check	Response

			Time Target
Agent	GET /health/live	Tidak ada (proses saja)	< 10ms
Agent	GET /health/ready	Hub connection, service discovery	< 5s (timeout)

14.2.3 Response Format

Setiap health check endpoint mengembalikan JSON response dengan format berikut:

Field	Tipe	Keterangan
status	string	healthy | degraded | unhealthy
checks	array	Detail per dependency check
checks[].name	string	Nama dependency (sqlite, valkey, websocket)
checks[].status	string	healthy | unhealthy
checks[].duration_ms	number	Waktu eksekusi check dalam milidetik
checks[].message	string	Pesan error jika unhealthy (opsional)
timestamp	string	ISO 8601 timestamp
version	string	Versi PRISM yang sedang berjalan
uptime_seconds	number	Durasi proses telah berjalan

14.2.4 Concurrent Dependency Checks

Penentuan status keseluruhan mengikuti logika berikut:

- healthy: Semua dependency checks berhasil.
- degraded: Setidaknya satu dependency check gagal, namun sistem masih bisa beroperasi (contoh: Valkey down, fallback ke direct DB query).
- unhealthy: Dependency kritis gagal (contoh: SQLite tidak accessible), sistem tidak dapat melayani request.

Liveness endpoint selalu mengembalikan healthy selama proses berjalan — tidak melakukan dependency check. Readiness endpoint menjalankan semua dependency checks secara concurrent dengan timeout 5 detik.

14.3 Langkah Implementasi

52. Implementasikan health check framework di Go: interface HealthChecker dengan method Check() yang mengembalikan status dan duration per dependency.
53. Buat implementasi checker untuk setiap dependency: SQLiteChecker (PRAGMA quick_check), ValkeyChecker (PING), WebSocketChecker (listener active).
54. Implementasikan endpoint handler yang menjalankan checker secara concurrent (goroutines) dengan context timeout dan mengagregasi hasilnya.

55. Registrasi endpoint pada HTTP router Hub (/health/live, /health/ready, /health/startup) dan Agent (/health/live, /health/ready).
56. Integrasikan health status dengan dashboard: tambahkan indicator per server/agent yang menampilkan last health status, dan buat alert jika status berubah dari healthy.
57. Implementasikan graceful shutdown sequence: set readiness ke unhealthy → drain existing connections (timeout 30 detik) → shutdown proses.
58. Konfigurasi monitoring tools (Prometheus, systemd, atau Uptime Kuma) untuk menggunakan health check endpoints.

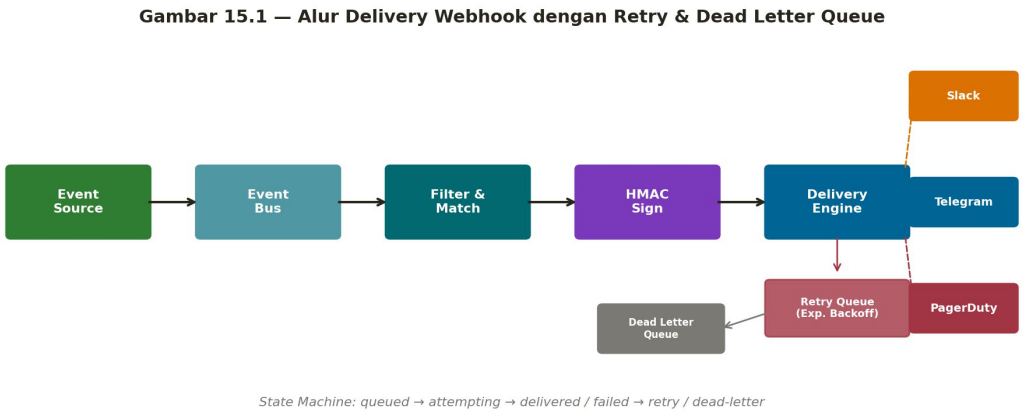
14.4 Prioritas & Estimasi

Prioritas: High

Estimasi Effort: 1 sprint (framework + checker implementations + endpoint registration + dashboard integration)

Justifikasi: Health check endpoints adalah fondasi observability yang relatif sederhana namun memberikan visibilitas operasional yang signifikan. Implementasi cepat dengan dampak besar.

15. Sistem Webhook & Notifikasi Eksternal



Gambar 15.1 — Alur Delivery Webhook dengan Retry & Dead Letter Queue

15.1 Latar Belakang & Kebutuhan

PRISM saat ini beroperasi sebagai sistem yang relatif tertutup — event-event penting seperti server down, service error, agent disconnect, atau certificate yang akan expire hanya dapat diketahui melalui dashboard web. Tidak ada mekanisme untuk mendorong notifikasi secara proaktif ke sistem atau channel eksternal. Hal ini berarti tim operasional harus secara aktif memantau dashboard untuk mengetahui insiden, yang tidak efektif terutama di luar jam kerja.

Webhook merupakan pola integrasi standar yang memungkinkan sistem mengirimkan HTTP callback ke endpoint yang didaftarkan ketika event tertentu terjadi. Berbeda dengan polling, webhook memberikan notifikasi near-real-time dengan overhead jaringan yang minimal. Hampir semua platform modern — dari GitHub hingga Stripe — menggunakan webhook sebagai mekanisme integrasi utama.

Implementasi sistem webhook memerlukan pertimbangan yang matang terhadap reliability (garansi delivery), security (verifikasi bahwa notifikasi berasal dari PRISM), dan observability (monitoring kesehatan delivery). Desain yang baik harus mengakomodasi skenario kegagalan seperti endpoint yang tidak responsif, rate limiting dari sisi penerima, dan data consistency.

Dari perspektif operasional, sistem webhook memungkinkan integrasi dengan ekosistem tools yang sudah digunakan tim: Slack untuk notifikasi real-time, Telegram untuk alerting mobile, PagerDuty untuk on-call escalation, dan email SMTP untuk stakeholder non-teknis. Webhook juga membuka kemungkinan untuk integrasi dengan sistem ticketing (Jira, ServiceNow) dan custom automation.

Kebutuhan ini menjadi semakin mendesak seiring pertumbuhan jumlah server yang dikelola PRISM, di mana monitoring manual melalui dashboard menjadi tidak scalable.

15.2 Desain yang Direkomendasikan

15.2.1 Event Types

Event Type	Trigger	Severity Default
server.down	Server tidak merespons setelah threshold	Critical
server.recovered	Server kembali online	Info
service.error	Service melaporkan error state	Warning
service.stopped	Service berhenti (unexpected)	Critical
agent.disconnected	WebSocket connection ke agent terputus	Warning
agent.connected	Agent berhasil terhubung	Info
certificate.expiring	Certificate akan expire dalam 14 hari	Warning
certificate.expired	Certificate sudah expired	Critical
backup.failed	Backup job gagal	Critical
backup.completed	Backup job selesai	Info
drift.detected	Configuration drift terdeteksi	Warning
health.degraded	Health check menunjukkan status degraded	Warning

15.2.2 Database Schema

Tabel	Kolom	Tipe	Keterangan
webhook_subscriptions	id	INTEGER PK	Auto-increment
webhook_subscriptions	name	TEXT	Nama deskriptif subscription
webhook_subscriptions	url	TEXT	Target endpoint URL (HTTPS)
webhook_subscriptions	secret	TEXT	HMAC-SHA256 signing secret
webhook_subscriptions	events	TEXT (JSON)	Array event types yang di-subscribe
webhook_subscriptions	is_active	BOOLEAN	Enable/disable tanpa delete
webhook_subscriptions	failure_count	INTEGER	Consecutive failure counter
webhook_subscriptions	max_retries	INTEGER	Default 5
webhook_subscriptions	created_by	INTEGER FK	User yang membuat
webhook_subscriptions	created_at	DATETIME	Timestamp pembuatan
webhook_deliveries	id	INTEGER PK	Auto-increment
webhook_deliveries	subscription_id	INTEGER FK	Referensi ke

			subscriptions
webhook_deliveries	event_type	TEXT	Tipe event yang dikirim
webhook_deliveries	payload	TEXT (JSON)	Request body yang dikirim
webhook_deliveries	response_status	INTEGER	HTTP status code respons
webhook_deliveries	response_body	TEXT	Response body (truncated)
webhook_deliveries	attempt	INTEGER	Percobaan ke-n
webhook_deliveries	delivered_at	DATETIME	Timestamp delivery attempt
webhook_deliveries	status	TEXT	success/failed/pending

15.2.3 Delivery Engine

Mekanisme delivery menggunakan async dispatch pattern dengan goroutine pool:

- Setiap event yang terjadi di-publish ke internal event bus (Go channel).
- Delivery dispatcher menerima event, mencari semua subscription yang match, dan mengirimkan ke goroutine pool.
- Setiap delivery ditandatangani dengan HMAC-SHA256 menggunakan secret milik subscription. Header X-PRISM-Signature berisi hash untuk verifikasi.
- Retry menggunakan exponential backoff: 1 detik, 2 detik, 4 detik, 8 detik, 16 detik (maksimal 5 retry).
- Circuit breaker: setelah 10 consecutive failures, subscription otomatis di-disable dan notifikasi dikirim ke admin.
- Dead letter queue: delivery yang gagal setelah semua retry disimpan untuk investigasi manual.

15.2.4 Security

Setiap delivery request disertai header X-PRISM-Signature yang berisi HMAC-SHA256 dari payload menggunakan secret milik subscription. Penerima webhook memverifikasi signature ini untuk memastikan request berasal dari PRISM yang legitimate. Endpoint target harus menggunakan HTTPS untuk mencegah tampering payload dalam transit.

15.3 Langkah Implementasi

59. Implementasikan internal event bus menggunakan Go channel dengan buffered queue (kapasitas configurable, default 10.000 events).
60. Buat tabel webhook_subscriptions dan webhook_deliveries, termasuk index pada event_type dan subscription_id untuk query performance.
61. Implementasikan delivery engine: goroutine pool, HMAC signing, retry logic dengan exponential backoff, dan circuit breaker.

62. Buat API endpoints: CRUD subscription, list deliveries (with filter), retry failed delivery, test subscription (send test event).
63. Bangun halaman "Integrations" di dashboard: form subscription baru, daftar subscription aktif, delivery log dengan status indicator, tombol test dan retry.
64. Tambahkan event publishing pada semua komponen yang menghasilkan event: server monitor, service manager, agent connection handler, certificate checker, backup runner.
65. Dokumentasikan format payload untuk setiap event type dan buat contoh integrasi untuk Slack, Telegram, dan email.

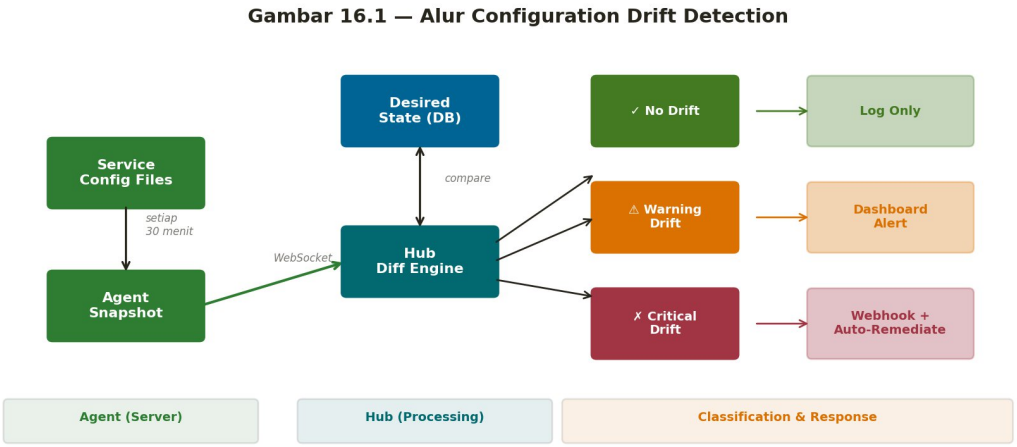
15.4 Prioritas & Estimasi

Prioritas: Medium

Estimasi Effort: 2–3 sprint (event bus + delivery engine + API + frontend + integrasi)

Justifikasi: Webhook meningkatkan responsiveness tim operasional secara signifikan. Prioritas medium karena monitoring manual melalui dashboard masih memungkinkan, namun tidak scalable untuk deployment besar.

16. Configuration Drift Detection



Gambar 16.1 — Alur Configuration Drift Detection

16.1 Latar Belakang & Kebutuhan

PRISM mengelola konfigurasi berbagai service di server-server yang dipantau, termasuk Nginx, PostgreSQL, Redis, dan service lainnya. Saat ini, konfigurasi desired state disimpan di database Hub dan diterapkan melalui Agent. Namun, tidak ada mekanisme untuk mendeteksi apakah konfigurasi aktual di server masih sesuai dengan desired state yang tercatat.

Configuration drift — perbedaan antara konfigurasi yang seharusnya (desired state) dan konfigurasi yang sebenarnya berjalan (actual state) — merupakan masalah umum dalam manajemen infrastruktur. Drift dapat terjadi karena berbagai sebab: perubahan manual oleh administrator yang tidak melalui PRISM, update software yang mengubah configuration files, atau error pada proses deployment.

Tanpa deteksi drift, sistem beroperasi dengan asumsi bahwa konfigurasi aktual sesuai dengan yang tercatat, yang dapat menyesatkan. Seorang administrator yang melihat konfigurasi Nginx di dashboard PRISM mungkin yakin bahwa worker_processes diset ke 4, sementara kenyataannya sudah diubah secara manual menjadi 8 oleh seseorang yang mengakses server langsung. Ketidakkonsistenan ini dapat menyebabkan masalah saat troubleshooting dan membuat dokumentasi konfigurasi menjadi tidak reliable.

Deteksi drift juga penting dari perspektif keamanan dan compliance. Perubahan konfigurasi yang tidak terotorisasi bisa menjadi indikator adanya kompromi keamanan. Dengan drift detection, perubahan tersebut dapat dideteksi secara otomatis dan memicu alert yang meminta investigasi.

Implementasi drift detection membutuhkan kerja sama antara Agent (yang memiliki akses ke konfigurasi aktual di server) dan Hub (yang menyimpan desired state dan melakukan perbandingan). Pendekatan yang direkomendasikan adalah Agent secara periodik mengambil snapshot konfigurasi dan mengirimkannya ke Hub untuk diff analysis.

16.2 Desain yang Direkomendasikan

Arsitektur

Sistem drift detection bekerja dalam tiga tahap:

- Snapshot Collection: Agent mengambil snapshot konfigurasi aktual dari setiap service yang dikelola secara periodik (default setiap 30 menit, configurable per-service).
- Diff Analysis: Hub membandingkan snapshot terbaru dengan desired state yang tersimpan di database. Perbandingan dilakukan pada level key-value untuk structured configs dan line-by-line untuk file-based configs.
- Reporting & Remediation: Drift yang terdeteksi diklasifikasikan berdasarkan severity, ditampilkan di dashboard, dan memicu webhook notification jika dikonfigurasi.

Database Schema

Tabel	Kolom	Tipe	Keterangan
configuration_snapshots	id	INTEGER PK	Auto-increment
configuration_snapshots	server_id	INTEGER FK	Server yang di-snapshot
configuration_snapshots	service_name	TEXT	Nama service (nginx, postgresql, dll)
configuration_snapshots	config_hash	TEXT	SHA-256 hash dari konfigurasi
configuration_snapshots	config_data	TEXT (JSON)	Konfigurasi aktual terstruktur
configuration_snapshots	captured_at	DATETIME	Timestamp pengambilan snapshot
drift_events	id	INTEGER PK	Auto-increment
drift_events	server_id	INTEGER FK	Server yang mengalami drift
drift_events	service_name	TEXT	Service yang drift
drift_events	severity	TEXT	critical / warning / info
drift_events	drift_details	TEXT (JSON)	Detail perbedaan (key, expected, actual)
drift_events	status	TEXT	detected / acknowledged / remediated / ignored
drift_events	detected_at	DATETIME	Timestamp deteksi
drift_events	resolved_at	DATETIME	Timestamp resolusi (nullable)

drift_events	resolved_by	INTEGER FK	User yang menyelesaikan (nullable)

Severity Classification

Severity	Kriteria	Contoh	Aksi Otomatis
Critical	Perubahan pada parameter security atau parameter yang mempengaruhi availability	SSL disabled, listen port berubah, authentication dimatikan	Alert + block auto-remediate (perlu approval)
Warning	Perubahan pada parameter performa atau operasional	Worker count berubah, timeout diubah, log level berubah	Alert + opsi auto-remediate
Info	Perubahan pada parameter non-kritis	Comment berubah, formatting berubah	Log only

16.3 Langkah Implementasi

66. Implementasikan snapshot collector di Agent: untuk setiap plugin service, tambahkan method GetConfigSnapshot() yang mengembalikan konfigurasi aktual dalam format terstruktur.
67. Buat scheduled job di Agent yang menjalankan snapshot collection secara periodik dan mengirimkan hasilnya ke Hub melalui WebSocket.
68. Implementasikan diff engine di Hub yang membandingkan snapshot dengan desired state. Engine harus mendukung comparison berbasis key-value dan line-by-line.
69. Buat tabel configuration_snapshots dan drift_events. Implementasikan retention policy untuk snapshots (default 30 hari).
70. Bangun komponen dashboard: drift timeline per server, diff viewer (side-by-side expected vs actual), dan bulk remediation interface.
71. Integrasikan drift detection dengan sistem webhook — drift.detected event memicu notifikasi ke channel yang dikonfigurasi.
72. Implementasikan auto-remediation workflow: untuk drift dengan severity warning, sediakan opsi one-click revert ke desired state dengan confirmation dialog.

16.4 Prioritas & Estimasi

Prioritas: Medium

Estimasi Effort: 2–3 sprint (agent snapshot + hub diff engine + dashboard + auto-remediation)

Justifikasi: Drift detection meningkatkan reliability dan auditability konfigurasi. Prioritas medium karena dampaknya signifikan namun tidak sekritis keamanan akses dan enkripsi.

17. Strategi Backup dan Disaster Recovery

Untuk sistem yang mengelola infrastruktur server secara terpusat, strategi backup dan disaster recovery sangat kritikal.

17.1 SQLite Backup Strategy

Mengingat SQLite adalah database berbasis file, strategi backup didefinisikan secara jelas:

- Metode Backup: Gunakan SQLite .backup API atau file-level copy saat WAL checkpoint untuk memastikan konsistensi data.
- Jadwal Otomatis: Backup inkremental setiap jam (hourly) dan backup penuh setiap hari (daily full backup).
- Penyimpanan: Simpan backup di object storage (MinIO/S3) yang terpisah dari server Hub utama.

17.2 Definisi RPO dan RTO

Recovery Point Objective (RPO) dan Recovery Time Objective (RTO) untuk tier data yang berbeda:

Tier Data	RPO	RTO	Keterangan
Akun & Konfigurasi	< 1 jam	< 30 menit	Data kritis, tidak boleh hilang
Audit Log	< 4 jam	< 1 jam	Penting untuk compliance
Telemetri	< 24 jam	< 2 jam	Dapat ditoleransi kehilangan parsial

17.3 Hub Failover Plan

Skenario ketika Hub server mengalami kegagalan:

- Warm Standby: Pertimbangkan penggunaan warm standby dengan replikasi SQLite WAL menggunakan tools seperti Litestream untuk streaming real-time ke S3.
- Failover Otomatis: Definisikan mekanisme deteksi kegagalan dan prosedur failover ke standby server.

17.4 Valkey Persistence

Strategi persistence RDB+AOF untuk Valkey cache guna meminimalkan kehilangan cache saat restart.

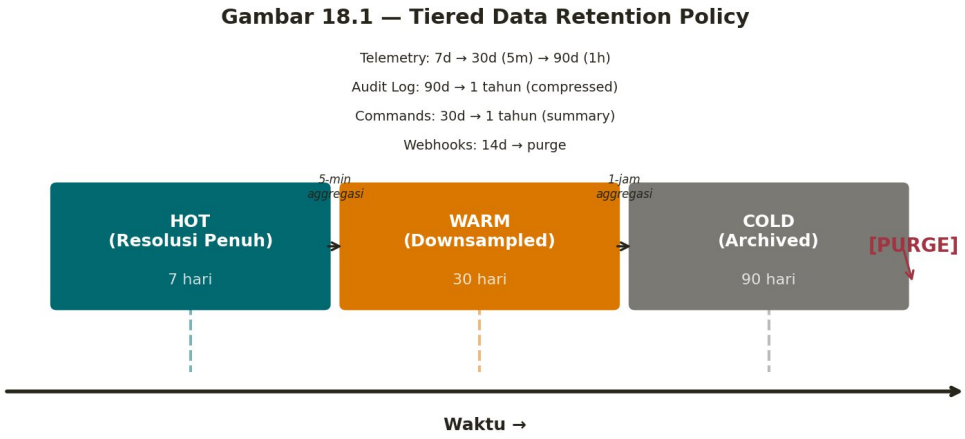
17.5 Agent Behavior Selama Extended Outage

Spesifikasi perilaku agent saat Hub tidak dapat dijangkau dalam waktu lama:

- Batas ukuran buffer (size limit) — misalnya maksimum 100 MB atau 24 jam data.
- Perilaku saat buffer overflow — drop oldest data atau stop collecting.

- Prioritisasi data — data mana yang dipertahankan jika buffer penuh.

18. Data Retention & Automated Cleanup Policy



Gambar 18.1 — Tiered Data Retention Policy

18.1 Latar Belakang & Kebutuhan

PRISM menghasilkan volume data yang terus bertambah dari berbagai sumber: telemetry metrics dari setiap server dan service, audit trail dari aktivitas pengguna, command execution history, dan webhook delivery logs. Tanpa kebijakan retensi yang jelas, pertumbuhan data ini akan berdampak pada performa query, kapasitas storage, dan biaya operasional.

Dalam deployment tipikal dengan 50 server dan 150 service, PRISM dapat menghasilkan ratusan ribu telemetry data points per hari. Setelah beberapa bulan operasi, tabel telemetry bisa mencapai puluhan juta baris, memperlambat query dashboard dan meningkatkan waktu backup. Tanpa downsampling dan cleanup otomatis, administrator harus melakukan purge manual yang berisiko menghapus data yang masih diperlukan.

Kebijakan retensi juga memiliki dimensi compliance. Audit log, misalnya, mungkin perlu disimpan lebih lama untuk memenuhi persyaratan audit internal atau regulasi. Sebaliknya, telemetry raw mungkin hanya perlu resolusi detik untuk 7 hari terakhir, sementara data yang lebih lama cukup dalam bentuk agregasi per-5-menit atau per-jam.

Strategi retensi yang baik mengimplementasikan tiered storage: hot tier untuk data terbaru dengan resolusi penuh, warm tier untuk data yang sudah didownsample, dan cold tier (archived) untuk data yang jarang diakses namun perlu disimpan untuk compliance. Pendekatan ini menyeimbangkan antara kebutuhan akses data real-time, efisiensi storage, dan kepatuhan terhadap kebijakan retensi.

Penting untuk membuat kebijakan retensi yang configurable — kebutuhan retensi dapat berbeda antar deployment PRISM tergantung pada skala, kebijakan perusahaan, dan persyaratan regulasi yang berlaku.

18.2 Desain yang Direkomendasikan

18.2.1 Retention Policies

Tipe Data	Hot (Resolusi Penuh)	Warm (Downsampled)	Cold (Archived)	Total Retensi
Telemetry raw	7 hari	30 hari (per-5-menit)	90 hari (per-jam)	~127 hari
Audit log	90 hari	—	1 tahun (compressed)	~1 tahun
Command history	30 hari (detail)	—	1 tahun (summary)	~1 tahun
Webhook deliveries	14 hari	—	—	14 hari
Config snapshots	30 hari	—	—	30 hari
Drift events	90 hari	—	1 tahun (summary)	~1 tahun
Health check logs	7 hari	—	—	7 hari

18.2.2 Downsampling Strategy

Downsampling mengubah data resolusi tinggi menjadi agregasi yang lebih rendah:

- Setelah 7 hari: telemetry raw diaggregasi per-5-menit. Untuk setiap metric, disimpan: min, max, avg, p95, count.
- Setelah 30 hari: data per-5-menit diaggregasi per-1-jam dengan statistik yang sama.
- Proses downsampling berjalan sebagai scheduled goroutine di Hub, default pukul 02:00 waktu lokal.
- Data raw dihapus setelah downsampled version dibuat dan diverifikasi.

18.2.3 Database Schema

Tabel	Kolom	Tipe	Keterangan
retention_policies	id	INTEGER PK	Auto-increment
retention_policies	data_type	TEXT	telemetry / audit / command / webhook / snapshot
retention_policies	hot_days	INTEGER	Hari penyimpanan resolusi penuh
retention_policies	warm_days	INTEGER	Hari penyimpanan downsampled (0 = skip)
retention_policies	cold_days	INTEGER	Hari penyimpanan archived (0 = skip)
retention_policies	is_global	BOOLEAN	Berlaku global atau per-server

retention_policies	updated_at	DATETIME	Terakhir diubah
retention_policies	updated_by	INTEGER FK	User yang mengubah

18.2.4 Archive Strategy

Data yang masuk ke cold tier diekspor ke format compressed (gzip) dan dapat disimpan di:

- Local filesystem: direktori archive di server Hub (default).
- S3-compatible storage: MinIO (self-hosted) atau AWS S3 untuk cold storage yang scalable.
- Archive file di-encrypt menggunakan kunci dari secrets manager (Rekomendasi 3).

18.2.5 Dashboard

Widget dashboard untuk retensi menampilkan: storage usage per data type (chart), timeline estimasi kapan data masing-masing tier akan dihapus, dan tombol trigger manual cleanup/archive.

18.3 Langkah Implementasi

73. Buat tabel retention_policies dengan default values sesuai tabel retention di atas. Sediakan API endpoint untuk CRUD policies.
74. Implementasikan cleanup scheduler: goroutine yang berjalan pada jadwal yang dikonfigurasi (default 02:00), iterasi setiap data type, dan menjalankan DELETE WHERE berdasarkan retention period.
75. Implementasikan downsampling job untuk telemetry: SELECT agregasi, INSERT ke tabel downsampled, DELETE raw data yang sudah diproses.
76. Implementasikan archive job: export data ke format compressed, upload ke storage target, verifikasi integritas, hapus data yang sudah diarchive dari database.
77. Bangun dashboard widget: storage usage per data type (chart), timeline estimasi, dan tombol trigger manual cleanup/archive.
78. Tambahkan monitoring untuk cleanup jobs: alert jika job gagal, laporan size reduction setelah setiap run.

18.4 Prioritas & Estimasi

Prioritas: Medium

Estimasi Effort: 1–2 sprint (scheduler + downsampling + archive + dashboard widget)

Justifikasi: Data retention penting untuk menjaga performa dan efisiensi storage jangka panjang. Prioritas medium karena dampak baru terasa setelah beberapa bulan operasi.

19. Concurrency dan Performance Tuning

Beberapa aspek performance tuning yang perlu didokumentasikan untuk memastikan performa optimal.

19.1 SQLite Write Serialization

Strategi single writer goroutine dengan channel-based queue untuk menghindari database locking. Semua operasi write dikirim melalui channel ke goroutine writer tunggal yang mengeksekusi secara berurutan.

19.2 Connection Pool Settings

Konfigurasi pool untuk Valkey client agar optimal di berbagai skala. Termasuk pengaturan MaxIdleConns, MaxActiveConns, dan IdleTimeout.

19.3 WebSocket Connection Limits

Batas koneksi dan OS-level tuning:

- File descriptor limits (ulimit -n) — minimal 65536 untuk production.
- TCP keepalive settings untuk deteksi koneksi mati.
- Maximum concurrent WebSocket connections per Hub instance.

19.4 Memory Budget

Estimasi konsumsi resource per goroutine dan proyeksi kebutuhan pada berbagai skala:

Skala Agent	RAM Hub (est.)	CPU Hub (est.)	Goroutines (est.)
100 agents	512 MB - 1 GB	2 cores	~500
500 agents	2 - 4 GB	4 cores	~2.500
1.000 agents	4 - 8 GB	8 cores	~5.000

19.5 Agent Graceful Degradation untuk Plugin Errors

Perilaku agent ketika sebuah plugin spesifik mengalami kegagalan:

- Continuity: Agent harus tetap melaporkan service lain meskipun satu plugin gagal.
- Error isolation: Satu plugin yang crash tidak boleh menjatuhkan keseluruhan agent. Implementasikan recovery per plugin.
- Plugin health reporting: Agent melaporkan status kesehatan setiap plugin ke Hub.

Pola yang direkomendasikan adalah menjalankan setiap plugin dalam goroutine terpisah dengan panic recovery, dan menggunakan channel untuk komunikasi antar plugin dan core agent.

20. Strategi Update dan Rollback Agent

Mekanisme update agent binary di seluruh managed servers perlu didefinisikan secara jelas untuk memastikan keandalan dan keamanan proses update.

20.1 Push Update via Hub

Pertimbangkan mekanisme dimana Hub mengirimkan binary baru ke agent melalui channel yang sudah ada. Proses update meliputi: download binary baru, verifikasi checksum/signature, graceful restart agent.

20.2 Compatibility Matrix

Definisikan matriks kompatibilitas versi Agent dengan Hub untuk memastikan interoperabilitas. Hub harus mendukung minimal N-2 versi agent untuk memberikan waktu upgrade.

20.3 Rollback Procedure

Prosedur jika versi agent baru bermasalah — agent menyimpan binary sebelumnya dan bisa revert. Rollback harus otomatis jika agent baru gagal startup atau gagal heartbeat dalam waktu tertentu.

20.4 Canary Deployment

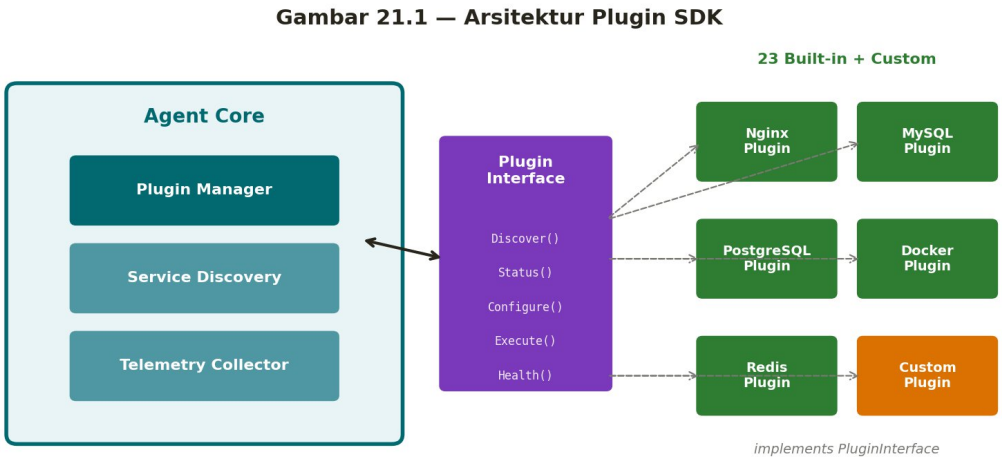
Strategi roll out ke subset agent terlebih dahulu sebelum deployment menyeluruh:

- Mulai dengan 5-10% agent (canary group).
- Monitor health metrics selama 1-2 jam.
- Jika stabil, lanjutkan ke batch berikutnya (25%, 50%, 100%).
- Jika ada masalah, otomatis rollback canary group.

20.5 Self-Update vs. External Orchestration

Tentukan apakah agent melakukan self-update atau menggunakan tools eksternal seperti Ansible. Rekomendasi: self-update melalui Hub untuk kemudahan operasional, dengan fallback ke Ansible/SSH untuk skenario darurat.

21. Plugin SDK & Panduan Pengembangan Plugin



Gambar 21.1 — Arsitektur Plugin SDK

21.1 Latar Belakang & Kebutuhan

PRISM saat ini mendukung 23 service plugins yang dikembangkan secara internal untuk berbagai layanan infrastruktur seperti Nginx, Apache, PostgreSQL, MySQL, Redis, dan lainnya. Namun, pengembangan plugin dilakukan tanpa kontrak formal — setiap plugin diimplementasikan dengan pendekatan yang sedikit berbeda, membuat maintenance dan onboarding developer baru menjadi lebih sulit.

Tanpa SDK formal, beberapa masalah muncul. Pertama, inkonsistensi interface: beberapa plugin mengimplementasikan health check, sementara lainnya tidak. Kedua, error handling yang tidak seragam: crash di satu plugin dapat mempengaruhi stabilitas agent secara keseluruhan. Ketiga, tidak ada panduan testing: setiap developer plugin membuat test dengan pendekatan berbeda, atau tidak membuat test sama sekali.

Plugin SDK yang formal mendefinisikan kontrak yang jelas antara Agent core dan plugin, menyediakan utilitas umum (logging, configuration parsing, error handling), dan menyertakan template serta testing harness yang mempercepat pengembangan plugin baru. SDK juga memungkinkan kontribusi dari pihak luar — misalnya vendor yang ingin membuat plugin untuk layanan proprietary mereka.

Dari perspektif arsitektur, Go menyediakan beberapa mekanisme untuk plugin system. Pendekatan paling sederhana adalah compile-time plugins menggunakan Go interfaces — plugin di-compile bersama agent. Untuk extensibility tanpa recompile, Go plugin package atau gRPC-based plugins dapat digunakan, masing-masing dengan trade-off yang berbeda dalam hal performa dan kompleksitas.

Untuk fase awal, direkomendasikan pendekatan compile-time interface-based plugins yang sudah digunakan saat ini, namun dengan formalisasi interface dan

tambahan SDK tooling. Migrasi ke plugin system yang mendukung dynamic loading dapat dilakukan di fase berikutnya jika ada kebutuhan.

21.2 Desain yang Direkomendasikan

Plugin Interface

Setiap plugin harus mengimplementasikan interface berikut:

Method	Signature	Keterangan
PluginInfo()	PluginMetadata	Mengembalikan metadata plugin (nama, versi, author, OS support)
Discover()	(bool, error)	Mendeteksi apakah service terinstal di server
Status()	(ServiceStatus, error)	Mengambil status service saat ini (running, stopped, error)
Configure(config)	error	Menerapkan konfigurasi ke service
Execute(cmd)	(Result, error)	Menjalankan perintah service-specific
HealthCheck()	(HealthResult, error)	Menjalankan health check plugin-level
GetConfigSnapshot()	(ConfigData, error)	Mengambil snapshot konfigurasi aktual (untuk drift detection)
Shutdown()	error	Cleanup saat agent shutdown

Plugin Metadata

Field	Tipe	Keterangan
Name	string	Nama unik plugin (misal: nginx, postgresql)
DisplayName	string	Nama tampilan (misal: Nginx Web Server)
Version	string	Semver (misal: 1.2.0)
Description	string	Deskripsi singkat fungsi plugin
Author	string	Pembuat plugin
SupportedOS	[]string	Daftar OS yang didukung (linux, windows)
MinAgentVersion	string	Versi minimum agent yang diperlukan
Dependencies	[]string	Plugin lain yang diperlukan (opsional)

Plugin Lifecycle

Setiap plugin mengikuti lifecycle berikut:

- Load: Agent memuat plugin dan memanggil PluginInfo() untuk registrasi.
- Initialize: Agent memanggil Discover() untuk mengecek ketersediaan service. Plugin yang gagal discover dilewati.
- Ready: Plugin yang berhasil discover masuk ke state ready dan mulai menerima perintah.

- Execute: Plugin menjalankan command dari Hub (status check, configuration, execution).
- Shutdown: Saat agent shutdown, setiap plugin dipanggil Shutdown() untuk cleanup.

Setiap plugin berjalan di goroutine terpisah dengan panic recovery — crash di satu plugin tidak menyebabkan seluruh agent crash. Plugin yang panic akan di-mark sebagai errored dan dapat di-restart secara independen.

Plugin Registry di Hub

Field	Keterangan
Plugin name + version	Identifikasi unik plugin
Compatibility matrix	Versi agent minimum dan OS yang didukung
Deployment count	Jumlah agent yang menjalankan plugin ini
Health status aggregate	Persentase agent dengan plugin healthy
Latest version available	Untuk mendeteksi plugin yang perlu update

21.3 Langkah Implementasi

79. Formalisasi Go interface: definisikan ServicePlugin interface dengan semua method yang diperlukan, termasuk type definitions untuk parameter dan return values.
80. Refactor existing 23 plugins untuk mengimplementasikan interface baru. Untuk setiap plugin, tambahkan method yang belum ada (GetConfigSnapshot, HealthCheck) secara bertahap.
81. Implementasikan plugin runner di Agent: goroutine manager dengan panic recovery, plugin state tracking, dan restart capability.
82. Buat plugin template project: scaffold dengan Makefile, directory structure, contoh implementasi, dan pre-configured test harness.
83. Bangun testing harness: mock Agent environment yang memungkinkan plugin developer menjalankan dan menguji plugin secara isolated tanpa full agent deployment.
84. Implementasikan plugin registry di Hub: endpoint untuk melaporkan plugin inventory per agent, compatibility check, dan aggregate health status.
85. Tulis Plugin Development Guide: panduan langkah-demi-langkah untuk membuat plugin baru, mulai dari template hingga deployment.
86. (Opsional, fase lanjut) Evaluasi gRPC-based plugin system untuk mendukung third-party plugins tanpa recompile agent.

21.4 Prioritas & Estimasi

Prioritas: Low

Estimasi Effort: 3–4 sprint (interface formalization + refactor existing plugins + template + testing harness + documentation)

Justifikasi: Plugin SDK meningkatkan maintainability dan extensibility jangka panjang. Prioritas low karena sistem saat ini berfungsi meskipun tanpa SDK formal, dan benefit utama baru terasa ketika ada kebutuhan plugin baru dari developer eksternal.

22. Data Export/Import & Portability

22.1 Latar Belakang & Kebutuhan

PRISM saat ini tidak memiliki mekanisme formal untuk mengekspor dan mengimpor data. Hal ini menimbulkan beberapa keterbatasan: administrator tidak dapat mengekspor data untuk analisis di tool eksternal, migrasi antar instance Hub memerlukan proses manual, dan compliance reporting yang membutuhkan data audit harus dilakukan melalui query database langsung.

Kemampuan export/import data menjadi penting dalam beberapa skenario operasional. Pertama, compliance reporting: auditor mungkin membutuhkan data audit trail dalam format yang dapat diproses (CSV, JSON). Kedua, migration: ketika melakukan upgrade major atau memindahkan Hub ke server baru, kemampuan export/import memastikan data tidak hilang. Ketiga, disaster recovery: export periodik ke external storage memberikan layer tambahan di atas backup database.

Data portability juga mendukung integrasi dengan ekosistem yang lebih luas. Tim data analytics dapat mengimpor telemetry data ke sistem BI (Business Intelligence). Tim security dapat mengekspor audit log ke SIEM (Security Information and Event Management). Tim DevOps dapat mengekspor konfigurasi server dalam format YAML untuk dikelola sebagai Infrastructure as Code.

Fitur export/import perlu didesain dengan mempertimbangkan volume data yang besar — export telemetry untuk 50 server selama 30 hari bisa menghasilkan file berukuran ratusan megabyte. Oleh karena itu, export harus mendukung streaming, compression, dan scope filtering.

Keamanan juga menjadi pertimbangan penting. Data yang diekspor mungkin mengandung informasi sensitif (credential, konfigurasi). Export file harus mendukung enkripsi opsional, dan import harus melalui proses validasi yang ketat untuk mencegah injeksi data yang tidak valid atau berbahaya.

22.2 Desain yang Direkomendasikan

22.2.1 Export Scopes & Formats

Scope	Konten	Format	Estimasi Size
Full system	Semua konfigurasi + server data + metadata	JSON (compressed)	50–500 MB
Per-server	Server config + services + telemetry summary	JSON	1–10 MB per server
Per-service	Service config + credentials (encrypted) + history	JSON / YAML	< 1 MB
Audit log	Audit entries dengan filter waktu	CSV / JSON	10–100 MB per tahun
Telemetry	Metric data dengan filter waktu dan resolusi	CSV	10–500 MB tergantung range
Configuration	Desired state semua server dan service	YAML	< 5 MB

22.2.2 API Endpoints

Endpoint	Method	Keterangan
/api/export	POST	Memulai export job (async). Body: scope, format, filters, encryption options.
/api/export/{job_id}	GET	Mengecek status export job (progress percentage).
/api/export/{job_id}/download	GET	Mendownload file hasil export.
/api/import	POST	Memulai import job (async). Body: multipart file upload.
/api/import/{job_id}	GET	Mengecek status import job (progress, validation results).
/api/import/{job_id}/confirm	POST	Konfirmasi import setelah preview (conflict resolution).

22.2.3 Import Validation & Conflict Resolution

Proses import dilakukan dalam tiga tahap:

- Validation: File import divalidasi terhadap schema version yang terdapat dalam file. Setiap field diperiksa tipe dan constraint-nya.
- Preview: Sistem menampilkan ringkasan apa yang akan di-import, termasuk konflik yang ditemukan (data yang sudah ada di database).
- Execution: Setelah konfirmasi, import dijalankan. Untuk setiap konflik, user memilih strategi: skip (lewati), overwrite (timpa data existing), atau merge (gabungkan data baru dan existing).

22.2.4 Scheduled Exports

Fitur scheduled export memungkinkan administrator mengonfigurasi export periodik otomatis:

- Schedule: harian, mingguan, atau bulanan.
- Scope dan format yang dikonfigurasi.
- Target storage: local filesystem, S3-compatible, atau SFTP.
- Retention: jumlah export file yang disimpan (FIFO). Misalnya, simpan 7 file harian terakhir.
- Notifikasi: webhook notification jika scheduled export gagal.

22.3 Langkah Implementasi

87. Desain export file format dengan schema versioning: setiap file export menyertakan metadata (versi PRISM, schema version, timestamp, scope).
88. Implementasikan export engine: streaming JSON/CSV writer yang memproses data per-batch untuk menghindari memory spike pada dataset besar.

89. Implementasikan import engine: file parser, schema validator, conflict detector, dan executor dengan support untuk tiga conflict resolution strategy.
90. Buat API endpoints untuk export/import dengan async job processing (goroutine) dan progress reporting melalui polling atau WebSocket.
91. Bangun Export/Import wizard di dashboard: scope selector, format picker, progress bar, validation result display, conflict resolution UI.
92. Implementasikan scheduled export: cron-like scheduler, storage adapter (local/S3/SFTP), retention management.
93. Tambahkan enkripsi opsional untuk export files menggunakan AES-256-GCM dengan kunci yang disediakan user atau dari secrets manager.

22.4 Prioritas & Estimasi

Prioritas: Low

Estimasi Effort: 2–3 sprint (export engine + import engine + API + dashboard wizard + scheduled exports)

Justifikasi: Data export/import meningkatkan portability dan mendukung compliance reporting. Prioritas low karena kebutuhan utama (backup) sudah tercakup oleh fitur backup existing, dan kebutuhan export untuk analytics/compliance baru muncul pada deployment yang lebih mature.

23. Perencanaan Pembangunan

Bagian ini memberikan rekomendasi untuk aspek-aspek perencanaan pembangunan yang akan mendukung keberhasilan implementasi sistem PRISM v4.1, termasuk update roadmap untuk mencakup rekomendasi-rekomendasi baru.

23.1 Roadmap Pengembangan Bertahap

Direkomendasikan pendekatan pengembangan 4 fase selama 12 bulan, dengan update pada Fase 3 dan 4 untuk menyertakan rekomendasi-rekomendasi baru dari v4.1:

Fase 1 — Foundation (Bulan 1–3)

Membangun fondasi inti sistem yang stabil:

- Hub core: REST API, WebSocket gateway, SQLite database layer, autentikasi dasar
- Agent core: enrollment, heartbeat, telemetri dasar
- Web App: login, daftar server, dashboard dasar
- Health check endpoints dasar (Rekomendasi #7 — prioritas High)

Target: 1 Hub + 5 agents berjalan stabil

Fase 2 — Service Management (Bulan 4–6)

Mengembangkan kemampuan manajemen layanan:

- Agent plugins: MySQL, PostgreSQL, Nginx (mulai dari 3 yang paling umum)
- Web App: server card, service modal (tab Overview + Accounts)
- Operasi CRUD akun melalui agent
- Implementasi penuh mTLS + JWT
- Enkripsi database SQLite at Rest dengan SQLCipher (Rekomendasi #2 — prioritas High)
- Secrets management terpusat dengan Infisical (Rekomendasi #3 — prioritas High)

Target: Full lifecycle untuk 3 tipe layanan + keamanan data terjamin

Fase 3 — Advanced Features (Bulan 7–9)

Melengkapi fitur-fitur lanjutan:

- Plugin layanan tersisa (MongoDB, Valkey, RabbitMQ, MinIO, dll.)
- Sistem manajemen proyek
- Process supervisors (PM2, Supervisor, systemd user)
- Dukungan container runtime (Docker, Podman)
- Audit logging dan RBAC granular (Rekomendasi #1 — prioritas High)
- Sistem webhook & notifikasi eksternal (Rekomendasi #4 — prioritas Medium)
- Configuration drift detection (Rekomendasi #5 — prioritas Medium)
- Data retention & automated cleanup (Rekomendasi #6 — prioritas Medium)

Target: Feature complete untuk seluruh 23 plugin layanan + fitur operasional

Fase 4 — Hardening & Scale (Bulan 10–12)

Penguatan dan persiapan produksi:

- Performance testing dan optimasi
- Implementasi backup/DR lengkap
- Integrasi secure tunnel (Cloudflare/WireGuard)
- Modul sinkronisasi PATRA
- Plugin SDK & panduan pengembangan (Rekomendasi #8 — prioritas Low)
- Data export/import & portability (Rekomendasi #9 — prioritas Low)
- Dokumentasi dan panduan deployment

Target: Sistem siap produksi (production-ready) dengan full extensibility

Ringkasan Roadmap

Fase	Periode	Fokus Utama	Target
1 — Foundation	Bulan 1–3	Hub core, Agent core, Web App dasar, Health checks	1 Hub + 5 agents
2 — Service Mgmt	Bulan 4–6	3 plugin utama, mTLS+JWT, CRUD akun, SQLCipher, Secrets	3 tipe layanan + data aman
3 — Advanced	Bulan 7–9	23 plugin, RBAC granular, Webhook, Drift Detection, Retention	Feature complete
4 — Hardening	Bulan 10–12	Performance, DR, tunnel, Plugin SDK, Export/Import, dokumentasi	Production-ready

23.2 Strategi Testing

Strategi testing yang terstruktur sangat penting untuk memastikan kualitas dan keandalan sistem:

Unit Tests

Untuk setiap modul Go (api, ws, auth, enrollment, discovery, commander, telemetry, cache, db). Target coverage: 80%+.

Integration Tests

- Gunakan testcontainers-go untuk testing komunikasi agent-hub, operasi database, dan alur WebSocket.
- Test setiap plugin terhadap instance service yang sebenarnya (Docker containers).

End-to-End Tests

Full workflow testing mencakup alur lengkap:

- Enrollment → Approval → Discovery → Command Execution → Telemetry
- Skenario error: koneksi terputus, timeout, invalid credentials

Performance Tests

Load testing dengan simulated agents:

- Gunakan k6 atau custom Go benchmarks
- Test dengan 100, 500, dan 1.000 koneksi agent secara bersamaan
- Ukur throughput, latency, dan resource consumption

Security Tests

Pengujian keamanan mencakup:

- Validasi mTLS — penolakan koneksi tanpa sertifikat valid
- JWT expiry handling — perilaku saat token kadaluarsa
- PSK brute-force protection — rate limiting pada percobaan enrollment
- SQL injection testing pada API endpoints

23.3 CI/CD Pipeline

Pipeline CI/CD yang terotomasi diperlukan untuk mendukung proses development yang efisien:

- Git branching strategy: Trunk-based development atau Gitflow, disesuaikan dengan ukuran tim.
- Automated build: Pipeline build otomatis untuk Hub binary, Agent binary (multi-arch: amd64, arm64), dan Web App (static build).
- Container image builds: Untuk deployment Hub dalam container.
- Automated test stages: Alur unit → integration → e2e secara berurutan.
- Release artifact management: Menggunakan GitHub Releases atau internal registry.
- Agent distribution: Mekanisme distribusi binary agent — download server, package repository, atau embedded dalam PRISM.

23.4 Dokumentasi Teknis Tambahan

Dokumentasi berikut perlu disiapkan untuk mendukung pengembangan dan operasional:

- API reference: Dokumentasi referensi API yang di-generate otomatis dari kode sumber.
- Plugin development guide: Panduan pengembangan plugin agent untuk menambah dukungan tipe layanan baru.
- Deployment guide: Panduan instalasi, konfigurasi, dan upgrade sistem.
- Troubleshooting guide: Kumpulan masalah umum beserta solusinya.
- Architecture Decision Records (ADRs): Dokumentasi keputusan arsitektur penting beserta alasan dan konsekuensinya.

23.5 Estimasi Effort dan Tim

Rekomendasi komposisi tim dan estimasi kasar untuk pembangunan penuh:

Jumlah	Peran	Tanggung Jawab
2	Backend Engineer (Go)	Pengembangan Hub dan Agent, termasuk plugin system
1	Frontend Engineer (React)	Pengembangan Web App, dashboard, dan UI komponen
1	DevOps/QA Engineer	CI/CD pipeline, testing infrastructure, deployment automation
1*	Security Reviewer	Review keamanan berkala (*part-time/opsional)

Total estimasi effort: ~12–15 person-months untuk full feature set PRISM v4.0. Dengan tambahan 9 rekomendasi dari v4.1, estimasi total menjadi ~29–40 person-months (17–25 sprint tambahan untuk rekomendasi). Estimasi ini mencakup pengembangan, testing, dan dokumentasi, namun belum termasuk waktu untuk infrastructure setup dan overhead manajemen proyek.

23.6 Prioritas Implementasi Rekomendasi

Tabel berikut mencakup seluruh 18 rekomendasi — 9 rekomendasi lama (dari v4.0) dan 9 rekomendasi baru (v4.1):

No.	Area Rekomendasi	Prioritas	Fase Relevan
1	Strategi Backup & Disaster Recovery	Tinggi (Kritikal)	Fase 1–2
2	Observability & Monitoring Internal	Tinggi (Kritikal)	Fase 1–2
3	Roadmap Pengembangan Bertahap	Tinggi (Kritikal)	Semua Fase
4	Strategi Testing & CI/CD	Tinggi (Kritikal)	Fase 1
5	Manajemen Sertifikat Lengkap	Sedang	Fase 2
6	Performance Tuning & Concurrency	Sedang	Fase 4
7	API Versioning & Dokumentasi	Sedang	Fase 2–3
8	Strategi Update & Rollback Agent	Sedang	Fase 3–4
9	Strategi Migrasi Database	Rendah (Planning)	Fase 4+
10 (Baru)	Model RBAC Granular — Resource-Level Access (Bab 7)	High	Fase 3
11 (Baru)	Enkripsi Database SQLite at Rest dengan SQLCipher (Bab 8)	High	Fase 2
12 (Baru)	Secrets & Credential Management Terpusat (Bab 9)	High	Fase 2
13 (Baru)	Sistem Webhook & Notifikasi Eksternal (Bab 15)	Medium	Fase 3
14 (Baru)	Configuration Drift Detection (Bab 16)	Medium	Fase 3
15 (Baru)	Data Retention & Automated Cleanup Policy (Bab 18)	Medium	Fase 3

16 (Baru)	Health Check Endpoints (Bab 14)	High	Fase 1
17 (Baru)	Plugin SDK & Panduan Pengembangan Plugin (Bab 21)	Low	Fase 4
18 (Baru)	Data Export/Import & Portability (Bab 22)	Low	Fase 4

Rekomendasi 10: Optimalisasi Skalabilitas Telemetri dengan Time-Series Partitioning

10.1 Latar Belakang

SQLite dengan mode WAL (Write-Ahead Logging) sangat efisien untuk operasi konkuren. Namun, tabel telemetry akan menjadi bottleneck pertama saat jumlah server tumbuh melampaui 1.000 unit. Satu tabel telemetry raksasa menyebabkan:

- Index B-tree yang semakin dalam dan lambat untuk INSERT
- DELETE baris per baris saat cleanup yang membebani I/O dan fragmentasi
- VACUUM yang semakin lama seiring pertumbuhan tabel
- Query range yang harus scan seluruh index

10.2 Arsitektur Time-Series Partitioning

Implementasikan partitioning di level aplikasi menggunakan tabel per periode waktu. SQLite tidak mendukung native partitioning, sehingga pendekatan ini dilakukan di level Go application code.

Strategi Partisi: Tabel mingguan dengan format `telemetry_YYYY_wWW` (contoh: `telemetry_2026_w12`).

Komponen Utama:

Komponen	Fungsi
Partition Router	Menentukan tabel target berdasarkan timestamp data
Partition Manager	Membuat tabel baru secara otomatis saat minggu berganti
Query Fanout	Menggabungkan query dari beberapa partisi untuk range query
Cleanup Executor	DROP TABLE untuk partisi yang sudah melewati retention period

10.3 Skema Database

Tabel Registry Partisi:

```
CREATE TABLE partition_registry (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    table_name TEXT UNIQUE NOT NULL,
    partition_key TEXT NOT NULL,        -- 'YYYY_wWW'
    start_timestamp DATETIME NOT NULL,
    end_timestamp DATETIME NOT NULL,
    row_count INTEGER DEFAULT 0,
    size_bytes INTEGER DEFAULT 0,
    status TEXT DEFAULT 'active',       -- active | readonly | archived | dropped
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Template Tabel Partisi (dibuat otomatis per minggu):

```
CREATE TABLE telemetry_2026_w12 (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    server_id INTEGER NOT NULL,
    metric_name TEXT NOT NULL,
    metric_value REAL NOT NULL,
    labels TEXT,                        -- JSON
    timestamp DATETIME NOT NULL,
    FOREIGN KEY (server_id) REFERENCES servers(id)
);
CREATE INDEX idx_tel_2026_w12_server_ts
    ON telemetry_2026_w12(server_id, timestamp);
CREATE INDEX idx_tel_2026_w12_metric
    ON telemetry_2026_w12(metric_name, timestamp);
```

10.4 Alur Kerja

Insert Path:

1. Data telemetry masuk dari agent via WebSocket
2. Partition Router menghitung partition key dari timestamp
3. Jika tabel partisi belum ada, Partition Manager membuatnya dari template
4. Data ditulis ke tabel partisi yang tepat

Query Path:

1. Query dengan range waktu diterima dari API
2. Query Fanout mengidentifikasi partisi yang perlu di-query dari registry
3. Query dijalankan paralel pada setiap partisi yang relevan
4. Hasil digabungkan dan dikembalikan ke client

Cleanup Path:

1. Scheduler berjalan setiap 24 jam (02:00 WIB)
2. Cek partition_registry untuk partisi yang sudah melewati retention period
3. Data yang masuk tier WARM: buat tabel downsampled, lalu DROP tabel asli
4. Data yang masuk tier COLD: archive ke file, lalu DROP tabel downsampled
5. Data yang melewati cold retention: DROP TABLE langsung (jauh lebih cepat dari DELETE row per row)

10.5 Konfigurasi

```
telemetry:
  partitioning:
    enabled: true
    interval: "weekly"            # weekly | daily | monthly
    naming_pattern: "telemetry_{year}_w{week}"
    auto_create: true
    retention:
      hot: "7d"                   # Full resolution
      warm: "30d"                 # 5-minute aggregation
      cold: "90d"                 # 1-hour aggregation
    cleanup_schedule: "0 2 * * *" # Setiap hari jam 02:00
    max_partitions: 52            # Safety limit
```

10.6 Estimasi Performa

Operasi	Tanpa Partitioning (1M rows)	Dengan Partitioning (1M rows / 52 partisi)

INSERT latency	~5ms (index rebuild)	~1ms (index kecil)
DELETE cleanup	~30 detik (row-by-row)	~10ms (DROP TABLE)
Range query (1 minggu)	Scan seluruh index	Scan 1 partisi saja
VACUUM	~60 detik	Tidak diperlukan (tabel di-drop)

10.7 Langkah Implementasi

1. Buat migration untuk tabel partition_registry
2. Implementasikan PartitionRouter dan PartitionManager di package telemetry
3. Refactor INSERT path telemetry untuk menggunakan PartitionRouter
4. Implementasikan QueryFanout untuk read path
5. Refactor cleanup scheduler untuk menggunakan DROP TABLE
6. Buat migration script untuk mempartisi tabel telemetry yang sudah ada
7. Tambahkan monitoring: jumlah partisi aktif, ukuran per partisi, durasi cleanup

10.8 Prioritas & Estimasi

Prioritas: High — menjadi bottleneck pertama saat scaling ke 1.000+ server.

Estimasi Effort: 2–3 sprint (partition manager + query refactor + migration script + testing).

Rekomendasi 11: Penguatan Keamanan Update Agent dengan Code Signing

11.1 Latar Belakang

Mekanisme self-update agent (Bab 20) memungkinkan Hub mengirimkan binary baru ke Agent. Tanpa verifikasi kriptografis, skenario Hub yang terkompromi dapat menyebarkan malware ke seluruh infrastruktur yang dikelola. Ini adalah vektor serangan supply-chain yang serius.

11.2 Arsitektur Code Signing

Algoritma: Ed25519 — dipilih karena:

- Native support di Go standard library (crypto/ed25519)
- Signature hanya 64 bytes (efisien untuk transfer)
- Kecepatan verifikasi sangat tinggi (~70.000 verifikasi/detik)
- Tidak memerlukan certificate chain (lebih sederhana dari RSA/ECDSA PKI)

Komponen:

Komponen	Lokasi	Fungsi
Signing Key (Private)	Build Server (offline)	Menandatangani binary agent saat build
Verification Key (Public)	Embedded di binary Agent	Memverifikasi binary baru sebelum eksekusi
Signature File	Dikirim bersama binary	Bukti keaslian binary
Manifest File	Dikirim bersama binary	Metadata versi, checksum SHA-256, platform

11.3 Alur Update yang Aman

Build-Time Signing:

1. CI/CD build menghasilkan binary agent baru untuk setiap platform (linux/amd64, linux/arm64)
2. SHA-256 checksum dihitung untuk setiap binary
3. Manifest JSON dibuat berisi: versi, checksum, timestamp, min_version_compatible
4. Manifest ditandatangani dengan Ed25519 private key
5. Binary + manifest + signature di-upload ke Hub

Agent-Side Verification:

1. Agent menerima notifikasi update tersedia dari Hub via WebSocket
2. Agent download binary baru, manifest, dan signature
3. Agent memverifikasi signature manifest dengan embedded public key
4. Agent menghitung SHA-256 binary yang didownload
5. Agent membandingkan checksum dengan yang tercantum di manifest

6. Jika verifikasi sukses: binary baru disimpan ke temporary path
7. Agent melakukan self-restart dengan binary baru
8. Jika startup binary baru gagal: rollback otomatis ke binary sebelumnya

11.4 Struktur Manifest

```
{
    "version": "1.5.2",
    "channel": "stable",
    "timestamp": "2026-03-19T10:00:00Z",
    "min_hub_version": "4.1.0",
    "binaries": {
        "linux/amd64": {
            "filename": "prism-agent-1.5.2-linux-amd64",
            "sha256": "a1b2c3d4e5f6...",
            "size_bytes": 12582912
        },
        "linux/arm64": {
            "filename": "prism-agent-1.5.2-linux-arm64",
            "sha256": "f6e5d4c3b2a1...",
            "size_bytes": 11534336
        }
    },
    "release_notes": "Bug fix: memory leak pada plugin Docker",
    "rollback_version": "1.5.1"
}
```

11.5 Key Management

Aspek	Detail
Key Generation	Ed25519 keypair dibuat offline di mesin build yang aman
Private Key Storage	Encrypted dengan passphrase, disimpan di HSM atau secrets manager build server
Public Key Distribution	Embedded di binary agent saat compile-time (hardcoded constant)
Key Rotation	Public key baru di-embed di versi agent berikutnya. Selama transisi, agent menerima signature dari key lama DAN baru (dual-key verification window)
Revocation	Jika private key bocor: terbitkan update darurat yang ditandatangani dengan emergency backup key

11.6 Proteksi Tambahan

- **Version Downgrade Protection:** Agent menolak binary dengan versi lebih rendah dari yang sedang berjalan
- **Timestamp Validation:** Manifest dengan timestamp lebih dari 72 jam dianggap expired
- **Staged Rollout:** Hub dapat mengirim update ke subset agent (canary deployment) sebelum rollout penuh

- **Audit Trail:** Setiap proses update dicatat di audit log (versi lama, versi baru, checksum, hasil verifikasi)

11.7 Langkah Implementasi

1. Generate Ed25519 keypair dan simpan di build infrastructure
2. Buat signing tool CLI: `prism-sign --key private.key --manifest manifest.json`
3. Integrasikan signing ke CI/CD pipeline
4. Embed public key di binary agent
5. Implementasikan verification flow di agent update module
6. Implementasikan rollback mechanism jika binary baru gagal startup
7. Tambahkan staged rollout support di Hub
8. Buat test suite: tampered binary detection, expired manifest, version downgrade rejection

11.8 Prioritas & Estimasi

Prioritas: Critical — supply-chain attack prevention.

Estimasi Effort: 1–2 sprint (signing tool + agent verification + CI/CD integration).

Rekomendasi 12: Integrasi Identity Provider (IdP) Eksternal dengan Group-to-Role Mapping

12.1 Latar Belakang

Dokumen v4.1 (Bab 7) sudah menyebutkan rencana integrasi SSO di masa depan. Rekomendasi ini memperdalam desain integrasi agar tidak hanya memetakan user, tetapi juga group/role dari IdP langsung ke Server Groups dan permission di PRISM.

12.2 Arsitektur Integrasi

Protokol yang Didukung:

- OIDC (OpenID Connect) — untuk IdP modern (Keycloak, Okta, Azure AD, Google Workspace)
- LDAP/LDAPS — untuk Active Directory dan OpenLDAP existing

Alur Autentikasi OIDC:

1. User mengakses PRISM Web App
2. Web App redirect ke IdP login page
3. User login di IdP
4. IdP mengembalikan ID Token + Access Token ke callback URL PRISM
5. PRISM Hub memvalidasi token (signature, issuer, audience, expiry)
6. PRISM Hub extract claim `groups` atau `roles` dari token
7. PRISM Hub memetakan groups ke roles dan server_groups di PRISM
8. Session dibuat dengan permission yang sesuai

12.3 Group-to-Role Mapping

Konsep: Setiap group di IdP dipetakan ke satu atau lebih role + server group combination di PRISM. Ini menghilangkan kebutuhan untuk mengelola permission individual setiap karyawan baru.

Tabel Mapping:

```
CREATE TABLE idp_group_mappings (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    idp_provider TEXT NOT NULL,           -- 'oidc' | 'ldap'
    idp_group_name TEXT NOT NULL,         -- 'DBA-Team' (dari IdP)
    prism_role_id INTEGER NOT NULL,       -- Referensi ke roles.id
    prism_server_group_id INTEGER,        -- NULL = semua server
    auto_provision BOOLEAN DEFAULT true,  -- Auto-create user saat login pertama
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (prism_role_id) REFERENCES roles(id),
    FOREIGN KEY (prism_server_group_id) REFERENCES server_groups(id),
    UNIQUE(idp_provider, idp_group_name, prism_role_id, prism_server_group_id)
);

CREATE TABLE idp_sync_log (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    idp_provider TEXT NOT NULL,
    idp_groups TEXT NOT NULL,              -- JSON array dari groups
```

```
    mapped_roles TEXT NOT NULL,            -- JSON array dari mapped roles
    action TEXT NOT NULL,                  -- 'provisioned' | 'updated'
| 'deprovisioned'
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id)
);
```

Contoh Mapping:

IdP Group	PRISM Role	Server Group	Efek
DBA-Team	db-admin	Database-Cluster	Akses penuh ke server DB
Web-Ops	web-ops	Production-Web	Kontrol service di web servers
Security-Audit	viewer	* (semua)	Read-only di semua server
DevOps-Lead	admin	* (semua)	Akses penuh semua
Junior-Dev	viewer	Staging-Web	Read-only di staging saja

12.4 Konfigurasi

```yaml
auth:
  idp:
    oidc:
      enabled: true
      issuer_url: "https://keycloak.company.com/realms/infra"
      client_id: "prism-hub"
      client_secret: "${OIDC_CLIENT_SECRET}"
      scopes: ["openid", "profile", "groups"]
      claims:
        username: "preferred_username"
        email: "email"
        groups: "groups"              # Claim yang berisi group membership
      callback_url: "https://prism.smartsystem.id/auth/callback"
    ldap:
      enabled: false
      url: "ldaps://ad.company.com:636"
      bind_dn: "cn=prism-svc,ou=services,dc=company,dc=com"
      bind_password: "${LDAP_BIND_PASSWORD}"
      user_search_base: "ou=users,dc=company,dc=com"
      user_search_filter: "(sAMAccountName={username})"
      group_search_base: "ou=groups,dc=company,dc=com"
      group_search_filter: "(member={dn})"
      group_name_attr: "cn"
  group_mapping:
    sync_interval: "5m"            # Sinkronisasi group setiap 5 menit
    auto_provision: true           # Auto-create user saat pertama login
    auto_deprovision: false        # Jangan hapus user otomatis saat dihapus dari IdP
    fallback_role: "viewer"        # Role default jika tidak ada group mapping
```

12.5 Fitur Utama

1. **Auto-Provisioning:** User baru yang login via IdP otomatis dibuat di PRISM dengan role sesuai group mapping
2. **Dynamic Role Update:** Setiap login, group membership di-sync — jika user pindah tim di IdP, permission di PRISM ikut berubah

3. **Hybrid Mode:** User lokal (non-IdP) tetap bisa dikelola manual. Berguna untuk emergency access
4. **Group Sync Log:** Setiap perubahan group/role dicatat di audit trail untuk compliance
5. **Nested Group Support:** Untuk LDAP, mendukung recursive group membership lookup

12.6 Langkah Implementasi

1. Buat migrasi tabel idp_group_mappings dan idp_sync_log
2. Implementasikan OIDC client di Hub (menggunakan library golang.org/x/oauth2)
3. Implementasikan LDAP client di Hub (menggunakan library go-ldap/ldap/v3)
4. Buat UI konfigurasi di Web App untuk mengelola group mapping
5. Implementasikan auto-provisioning dan dynamic role sync
6. Tambahkan halaman admin untuk melihat IdP sync log
7. Testing dengan Keycloak sebagai IdP referensi

12.7 Prioritas & Estimasi

Prioritas: Medium-High — sangat membantu saat tim IT besar (10+ admin).

Estimasi Effort: 2–3 sprint (OIDC client + LDAP client + group mapping UI + testing).

Rekomendasi 13: Circuit Breaker pada Plugin Agent

13.1 Latar Belakang

Saat ini setiap plugin agent berjalan di goroutine terpisah dengan panic recovery (Bab 21). Namun, jika sebuah plugin terus gagal mendeteksi service (misalnya MySQL down atau uninstalled), agent akan terus mencoba tanpa henti, membuang CPU server target. Diperlukan mekanisme Circuit Breaker per plugin.

13.2 Arsitektur Circuit Breaker

Library: sony/gobreaker — battle-tested, generics support, digunakan luas di ekosistem Go.

State Machine per Plugin:

State	Behavior	Transisi
CLOSED (Normal)	Plugin beroperasi normal, setiap kegagalan dicatat	Jika 5 kegagalan berturut-turut → OPEN
OPEN (Backoff)	Plugin di-skip, tidak ada upaya deteksi/polling	Setelah timeout (default 60 detik) → HALF-OPEN
HALF-OPEN (Probing)	1 request percobaan diizinkan	Jika sukses → CLOSED. Jika gagal → OPEN (dengan timeout lebih lama)

Exponential Backoff pada OPEN state:

- Pertama kali open: timeout 60 detik
- Kedua kali open: timeout 120 detik
- Ketiga kali: timeout 240 detik
- Maksimum: timeout 15 menit (900 detik)
- Reset ke 60 detik jika pernah CLOSED selama 5 menit

13.3 Integrasi dengan Plugin Manager

Setiap plugin mendapatkan circuit breaker sendiri:

```
type ManagedPlugin struct {
    Plugin       PluginInterface
    CircuitBreaker *gobreaker.CircuitBreaker[*ServiceStatus]
    BackoffLevel int
    LastStateChange time.Time
}
```

Plugin Manager Registry:

```
type PluginManager struct {
    plugins map[string]*ManagedPlugin
    mu      sync.RWMutex
}
```

13.4 Konfigurasi per Plugin

```
agent:
  plugins:
    circuit_breaker:
      enabled: true
```

```
    defaults:
      consecutive_failures: 5     # Trip setelah 5 kegagalan berturut
      initial_timeout: "60s"      # Durasi OPEN pertama
      max_timeout: "15m"          # Durasi OPEN maksimum
      half_open_max_requests: 1   # Request diizinkan saat HALF-OPEN
      reset_after: "5m"           # Reset backoff jika CLOSED > 5 menit
    overrides:
      mysql:
        consecutive_failures: 3   # MySQL lebih sensitif
        initial_timeout: "30s"
      docker:
        consecutive_failures: 10  # Docker lebih toleran
        max_timeout: "30m"
```

13.5 Monitoring & Reporting

Metrics yang dilaporkan ke Hub via telemetry:

Metric	Deskripsi
plugin_cb_state	State saat ini (closed=0, half_open=1, open=2)
plugin_cb_consecutive_failures	Jumlah kegagalan berturut-turut
plugin_cb_total_trips	Total berapa kali circuit di-trip
plugin_cb_last_trip_timestamp	Kapan terakhir circuit di-trip
plugin_cb_backoff_level	Level backoff saat ini (1-N)

Dashboard Alert Rules:

- Jika plugin dalam state OPEN lebih dari 30 menit → Warning alert ke admin
- Jika lebih dari 3 plugin dalam state OPEN di satu agent → Critical alert
- Jika circuit breaker trip rate meningkat tiba-tiba di banyak agent → Possible outage alert

13.6 Interaksi dengan Service Discovery

Saat circuit breaker dalam state OPEN:

- Plugin tidak melakukan service discovery polling
- Status service di Hub ditampilkan sebagai "Unavailable (Circuit Open)" — bukan "Down"
- Hub tahu bahwa agent sengaja menghentikan upaya, bukan karena koneksi terputus
- Saat circuit kembali CLOSED dan service terdeteksi, status otomatis di-update

13.7 Langkah Implementasi

1. Tambahkan dependency sony/gobreaker ke go.mod agent
2. Refactor PluginManager untuk membungkus setiap plugin dengan ManagedPlugin struct
3. Implementasikan circuit breaker wrapper pada method Discover(), Status(), dan Health()

4. Implementasikan exponential backoff logic untuk OPEN state timeout
5. Tambahkan circuit breaker metrics ke telemetry reporting
6. Buat konfigurasi per-plugin override di config YAML
7. Update dashboard Hub untuk menampilkan circuit breaker status per plugin per agent
8. Testing: simulasi service down/up, verifikasi state transitions dan CPU saving

13.8 Prioritas & Estimasi

Prioritas: Medium — dampak terasa saat agent mengelola banyak service heterogen.

Estimasi Effort: 1 sprint (circuit breaker wrapper + monitoring integration + testing).

Lampiran A: Detail Modal Konfigurasi per Service

Lampiran ini mendokumentasikan field dan aksi spesifik yang ditampilkan di modal konfigurasi untuk setiap service yang didukung PRISM.

Catatan v4.0: Untuk service yang memiliki account management (MySQL, PostgreSQL, MongoDB, Valkey, RabbitMQ, MQTT, MinIO, Garage, SeaweedFS, vsftpd, SFTPGo), tab "Accounts" menampilkan Application Accounts (project-based / independent). Management Credentials ditampilkan terpisah di tab "Connection".

A.1 MySQL

Tab	Field / Informasi	Tipe	Keterangan
Overview	Port, Socket, Data Dir, Server ID, Version, Uptime	Read-only (Descriptions)	Info dasar instance
Overview	Buffer Pool Size, Threads, Connections, Max Connections	Read-only (Statistic)	Metrik performa real-time
Connection	Management Credential: root user, password (masked), connection string	Read-only + Edit (otorisasi khusus)	Credential yang digunakan Agent
Accounts	User list: username, host, auth plugin, default schema	Table + CRUD (Drawer)	Manajemen Application Accounts MySQL
Accounts	Privileges: global, database, table level	Transfer/Checkbox dalam Drawer	Grant/Revoke per user
Accounts	Kategorisasi: project / independent	Tag + Select	Assign user ke project
Config	Database list: nama, size, charset, collation, tables	Table + Create/Drop	Inventory database
Config	Replication: role, binlog pos, slave lag	Descriptions (Read-only)	Status replikasi
Config	Backup: last backup, method, size	Descriptions + Button (trigger)	Info + trigger manual backup

Catatan: Tab Connection menampilkan Management Credential (root/admin) yang digunakan Agent untuk mengelola service ini. Credential di-mask dan memerlukan otorisasi khusus untuk melihat.

A.2 PostgreSQL

Tab	Field / Informasi	Tipe	Keterangan
Overview	Port, Data Dir, Cluster Name, Version, Uptime	Read-only	Info dasar
Overview	Shared Buffers, Work Mem, Active Conns, Max Conns, WAL Level	Statistic	Metrik performa

Connection	Management Credential: postgres superuser, password (masked)	Read-only + Edit	Credential Agent
Accounts	Role list: name, superuser, login, conn limit	Table + CRUD	Manajemen Application Accounts (roles)
Accounts	Privileges: DB ownership, schema usage, table perms	Edit (Grant/Revoke)	Permission per role
Config	Database list: name, owner, size, encoding, tablespace	Table + Create/Drop	Inventory DB
Config	Schema list per DB: name, owner, table count	Table + Create/Drop	Inventory schema
Config	Extensions: installed, available	Table + Install/Uninstall	Extension management
Config	Replication: streaming status, slots, subscriber lag	Read-only	Monitoring replikasi

A.3 MongoDB

Tab	Field / Informasi	Tipe	Keterangan
Overview	Port, Data Path, Version, Uptime, Storage Engine	Read-only	Info dasar
Overview	Connections, Memory (resident/virtual)	Statistic	Metrik
Connection	Management Credential: admin user, password (masked)	Read-only + Edit	Credential Agent
Accounts	User list: username, DB scope, roles	Table + CRUD	User management
Accounts	Roles: built-in, custom, privilege actions	Edit	Role management
Config	Database list: name, size, collections, indexes	Table + Create/Drop	Inventory DB
Config	Collection list per DB: name, docs, avg size, indexes	Read-only Table	Detail collection
Config	Replica Set: members, primary/secondary, oplog, lag	Read-only	Status RS

A.4 Valkey

Tab	Field / Informasi	Tipe	Keterangan
Overview	Port, Version, Uptime, Mode (standalone/cluster/sentinel)	Read-only	Info dasar

Overview	Memory Used/Peak, Clients, Keys, Hit Rate	Statistic + Progress	Metrik
Connection	Management Credential: default password (masked)	Read-only + Edit	Credential Agent
Accounts	ACL User list: name, enabled, commands, key patterns	Table + CRUD	ACL management
Config	Persistence: RDB last save, AOF status	Read-only + Button (trigger BGSAVE)	Persistence status
Config	Replication: role, connected replicas, offset	Read-only	Status replikasi
Config	Keyspace: DB index, keys, expires, avg TTL	Read-only Table	Key distribution

A.5 RabbitMQ

Tab	Field / Informasi	Tipe	Keterangan
Overview	Port (AMQP/Management), Version, Uptime, Erlang Version	Read-only	Info dasar
Overview	Queues, Messages, Rate (pub/deliver), Connections, Channels	Statistic	Metrik
Connection	Management Credential: admin user, password (masked)	Read-only + Edit	Credential Agent
Accounts	User list: name, tags, password hash algo	Table + CRUD	User management
Accounts	VHost permissions: configure/write/read regex	Edit per user per vhost	Permission management
Config	Virtual Host list: name, messages, queues, connections	Table + CRUD	VHost management
Config	Queue list per VHost: name, type, messages, consumers, state	Table + Create/Delete/Purge	Queue management
Config	Exchange list: name, type, bindings	Table + Create/Delete	Exchange management
Config	Policies: name, pattern, apply-to, definition	Table + CRUD	Policy management

A.6 MQTT (Mosquitto)

Tab	Field / Informasi	Tipe	Keterangan

Overview	Port (MQTT/WS), Version, Uptime	Read-only	Info dasar
Overview	Clients, Subscriptions, Messages In/Out, Retained	Statistic	Metrik
Connection	Management Credential: admin credentials (masked)	Read-only + Edit	Credential Agent
Accounts	User list: username, enabled/disabled	Table + CRUD	User management
Accounts	Topic ACL: pattern, access type, priority	Edit per user	Topic access control
Config	Listeners: port, protocol, max connections	Read-only	Listener info
Config	Bridge: remote broker, topics, QoS, status	Read-only	Bridge status

A.7 MinIO

Tab	Field / Informasi	Tipe	Keterangan
Overview	API Port, Console Port, Version, Mode, Uptime	Read-only	Info dasar
Overview	Total/Used/Available Storage, Buckets, Objects	Statistic + Progress	Kapasitas
Connection	Management Credential: admin access key + secret (masked)	Read-only + Edit	Credential Agent
Accounts	Access Key list: ID, status, created, last used	Table + CRUD	Key management
Accounts	Policy per key: built-in atau custom JSON	Edit (code editor)	IAM policy
Accounts	Service Accounts: parent, policy, session duration	Table + CRUD	Service account mgmt
Config	Bucket list: name, size, objects, versioning, lock	Table + CRUD	Bucket management
Config	Bucket Policy: public/private/custom	Edit per bucket	Access policy
Config	Lifecycle Rules: expiration, transition, filter	Table + CRUD per bucket	Lifecycle management
Config	Event Notification: target, events, filter	Table + CRUD	Notification config

A.8 Garage

Tab	Field / Informasi	Tipe	Keterangan

Overview	API Port, Admin Port, Version, Cluster Status	Read-only	Info dasar
Overview	Node list: ID, hostname, zone, capacity, status	Table (Read-only)	Cluster nodes
Connection	Management Credential: admin API key (masked)	Read-only + Edit	Credential Agent
Accounts	Key list: ID, name, access key, status	Table + CRUD	API key management
Accounts	Bucket permission per key: read/write/owner	Edit	Permission per bucket
Config	Bucket list: name, size, objects, quotas	Table + CRUD	Bucket management
Config	Cluster Layout: replication factor, zones	Read-only	Layout info

A.9 SeaweedFS

Tab	Field / Informasi	Tipe	Keterangan
Overview	Master/Filer/S3 Port, Version, Topology	Read-only	Info dasar
Overview	Volumes, Free Volumes, Total Size, Volume Servers	Statistic	Kapasitas
Connection	Management Credential: admin credentials (masked)	Read-only + Edit	Credential Agent
Accounts	S3 Access Key: ID, secret (masked), actions	Table + CRUD	S3 key management
Accounts	IAM Policy per key: bucket permissions	Edit	Policy management
Config	S3 Bucket list: name, size, objects	Table + CRUD	Bucket management
Config	Filer Config: collections, replication, TTL	Read-only	Filer info
Config	EC Status: encoded volumes, shards	Read-only	Erasure coding status

A.10 vsftpd

Tab	Field / Informasi	Tipe	Keterangan
Overview	Port (FTP/FTPS), Version, Uptime, Anonymous (on/off)	Read-only	Info dasar
Overview	Active Connections, Sessions Today	Statistic	Metrik
Connection	Management Credential:	Read-only + Edit	Credential Agent

	system-level access (masked)		
Accounts	User list: name, home dir, status, bandwidth limit	Table + CRUD	FTP user management
Accounts	Permissions per user: read/write/mkdir/delete	Edit	Hak akses
Config	TLS: cert path, min version, force encryption	Read-only	Enkripsi
Config	Passive Mode: port range, external IP	Read-only	Passive config
Config	Chroot: jail status, allowed dirs	Read-only	Isolasi directory

A.11 SFTPGo

Tab	Field / Informasi	Tipe	Keterangan
Overview	Port (SFTP/HTTP/FTP/WebDAV), Version, Uptime	Read-only	Info dasar
Overview	Active Connections, Transfers Today, Data Transferred	Statistic	Metrik
Connection	Management Credential: admin API credentials (masked)	Read-only + Edit	Credential Agent
Accounts	User list: name, status, home dir, quota, bandwidth, last login	Table + CRUD	User management
Accounts	Permissions per directory: list/download/upload/delete/rename/etc	Edit (Tree + Checkbox)	Granular permissions
Accounts	Public Keys per user: fingerprint, added date	Table + CRUD	SSH key management
Accounts	Virtual Folders: name, mapped path, quota	Table + CRUD	Folder mapping
Config	Data Provider: backend type, status	Read-only	Backend info
Config	Event Rules: trigger, conditions, actions	Table + CRUD	Event automation
Config	IP Filter: allow/deny list	Table + CRUD	IP access control

A.12 Nginx

Tab	Field / Informasi	Tipe	Keterangan
Overview	Port (HTTP/HTTPS), Version, Uptime, Workers, Config Path	Read-only	Info dasar
Overview	Active Conns, Req/sec,	Statistic	Metrik

	Reading/Writing/Waiting		(stub_status)
Config	Site list: domain, port, root/proxy_pass, SSL, enabled	Table + Enable/Disable	Virtual host management
Config	Upstream per site: backends, LB method, health check	Read-only	Reverse proxy info
Config	SSL per site: issuer, expiry, auto-renew	Read-only + Tag (warning jika <30d)	Certificate monitoring
Config	Config Test: nginx -t output	Button + Result display	Validasi konfigurasi

A.13 Caddy

Tab	Field / Informasi	Tipe	Keterangan
Overview	Port (HTTP/HTTPS/Admin), Version, Uptime	Read-only	Info dasar
Overview	Active Requests, Total Requests	Statistic	Metrik
Config	Site list: domain, upstream/root, TLS mode, status	Table + Edit	Site management
Config	Reverse Proxy: upstream, health check, LB, headers	Edit per site	Proxy config
Config	TLS/Auto HTTPS: ACME, certificates, renewal status	Read-only	HTTPS status
Config	Middleware: encode, rate limit, basic auth, IP filter	Edit per site	Middleware config

A.14 UFW

Tab	Field / Informasi	Tipe	Keterangan
Overview	Status (active/inactive), Default Policy, Logging Level	Read-only + Tag	Firewall status
Config	Rule list: action, direction, protocol, port, from/to, comment	Table + CRUD	Rule management
Config	App Profiles: name, title, ports	Read-only Table	Application profiles
Config	Enable/Disable	Switch (Popconfirm)	Toggle firewall

A.15 firewalld

Tab	Field / Informasi	Tipe	Keterangan
Overview	Status, Default Zone, Log Denied, Panic Mode	Read-only	Firewall status
Config	Zone list: name, target, interfaces, sources	Table + Edit	Zone management
Config	Services per zone	Add/Remove (Transfer)	Service assignment
Config	Port rules per zone: port, protocol, permanent/runtime	Table + CRUD	Port management
Config	Rich Rules: family, source, dest, action, log, limit	Table + CRUD	Rich rule management
Config	Reload (Runtime/Permanent)	Button	Apply changes

A.16 iptables

Tab	Field / Informasi	Tipe	Keterangan
Overview	Active tables, Total rules per table, Kernel module	Read-only	Status dasar
Config	Chain list per table: name, policy, packet/byte counters	Table + Create/Delete (custom)	Chain management
Config	Rules per chain: #, target, proto, src, dst, match, comment	Table + Insert/Append/Delete	Rule management
Config	NAT rules: SNAT, DNAT, MASQUERADE, REDIRECT	Table + CRUD	NAT config
Config	Save/Restore: export, import	Button (download/upload)	Persist rules

A.17 nftables

Tab	Field / Informasi	Tipe	Keterangan
Overview	Active tables, chains, rules count, ruleset size	Read-only	Status dasar
Config	Table list: name, family (ip/ip6/inet/arp/bridge/netdev)	Table + Create/Delete	Table management
Config	Chain list per table: name, type, hook, priority, policy	Table + CRUD	Chain management
Config	Rules per chain: handle, expression, counter, comment	Table + CRUD	Rule management
Config	Sets & Maps: name, type,	Table + CRUD	Set/map

	elements, flags		management
Config	Export/Import ruleset	Button (download/upload)	Ruleset backup

A.18 CrowdSec

Tab	Field / Informasi	Tipe	Keterangan
Overview	Version, Uptime, Local API, Machines, Console Status	Read-only	Info dasar
Overview	Alerts (day/week/month), Active Decisions, Top Attackers	Statistic	Security metrics
Config	Decisions: IP, type (ban/captcha/throttle), duration, reason	Table + Delete (unban)	Decision management
Config	Scenarios: name, status, description, event count	Table + Enable/Disable	Scenario management
Config	Bouncers: name, type, IP, last pull, status	Table + Register/Delete	Bouncer management
Config	Parsers: name, status, stage	Table + Enable/Disable	Parser management
Config	Allow List: IP/range, reason, date	Table + CRUD	Whitelist
Config	Collections: name, author, contents	Table + Install/Remove	Hub collection mgmt

A.19 PM2

Tab	Field / Informasi	Tipe	Keterangan
Overview	PM2 Version, Daemon PID, Node.js Version, PM2 Home Path	Read-only	Info dasar PM2 daemon
Overview	Total Processes, Running, Stopped, Errored, Total Restarts	Statistic	Metrik agregat
Config	Process list: id, name, mode (fork/cluster), instances, status, CPU, memory, uptime, restarts, script path	Table	Daftar seluruh proses
Config	Per process actions: Start, Stop, Restart, Reload (0-downtime), Delete	Button + Popconfirm	Aksi per proses
Config	Ecosystem file: path, contents (editable)	Read + Edit (code editor)	Edit ecosystem.config.js

Config	Environment variables per process: key-value pairs	Table (Read-only)	Env vars proses
Config	Log paths per process: stdout log, stderr log	Read-only + Link (live tail)	Path log
Config	Startup script: status (enabled/disabled), init system	Read-only + Button (generate)	pm2 startup / pm2 save
Config	Flush Logs, Reset Restart Count, Save Process List	Action buttons	Maintenance actions

A.20 Supervisor (supervisord)

Tab	Field / Informasi	Tipe	Keterangan
Overview	Supervisor Version, PID, Config Path, State (RUNNING/FATAL)	Read-only	Info dasar supervisord
Overview	Total Programs, Running, Stopped, Fatal	Statistic	Metrik agregat
Config	Program list: name, state (RUNNING/STOPPED/STARTING/BACKOFF/FATAL), PID, uptime, group	Table	Daftar program
Config	Per program actions: Start, Stop, Restart, Clear Log, Tail Log	Button + Popconfirm	Aksi per program
Config	Program config: command, directory, user, autostart, autorestart, startsecs, startretries, redirect_stderr, stdout_logfile, environment	Descriptions (Read-only)	Konfigurasi dari .conf file
Config	Process groups: group name, member programs, priority	Collapse + Table	Manajemen process groups
Config	Reload Config (supervisorctl reread + update)	Action button	Menerapkan perubahan config

A.21 systemd User Services

Khusus untuk systemd user services: PRISM hanya mengelola unit yang berjalan di scope user (systemctl --user), bukan system-wide.

Tab	Field / Informasi	Tipe	Keterangan
Overview	User context: username, UID, lingering status, user unit dir path	Read-only	Info dasar user service context
Overview	Total Units, Active, Inactive, Failed	Statistic	Metrik agregat per user
Config	Unit list: unit name, load state, active state, sub state,	Table	Daftar systemd user units

	description		
Config	Per unit actions: Start, Stop, Restart, Enable, Disable, Mask, Unmask	Button + Popconfirm	Aksi per unit
Config	Unit file content: [Unit], [Service], [Install] sections	Descriptions atau Read-only code view	Isi dari unit file
Config	Unit properties: ExecStart, WorkingDirectory, Environment, Restart policy, resource limits	Descriptions (Read-only)	Properties penting
Config	Journal log per unit: journalctl --user-unit=<name>	Live tail (monospace)	Log output via journald
Config	Daemon Reload (systemctl --user daemon-reload)	Action button	Reload setelah edit unit file
Config	Enable Lingering	Switch + Popconfirm	loginctl enable-linger agar services tetap berjalan

A.22 Docker

Tab	Field / Informasi	Tipe	Keterangan
Overview	Docker Version, API Version, Storage Driver, Cgroup Driver, Runtime, Root Dir, Server OS	Read-only	Info dasar Docker daemon
Overview	Total Containers (running/paused/stopped), Total Images, Total Volumes, Total Networks	Statistic	Metrik agregat
Overview	Disk Usage: images size, containers size, volumes size, build cache size	Statistic + Progress	Penggunaan disk Docker
Config	Container list: name, image, status, ports, created, networks, mounts, command	Table (filterable, sortable)	Daftar seluruh container
Config	Per container actions: Start, Stop, Restart, Pause, Unpause, Remove, View Logs	Button + Popconfirm	Aksi per container
Config	Container inspect: env vars, labels, resource limits (CPU, memory), restart policy, health check	Descriptions / Collapse	Detail konfigurasi container

Config	Container stats (live): CPU %, Memory usage/limit, Network I/O, Block I/O, PIDs	Table (auto-refresh)	Real-time monitoring
Config	Image list: repository, tag, image ID, size, created	Table + Remove (prune)	Manajemen images
Config	Volume list: name, driver, mountpoint, size, used by containers	Table + Create/Remove	Manajemen volumes
Config	Network list: name, driver, subnet, gateway, connected containers	Table + Create/Remove	Manajemen networks
Config	Docker Compose projects: project name, services, status, compose file path	Collapse + Table	Daftar compose project
Config	Compose actions: Up, Down, Restart, Pull, View Config	Button + Popconfirm	Aksi level compose project
Config	System Prune: remove unused containers, images, networks, volumes	Button (Popconfirm, destructive)	Cleanup disk space
Logs	Container log stream: per container, follow mode, tail lines, since/until filter	Live tail (monospace) + Select	Log viewer per container

Docker Compose Integration: Jika agent mendeteksi adanya docker-compose.yml atau compose.yaml, PRISM secara otomatis mengelompokkan container yang terkait sebagai satu Compose project. Admin dapat melakukan operasi level project (up/down/restart) selain operasi level individual container.

A.23 Podman

Podman memiliki API yang kompatibel dengan Docker, sehingga modal konfigurasinya sangat mirip. Perbedaan utama: Podman bersifat rootless by default dan mendukung pod sebagai unit pengelompokan container.

Tab	Field / Informasi	Tipe	Keterangan
Overview	Podman Version, API Version, Runtime (crun/runc), Storage Driver, Graph Root, Run Root	Read-only	Info dasar Podman
Overview	Total Containers (running/paused/stopped), Total Images, Total Volumes, Total Pods	Statistic	Metrik agregat
Overview	Rootless mode: active/inactive, user context	Tag + Descriptions	Informasi privilege mode
Config	Container list: name, image,	Table (filterable,	Daftar container

	status, ports, pod (jika ada), created, networks, mounts	sortable)	
Config	Per container actions: Start, Stop, Restart, Pause, Remove, View Logs	Button + Popconfirm	Aksi per container
Config	Container inspect: env vars, labels, resource limits, restart policy, health check, security options	Descriptions / Collapse	Detail container (termasuk Podman-specific)
Config	Container stats (live): CPU %, Memory, Network I/O, Block I/O	Table (auto-refresh)	Real-time monitoring
Config	Pod list: name, status, infra container ID, container count, created	Table + Create/Remove	Manajemen pods
Config	Per pod actions: Start, Stop, Restart, Remove, View Containers in Pod	Button + Popconfirm	Aksi level pod
Config	Image list: repository, tag, ID, size, created	Table + Pull/Remove	Manajemen images
Config	Volume list: name, driver, mountpoint, size	Table + Create/Remove	Manajemen volumes
Config	Network list: name, driver, subnet, connected containers	Table + Create/Remove	Manajemen networks
Config	Podman Compose / Quadlet: compose file atau systemd-based container management	Collapse + Table	Jika menggunakan podman-compose atau Quadlet
Config	System Prune	Button (Popconfirm)	Cleanup unused resources
Logs	Container/Pod log stream	Live tail + Select picker	Log viewer per container atau per pod

Lampiran B: Skema Database Lengkap

Lampiran ini menampilkan skema database lengkap PRISM v4.1, termasuk tabel-tabel baru dari rekomendasi v4.1.

B.1 Tabel servers

Kolom	Tipe	Constraint	Keterangan
id	TEXT	PRIMARY KEY (UUID)	Identifier unik server
name	TEXT	NOT NULL	Nama server (ditetapkan saat approval)
description	TEXT		Deskripsi server
hostname	TEXT	NOT NULL	Hostname asli dari OS
ip_address	TEXT	NOT NULL	IP address server
os	TEXT		Sistem operasi (e.g., Ubuntu 24.04 LTS)
status	TEXT	NOT NULL	pending / active / unreachable / removed
agent_version	TEXT		Versi Prism Agent yang berjalan
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp pembuatan
updated_at	DATETIME		Timestamp pembaruan terakhir

B.2 Tabel services

Kolom	Tipe	Constraint	Keterangan
id	TEXT	PRIMARY KEY (UUID)	Identifier unik service
server_id	TEXT	FOREIGN KEY → servers(id)	Server tempat service berjalan
name	TEXT	NOT NULL	Nama service (e.g., mysql, nginx)
type	TEXT	NOT NULL	Tipe service
version	TEXT		Versi service
port	INTEGER		Port utama service
status	TEXT	NOT NULL	running / stopped / error
config_path	TEXT		Path konfigurasi utama
pid	INTEGER		Process ID
discovered_at	DATETIME		Timestamp pertama kali ditemukan

updated_at	DATETIME		Timestamp pembaruan terakhir

B.3 Tabel management_credentials

Tabel baru di v4.0 yang menyimpan credential manajemen (root/admin/default) secara terpisah dari Application Accounts.

Kolom	Tipe	Constraint	Keterangan
id	TEXT	PRIMARY KEY (UUID)	Identifier unik credential
service_id	TEXT	FOREIGN KEY → services(id)	Service yang di-manage
credential_type	TEXT	NOT NULL	root / admin / default
username_encrypted	BLOB	NOT NULL	Username terenkripsi (AES-256-GCM)
password_encrypted	BLOB	NOT NULL	Password terenkripsi (AES-256-GCM)
connection_params_encrypted	BLOB		Parameter koneksi terenkripsi
last_verified_at	DATETIME		Terakhir kali credential diverifikasi
status	TEXT	NOT NULL DEFAULT 'active'	active / inactive / error
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp pembuatan
updated_at	DATETIME		Timestamp pembaruan terakhir

B.4 Tabel accounts (Diperbarui)

Tabel accounts kini hanya menyimpan Application Accounts — credential manajemen disimpan di tabel management_credentials.

Kolom	Tipe	Constraint	Keterangan
id	TEXT	PRIMARY KEY (UUID)	Identifier unik akun
service_id	TEXT	FOREIGN KEY → services(id)	Service tempat akun dibuat

server_id	TEXT	FOREIGN KEY → servers(id)	Server tempat akun berada
username	TEXT	NOT NULL	Username akun
account_type	TEXT	NOT NULL	user / service
category	TEXT	NOT NULL	project / independent
project_name	TEXT		Nama project (jika project-based)
permissions	TEXT (JSON)		Detail permissions dalam format JSON
status	TEXT	NOT NULL DEFAULT 'active'	active / disabled
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp pembuatan

B.5 Tabel agent_certificates

Kolom	Tipe	Constraint	Keterangan
id	TEXT	PRIMARY KEY (UUID)	Identifier unik sertifikat
server_id	TEXT	FOREIGN KEY → servers(id)	Server pemilik sertifikat
fingerprint	TEXT	UNIQUE, NOT NULL	SHA-256 fingerprint sertifikat
issued_at	DATETIME	NOT NULL	Tanggal penerbitan
expires_at	DATETIME	NOT NULL	Tanggal kedaluwarsa
revoked_at	DATETIME		Tanggal revokasi (jika di-revoke)
revocation_reason	TEXT		Alasan revokasi

B.6 Tabel enrollment_keys

Kolom	Tipe	Constraint	Keterangan
id	TEXT	PRIMARY KEY (UUID)	Identifier unik key
key_hash	TEXT	UNIQUE, NOT NULL	Hash dari enrollment key
created_by	TEXT		User yang membuat key
expires_at	DATETIME	NOT NULL	Waktu kedaluwarsa
used_at	DATETIME		Waktu digunakan
used_by_server_id	TEXT		Server yang

			menggunakan key ini

B.7 Tabel commands

Kolom	Tipe	Constraint	Keterangan
id	TEXT	PRIMARY KEY (UUID)	Identifier unik command
server_id	TEXT	FOREIGN KEY → servers(id)	Target server
service_id	TEXT	FOREIGN KEY → services(id)	Target service
type	TEXT	NOT NULL	Tipe command
payload	TEXT (JSON)		Payload command
status	TEXT	NOT NULL	pending / sent / executed / failed
issued_by	TEXT		User yang mengeluarkan command
issued_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Waktu command dibuat
executed_at	DATETIME		Waktu command dieksekusi
result	TEXT (JSON)		Hasil eksekusi

B.8 Tabel telemetry

Kolom	Tipe	Constraint	Keterangan
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Identifier unik (auto)
server_id	TEXT	FOREIGN KEY → servers(id), NOT NULL	Server sumber data
service_id	TEXT	FOREIGN KEY → services(id)	Service sumber data
metrics	TEXT (JSON)	NOT NULL	Data metrik dalam format JSON
collected_at	DATETIME	NOT NULL	Waktu data dikumpulkan oleh agent

Index: CREATE INDEX idx_telemetry_server_time ON telemetry(server_id, collected_at);

Index: CREATE INDEX idx_telemetry_service ON telemetry(service_id) WHERE service_id IS NOT NULL;

B.9 Tabel audit_log

Kolom	Tipe	Constraint	Keterangan
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Identifier unik (auto)
user_id	TEXT		User yang melakukan aksi
action	TEXT	NOT NULL	Jenis aksi yang dilakukan
resource_type	TEXT		Tipe resource yang terpengaruh
resource_id	TEXT		ID resource yang terpengaruh
details	TEXT (JSON)		Detail tambahan dalam format JSON
ip_address	TEXT		IP address pelaku
timestamp	DATETIME	DEFAULT CURRENT_TIMESTAMP	Waktu aksi

B.10 Tabel users

Kolom	Tipe	Constraint	Keterangan
id	TEXT	PRIMARY KEY (UUID)	Identifier unik user
username	TEXT	UNIQUE, NOT NULL	Username login
email	TEXT	UNIQUE, NOT NULL	Email user
password_hash	TEXT	NOT NULL	Hash password (bcrypt/argon2)
role	TEXT	NOT NULL	admin / operator / viewer
mfa_enabled	BOOLEAN	DEFAULT FALSE	Status MFA
last_login	DATETIME		Waktu login terakhir
status	TEXT	NOT NULL DEFAULT 'active'	active / disabled / locked

Catatan: Tabel users menyimpan user internal PRISM (administrator, operator, viewer) — bukan service accounts yang dikelola melalui tabel accounts.

B.11 Tabel permissions (BARU — Rekomendasi #1: RBAC Granular)

Kolom	Tipe	Constraint	Keterangan
id	INTEGER	PRIMARY KEY	Auto-increment
name	TEXT	UNIQUE, NOT NULL	Format resource:action

			(e.g., server:read)
description	TEXT		Deskripsi permission
category	TEXT		Kategori grouping (Server, Service, User, dll.)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp pembuatan

B.12 Tabel role_permissions (BARU — Rekomendasi #1: RBAC Granular)

Kolom	Tipe	Constraint	Keterangan
id	INTEGER	PRIMARY KEY	Auto-increment
role_id	INTEGER	FOREIGN KEY → roles(id), NOT NULL	Referensi ke roles
permission_id	INTEGER	FOREIGN KEY → permissions(id), NOT NULL	Referensi ke permissions

B.13 Tabel user_server_access (BARU — Rekomendasi #1: RBAC Granular)

Kolom	Tipe	Constraint	Keterangan
id	INTEGER	PRIMARY KEY	Auto-increment
user_id	INTEGER	FOREIGN KEY → users(id), NOT NULL	Referensi ke users
server_group_id	INTEGER	FOREIGN KEY → server_groups(id), NOT NULL	Referensi ke server_groups
permission_id	INTEGER	FOREIGN KEY → permissions(id), NOT NULL	Permission spesifik untuk resource
granted_by	INTEGER	FOREIGN KEY → users(id)	User yang memberikan akses
granted_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp pemberian akses

B.14 Tabel server_groups (BARU — Rekomendasi #1: RBAC Granular)

Kolom	Tipe	Constraint	Keterangan
id	INTEGER	PRIMARY KEY	Auto-increment

name	TEXT	UNIQUE, NOT NULL	Nama grup (misal: Production-Web)
description	TEXT		Deskripsi grup server
tags	TEXT (JSON)		Tags untuk flexible grouping

B.15 Tabel webhook_subscriptions (BARU — Rekomendasi #4: Webhook)

Kolom	Tipe	Constraint	Keterangan
id	INTEGER	PRIMARY KEY	Auto-increment
name	TEXT	NOT NULL	Nama deskriptif subscription
url	TEXT	NOT NULL	Target endpoint URL (HTTPS)
secret	TEXT	NOT NULL	HMAC-SHA256 signing secret
events	TEXT (JSON)	NOT NULL	Array event types yang di-subscribe
is_active	BOOLEAN	DEFAULT TRUE	Enable/disable tanpa delete
failure_count	INTEGER	DEFAULT 0	Consecutive failure counter
max_retries	INTEGER	DEFAULT 5	Jumlah maksimal retry
created_by	INTEGER	FOREIGN KEY → users(id)	User yang membuat
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp pembuatan

B.16 Tabel webhook_deliveries (BARU — Rekomendasi #4: Webhook)

Kolom	Tipe	Constraint	Keterangan
id	INTEGER	PRIMARY KEY	Auto-increment
subscription_id	INTEGER	FOREIGN KEY → webhook_subscriptions(id), NOT NULL	Referensi ke subscriptions
event_type	TEXT	NOT NULL	Tipe event yang dikirim
payload	TEXT (JSON)	NOT NULL	Request body yang dikirim

response_status	INTEGER		HTTP status code respons
response_body	TEXT		Response body (truncated ke 1KB)
attempt	INTEGER	DEFAULT 1	Percobaan ke-n
delivered_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp delivery attempt
status	TEXT	NOT NULL	success / failed / pending

B.17 Tabel configuration_snapshots (BARU — Rekomendasi #5: Drift Detection)

Kolom	Tipe	Constraint	Keterangan
id	INTEGER	PRIMARY KEY	Auto-increment
server_id	INTEGER	FOREIGN KEY → servers(id), NOT NULL	Server yang di-snapshot
service_name	TEXT	NOT NULL	Nama service (nginx, postgresql, dll)
config_hash	TEXT	NOT NULL	SHA-256 hash dari konfigurasi
config_data	TEXT (JSON)	NOT NULL	Konfigurasi aktual terstruktur
captured_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp pengambilan snapshot

B.18 Tabel drift_events (BARU — Rekomendasi #5: Drift Detection)

Kolom	Tipe	Constraint	Keterangan
id	INTEGER	PRIMARY KEY	Auto-increment
server_id	INTEGER	FOREIGN KEY → servers(id), NOT NULL	Server yang mengalami drift
service_name	TEXT	NOT NULL	Service yang drift
severity	TEXT	NOT NULL	critical / warning / info
drift_details	TEXT (JSON)	NOT NULL	Detail perbedaan (key, expected, actual)
status	TEXT	NOT NULL	detected / acknowledged / remediated /

			ignored
detected_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp deteksi
resolved_at	DATETIME		Timestamp resolusi (nullable)
resolved_by	INTEGER	FOREIGN KEY → users(id)	User yang menyelesaikan (nullable)

B.19 Tabel retention_policies (BARU — Rekomendasi #6: Data Retention)

Kolom	Tipe	Constraint	Keterangan
id	INTEGER	PRIMARY KEY	Auto-increment
data_type	TEXT	NOT NULL	telemetry / audit / command / webhook / snapshot
hot_days	INTEGER	NOT NULL	Hari penyimpanan resolusi penuh
warm_days	INTEGER	NOT NULL DEFAULT 0	Hari penyimpanan downsampled (0 = skip)
cold_days	INTEGER	NOT NULL DEFAULT 0	Hari penyimpanan archived (0 = skip)
is_global	BOOLEAN	DEFAULT TRUE	Berlaku global atau per-server
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Terakhir diubah
updated_by	INTEGER	FOREIGN KEY → users(id)	User yang mengubah